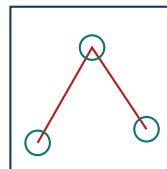
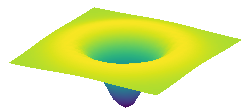

Deep Learning for Computer Vision

Week 2 — Complete Study Notes

Edge Detection • Blobs & Corners • Scale Space, Pyramids & Filter Banks
SIFT & Variants • Image Segmentation • Other Feature Spaces • Human Visual System



Based on the IITM / NPTEL course by Prof. Vineeth N. Balasubramanian (IIT Hyderabad)

Lectures §2.1 – §2.7

How to use these notes. Each module mirrors one lecture. Figures taken directly from the lecture slides are marked (*slide*) in the caption with the slide's original credit; all other diagrams are drawn for these notes. Coloured boxes mark intuition, key ideas, derivations, exam tips and pitfalls. Dimensions are annotated in teal brackets. Worked problems, a self-test MCQ bank and a formula sheet are at the end.

Contents

2.1 Edge Detection	4
2.1.1 What is Edge Detection?	4
2.1.1.1 How are edges caused?	4
2.1.1.2 Looking more locally	5
2.1.1.3 Why are edges important?	5
2.1.2 Edges in Images as Functions	5
2.1.3 Derivatives and Edges	5
2.1.4 Derivatives with Convolution	6
2.1.4.1 Sobel and other finite-difference filters	7
2.1.5 Image Gradients	8
2.1.6 The Effect of Noise	9
2.1.6.1 Derivative theorem of convolution	9
2.1.7 What about the Second Derivative?	10
2.1.8 Computing Gradients in Practice	12
2.1.9 Properties of a Good Edge Detector	12
2.1.10 Non-Maximum Suppression (NMS)	12
2.1.11 Hysteresis Thresholding	13
2.1.12 The Canny Edge Detector (1986)	14
2.1.12.1 Effect of σ	15
2.1.13 More Recent Methods in Edge Detection	15
2.1.13.1 Pb: learning boundaries from brightness, colour and texture (Martin, Fowlkes & Malik, 2004)	15
2.1.13.2 Structured Forests for fast edge detection (Dollár & Zitnick, 2013)	16
2.1.13.3 Crisp boundaries via Pointwise Mutual Information (Isola et al., 2014)	17
2.1.13.4 Holistically-Nested Edge Detection, HED (Xie & Tu, 2015)	17
2.1.14 Homework	18
2.2 From Edges to Blobs and Corners	19
2.2.1 Review: From Canny Edges to Straight Lines	19
2.2.2 Going Further: the Laplacian of Gaussian	19
2.2.2.1 Discrete Laplacian: how is the 3×3 filter obtained?	20
2.2.3 LoG as a Blob Detector	21
2.2.4 From Blobs to Corners	21
2.2.4.1 Shifting a small window	22
2.2.5 Autocorrelation	22
2.2.6 Classifying Points by the Eigenvalues	24
2.2.7 Harris Corner Detector	24
2.2.7.1 Harris in action	25
2.2.8 Harris Detector Variants	26
2.2.9 Properties of the Harris Detector	26
2.2.9.1 Rotation invariance — yes	26
2.2.9.2 Scale invariance — no	27
2.2.9.3 Photometric change: partially invariant to affine intensity change	27
2.2.10 Homework	27

2.3 Scale Space, Image Pyramids and Filter Banks	29
2.3.1 Review: What Harris Gives Us	29
2.3.2 Scale-Invariant Interest Point Detection	29
2.3.3 Automatic Scale Selection	30
2.3.3.1 Implementation with a pyramid	31
2.3.4 Gaussian and Laplacian Pyramids	31
2.3.5 Uses of Image Pyramids	32
2.3.6 Texture in Images	33
2.3.7 Filter Banks	33
2.3.8 Gabor Filters	34
2.3.8.1 Gabor filter banks	35
2.3.9 Steerable Filters	35
2.3.10 Homework Questions (with answers)	36
2.4 Feature Detectors: SIFT and Variants	37
2.4.1 SIFT: Scale-Invariant Feature Transform	37
2.4.2 Step 1: Scale-Space Extrema Detection	38
2.4.2.1 Constructing the scale space	38
2.4.2.2 Difference of Gaussians (DoG)	38
2.4.2.3 Finding extrema	39
2.4.3 Step 2: Keypoint Localisation	40
2.4.3.1 Low-contrast elimination	40
2.4.3.2 Edge elimination	41
2.4.3.3 The effect of each filter	41
2.4.4 Step 3: Orientation Estimation	41
2.4.5 Step 4: Keypoint Descriptor	42
2.4.5.1 Normalisation	43
2.4.6 SIFT in Practice	43
2.4.6.1 Application: image stitching	44
2.4.7 SURF: Speeded-Up Robust Features	45
2.4.8 MOPS: Multi-Scale Oriented PatcheS	45
2.4.9 GLOH: Gradient Location-Orientation Histogram	46
2.4.10 Homework Questions (with answers)	46
2.5 Image Segmentation	48
2.5.1 Human Vision: Gestalt and Grouping	48
2.5.1.1 Gestalt grouping factors	48
2.5.2 Watershed Segmentation (1979)	48
2.5.3 Region Splitting and Merging	50
2.5.4 Graph-Based Segmentation (Felzenszwalb & Huttenlocher, 2004)	50
2.5.5 Probabilistic Aggregation (Alpert et al., 2007)	52
2.5.6 Mean-Shift Segmentation	52
2.5.6.1 Kernel density estimate	53
2.5.6.2 Practical notes	54
2.5.7 Normalised Cuts for Segmentation	54
2.5.8 Other Methods	55
2.5.9 Do We Still Need These?	56
2.5.10 Beyond Images: Segmentation of Video	56

2.6 Other Feature Spaces	57
2.6.1 Shape Context (Belongie, Malik & Puzicha, 2002)	57
2.6.1.1 Matching with shape contexts	58
2.6.2 MSER: Maximally Stable Extremal Regions (Matas et al., 2004)	58
2.6.2.1 Methodology and notation	59
2.6.3 Histogram of Oriented Gradients (HOG) (Dalal & Triggs, 2005)	60
2.6.4 Pyramidal HOG (PHOG) (Bosch, Zisserman & Munoz, 2007)	60
2.6.5 Local Binary Patterns (LBP) (Ojala et al.)	61
2.6.6 Comparison of Feature Detectors	62
2.6.7 ...and Along Came Deep Learning	63
2.7 The Human Visual System	64
2.7.1 The Visual Pathway	64
2.7.2 Light Visible to the Human Eye	65
2.7.3 The Retina	65
2.7.3.1 Photoreceptors	65
2.7.4 Image Filtering in Space and Time in the Retina	66
2.7.4.1 Your retinal filters at work	67
2.7.5 Colour-Opponent Processing	67
2.7.6 The Lateral Geniculate Nucleus (LGN)	68
2.7.7 Primary Visual Cortex (V1)	68
2.7.7.1 A tale of two receptive fields	68
2.7.7.2 Cortical cells compute derivatives	69
2.7.7.3 Oriented filters and natural images	70
2.7.8 Dorsal and Ventral Pathways	71
2.7.8.1 Ventral — the “what” (object) pathway	71
2.7.8.2 Dorsal — the “where” pathway	71
2.7.9 The Visual Processing Hierarchy	72
A Numerical Problems with Worked Solutions	73
A.1 Module 2.1: Edges	73
A.2 Module 2.2: Corners, Blobs and Lines	74
A.3 Module 2.3: Scale Space, Pyramids, Filters	75
A.4 Module 2.4: SIFT	77
A.5 Module 2.5: Segmentation	78
A.6 Module 2.6: Other Feature Spaces	79
A.7 Module 2.7: HVS	80
B Multiple-Choice Self-Test	81
B.1 Module 2.1: Edge Detection	81
B.2 Module 2.2: Blobs and Corners	82
B.3 Module 2.3: Scale Space and Filter Banks	83
B.4 Module 2.4: SIFT and Variants	85
B.5 Module 2.5: Segmentation	86
B.6 Module 2.6: Other Feature Spaces	87
B.7 Module 2.7: Human Visual System	88
C Formula Sheet	89

Edge Detection

2.1.1 What is Edge Detection?

Key Idea: Goal

Map an image from a 2D matrix of pixels to a set of **curves, line segments or contours** $\Rightarrow$ a **more compact representation** than pixels.

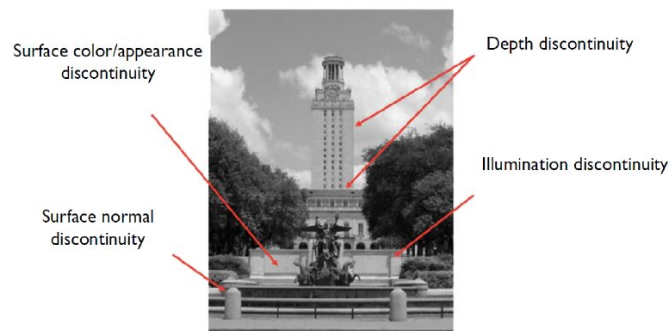
Key idea: look for **strong gradients**, then **post-process** (thin, link, threshold).

Intuition: Why a line drawing is enough

A cartoonist can make you recognise a face with a dozen strokes. Most of the “information” in an image that matters for recognition and geometry sits at places where intensity changes abruptly. Edges keep that information while discarding the (redundant) smooth interiors — a huge compression: a 512×512 image has 262k pixels but maybe a few thousand edge pixels.

2.1.1.1 How are edges caused?

- Variety of factors:



Source: R Urtasun

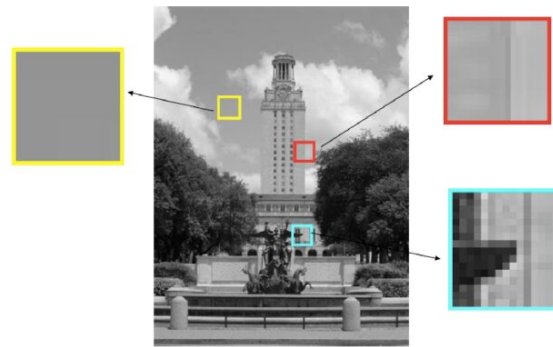
Figure 2.1.1: (slide) Edges arise from four kinds of discontinuity in the scene. Source: R. Urtasun.

Surface normal discontinuity	The surface bends sharply (a corner of a building, the edge of a step) — Lambert’s $\cos \theta$ changes abruptly.
Depth discontinuity	One surface occludes another at a different depth (tower against the sky).
Surface colour / appearance discontinuity	Reflectance (albedo, paint, texture) changes on a single smooth surface (markings).
Illumination discontinuity	Shadows and highlights — the <i>light</i> , not the object, changes.

Pitfall / Careful

Only the first three are properties of the *objects*. Shadow edges are real intensity edges but not object boundaries — one reason “edge” $\neq$ “object boundary” and why learned boundary detectors (§2.1.13) outperform pure gradient methods.

2.1.1.2 Looking more locally



Source: K Grauman, R Urtasun

Figure 2.1.2: (slide) Zooming in: a uniform patch (no edge), a patch across a step (strong edge) and a patch with a thin line. Locally, an edge is just a region where brightness changes quickly in one direction. Source: K. Grauman, R. Urtasun.

2.1.1.3 Why are edges important?

- **Group pixels into objects or parts.**
- Allow us to **track important features** (corners, curves, lines).
- **Cues for 3D shape** (occluding contours, creases).
- Guiding **interactive image editing** (e.g. “intelligent scissors”, edge-aware retouching).

2.1.2 Edges in Images as Functions

Viewing the image as a height map $z = f(x, y)$, edges look like **steep cliffs**.

- Edges look like steep cliffs



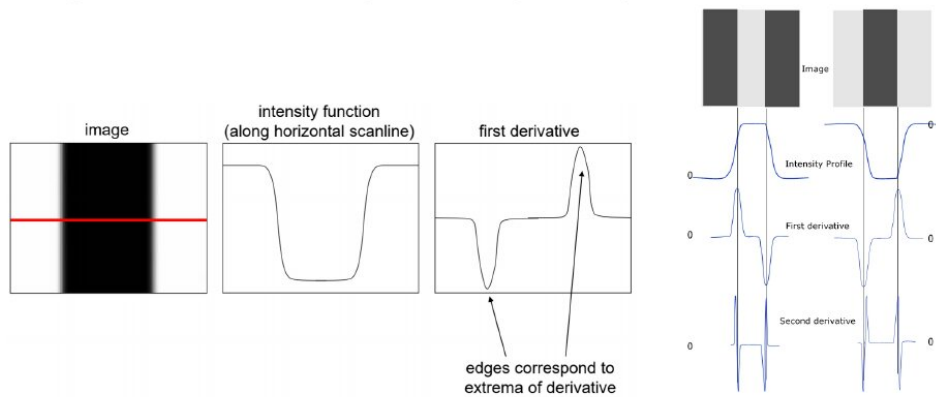
Source: N Snavely, R Urtasun

Figure 2.1.3: (slide) An image and its intensity surface: edges are cliffs. Source: N. Snavely, R. Urtasun.

2.1.3 Derivatives and Edges

An edge is a place of rapid change in the image intensity function. Along a scanline, rapid change = large first derivative.

- An edge is a place of rapid change in the image intensity function



Sources: L. Lazebnik, K. Grauman and <https://mipav.cit.nih.gov/>

Figure 2.1.4: (slide) Along a horizontal scanline, edges correspond to *extrema of the first derivative* (and zero-crossings of the second). Sources: L. Lazebnik, K. Grauman, mipav.cit.nih.gov.

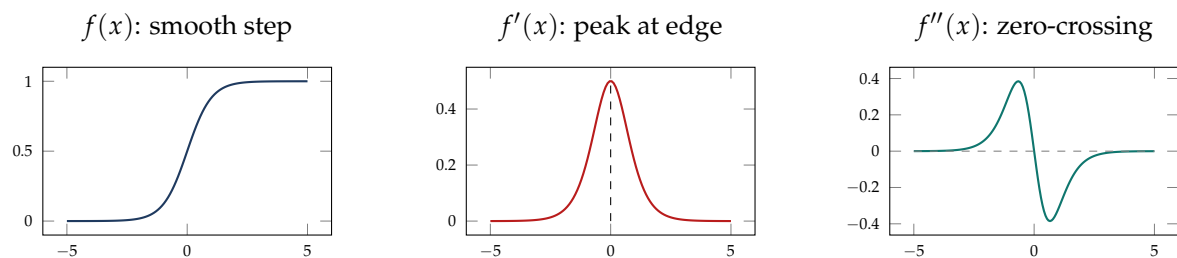


Figure 2.1.5: The two classical edge criteria: a **maximum of $|f'|$** or a **zero-crossing of f''** (with large slope around it).

2.1.4 Derivatives with Convolution

For a 2D function $f(x, y)$ the partial derivative is

$$\frac{\partial f(x, y)}{\partial x} = \lim_{\varepsilon \rightarrow 0} \frac{f(x + \varepsilon, y) - f(x, y)}{\varepsilon}.$$

For discrete data, approximate with **finite differences** ($\varepsilon = 1$ pixel):

$$\frac{\partial f(x, y)}{\partial x} \approx \frac{f(x + 1, y) - f(x, y)}{1}.$$

To implement this as a convolution, what is the filter? The slide shows $\begin{bmatrix} -1 & 1 \end{bmatrix}$ for $\partial/\partial x$ and $\begin{bmatrix} -1 \\ 1 \end{bmatrix}$ (or its flip) for $\partial/\partial y$.

- For 2D function, $f(x, y)$, the partial derivative is:

$$\frac{\partial f(x, y)}{\partial x} = \lim_{\varepsilon \rightarrow 0} \frac{f(x + \varepsilon, y) - f(x, y)}{\varepsilon}$$

- For discrete data, we can approximate using finite differences:

$$\frac{\partial f(x, y)}{\partial x} \approx \frac{f(x + 1, y) - f(x, y)}{1}$$

- To implement above as convolution, what would be the associated filter?

Source: K Grauman



Figure 2.1.6: (slide) Horizontal and vertical finite-difference filters applied to an image: $\partial f / \partial x$ highlights vertical structures, $\partial f / \partial y$ horizontal ones. Source: K. Grauman.

Derivation: Which way round? Correlation vs. convolution

We want $g(x) = f(x + 1) - f(x)$.

- As **cross-correlation** $g(x) = \sum_u H(u)f(x + u)$ with $u \in \{0, 1\}$: $H(0) = -1$, $H(1) = 1$, i.e. $H = [-1, 1]$ (read left to right).
- As **convolution** $g(x) = \sum_u H(u)f(x - u)$ with $u \in \{-1, 0\}$: $H(-1) = 1$, $H(0) = -1$, i.e. the kernel stored as $[1, -1]$ — the flip.

Both compute the same derivative; libraries that call correlation “convolution” use $[-1, 1]$. The **central difference** $\frac{f(x+1) - f(x-1)}{2} \leftrightarrow \frac{1}{2}[-1, 0, 1]$ is symmetric about the pixel and more accurate: its Taylor error is $O(h^2)$ vs. $O(h)$.

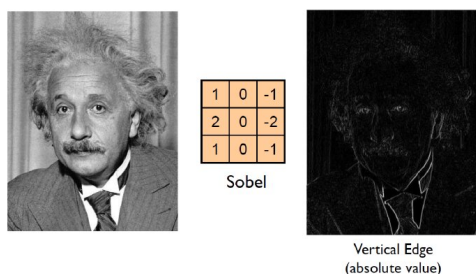
Derivation: Accuracy of forward vs. central differences

Taylor: $f(x \pm h) = f \pm hf' + \frac{h^2}{2}f'' \pm \frac{h^3}{6}f''' + \dots$

Forward: $\frac{f(x+h) - f(x)}{h} = f' + \frac{h}{2}f'' + O(h^2) \Rightarrow \text{error } O(h)$.

Central: $\frac{f(x+h) - f(x-h)}{2h} = f' + \frac{h^2}{6}f''' + O(h^4) \Rightarrow \text{error } O(h^2)$ (the even terms cancel).

2.1.4.1 Sobel and other finite-difference filters



Source: J Hays

(slide) Sobel M_x : vertical edges (absolute value).
Source: J. Hays.

	M_x	M_y																		
Prewitt	<table><tr><td>-1</td><td>0</td><td>1</td></tr><tr><td>-1</td><td>0</td><td>1</td></tr><tr><td>-1</td><td>0</td><td>1</td></tr></table>	-1	0	1	-1	0	1	-1	0	1	<table><tr><td>1</td><td>1</td><td>1</td></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>-1</td><td>-1</td><td>-1</td></tr></table>	1	1	1	0	0	0	-1	-1	-1
-1	0	1																		
-1	0	1																		
-1	0	1																		
1	1	1																		
0	0	0																		
-1	-1	-1																		
Sobel	<table><tr><td>-1</td><td>0</td><td>1</td></tr><tr><td>-2</td><td>0</td><td>2</td></tr><tr><td>-1</td><td>0</td><td>1</td></tr></table>	-1	0	1	-2	0	2	-1	0	1	<table><tr><td>1</td><td>2</td><td>1</td></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>-1</td><td>-2</td><td>-1</td></tr></table>	1	2	1	0	0	0	-1	-2	-1
-1	0	1																		
-2	0	2																		
-1	0	1																		
1	2	1																		
0	0	0																		
-1	-2	-1																		
Roberts	<table><tr><td>0</td><td>1</td></tr><tr><td>-1</td><td>0</td></tr></table>	0	1	-1	0	<table><tr><td>1</td><td>0</td></tr><tr><td>0</td><td>-1</td></tr></table>	1	0	0	-1										
0	1																			
-1	0																			
1	0																			
0	-1																			

Source: R Urtasun

(slide) Prewitt, Sobel and Roberts kernels. Source: R. Urtasun.

	M_x (responds to vertical edges)	M_y (responds to horizontal edges)
Prewitt	$\begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$	$\begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix}$
Sobel	$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$	$\begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$
Roberts	$\begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}$	$\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$

Intuition: Reading the kernels

Prewitt = central difference in $x \times$ box-average in y : $[1, 1, 1]^\top [-1, 0, 1]$. Sobel = central difference $\times$ *binomial* smoothing $[1, 2, 1]^\top [-1, 0, 1]$ — the centre row counts double, a cheap Gaussian. Both are **separable** (Week 1) and **sum to zero**. Roberts is a 2×2 *diagonal* difference: tiny and fast but very noise-sensitive. (Up to sign convention and a normalising factor $\frac{1}{3}$, $\frac{1}{4}$ or $\frac{1}{8}$.)

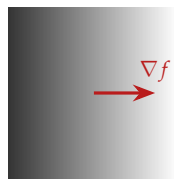
2.1.5 Image Gradients

Key Idea: The gradient

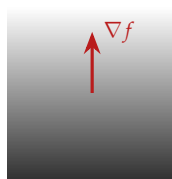
$$\nabla f = \left[\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right] \quad [2 \text{ per pixel}]$$

- Points in the direction of **most rapid increase** in intensity.
- **Gradient direction** (orientation of the *edge normal*): $\theta = \tan^{-1} \left(\frac{\partial f}{\partial y} / \frac{\partial f}{\partial x} \right)$ (use atan2).
- **Edge strength** = magnitude: $\|\nabla f\| = \sqrt{\left(\frac{\partial f}{\partial x} \right)^2 + \left(\frac{\partial f}{\partial y} \right)^2}$.

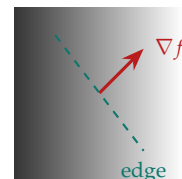
Special cases: vertical edge $\nabla f = [\partial f / \partial x, 0]$; horizontal edge $\nabla f = [0, \partial f / \partial y]$.



$$\nabla f = [\partial_x f, 0]$$



$$\nabla f = [0, \partial_y f]$$



$$\nabla f = [\partial_x f, \partial_y f]$$

Figure 2.1.7: The gradient points from dark to bright, perpendicular to the edge. The *edge direction* is $\theta + 90^\circ$. (Image coordinates often have y pointing down, which flips the sign of θ — be consistent.)

Derivation: Why the gradient is the direction of steepest ascent

The rate of change of f in a unit direction $\mathbf{u} = (\cos \phi, \sin \phi)$ is the directional derivative

$$D_{\mathbf{u}} f = \nabla f \cdot \mathbf{u} = \|\nabla f\| \cos(\phi - \theta).$$

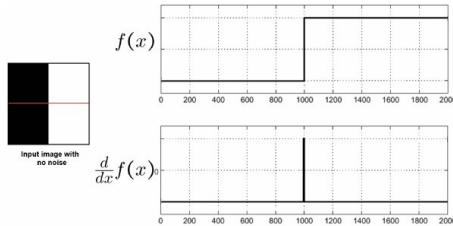
It is maximised at $\phi = \theta$ (value $\|\nabla f\|$), zero at $\phi = \theta \pm 90^\circ$ (along the edge, intensity does not

change) and most negative at $\phi = \theta + 180^\circ$. This is exactly why “gradient direction = edge normal” and “magnitude = edge strength”.

2.1.6 The Effect of Noise

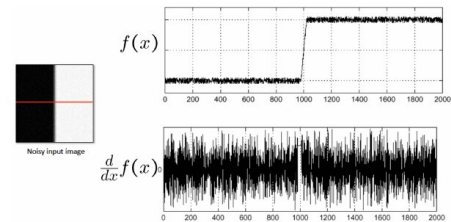
Take a single row (or column) of the image; plot intensity vs. position: a signal. **Where is the edge?** Without noise, the derivative has one clear spike. With noise, $\frac{d}{dx}f$ is a mess — the edge is invisible.

- Consider a single row or column of the image
 - Plotting intensity as a function of position gives a signal



Where is the edge?

(slide) No noise: derivative has a clean spike.



Now, where is the edge?

Source: S Seitz, K Grauman

(slide) With noise: “Now, where is the edge?”

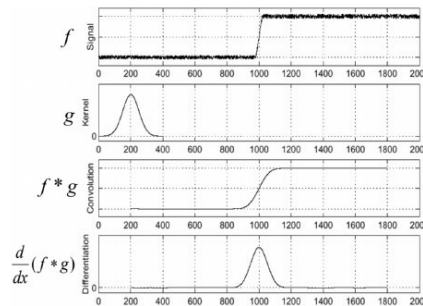
Source: S. Seitz, K. Grauman.

Derivation: Why differentiation amplifies noise

Spatial view: with i.i.d. noise $n \sim \mathcal{N}(0, \sigma^2)$, the difference $n(x+1) - n(x)$ has variance $2\sigma^2$ — noise doubles in power, while a smooth signal’s difference is small. **Frequency view (Week 1):** $\frac{d}{dx} \leftrightarrow j\omega$, so each frequency is amplified by $|\omega|$. Noise lives at high ω ; the derivative boosts it most.

Solution: smooth first, then look for peaks in $\frac{d}{dx}(f * g)$, where g is a Gaussian.

- Smooth first, and look for peaks in $\frac{d}{dx}(f * g)$



Source: S Seitz, R Urtasun

Figure 2.1.8: (slide) f , Gaussian g , smoothed $f * g$, and $\frac{d}{dx}(f * g)$: the peak marks the edge. Source: S. Seitz, R. Urtasun.

2.1.6.1 Derivative theorem of convolution

Differentiation is itself a convolution, and convolution is associative (and commutative):

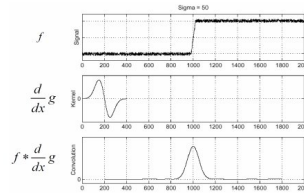
$$\frac{d}{dx}(f * g) = f * \frac{d}{dx}g$$

This saves us an operation: pre-compute the kernel $\frac{dg}{dx}$ (*derivative of Gaussian*) once and convolve the image a single time.

- Differentiation is achieved through convolution, and convolution is associative:

$$\frac{d}{dx}(f * g) = f * \frac{d}{dx}g = \frac{d}{dx}f * g$$

- This saves us an operation:



Source: S Seitz, R Urtasun

Figure 2.1.9: (slide) Convolving f with $\frac{dg}{dx}g$ directly gives the same peak. Source: S. Seitz, R. Urtasun.

Derivation: Proof

$\frac{d}{dx}(f * g)(x) = \frac{d}{dx} \int f(\tau)g(x - \tau) d\tau = \int f(\tau)g'(x - \tau) d\tau = (f * g')(x)$ (differentiate under the integral; only g depends on x). Discretely: $D * (g * f) = (D * g) * f$ by associativity, with D the difference kernel.

Key Idea: Derivative-of-Gaussian kernels

With $G_\sigma(x, y) = \frac{1}{2\pi\sigma^2}e^{-\frac{x^2+y^2}{2\sigma^2}}$:

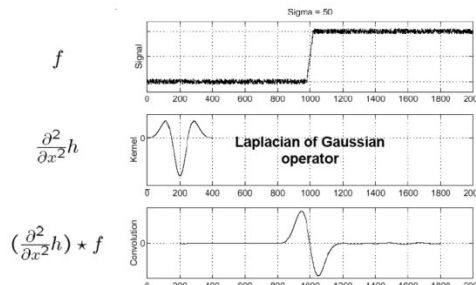
$$\frac{\partial G_\sigma}{\partial x} = -\frac{x}{\sigma^2}G_\sigma(x, y), \quad \frac{\partial G_\sigma}{\partial y} = -\frac{y}{\sigma^2}G_\sigma(x, y).$$

These are **separable**: $\frac{\partial G}{\partial x} = g'_\sigma(x) g_\sigma(y)$ — differentiate along x , smooth along y (Sobel is the tiny version).

2.1.7 What about the Second Derivative?

Detect edges as **zero-crossings** of $\frac{d^2}{dx^2}(f * g) = f * \frac{d^2g}{dx^2}$.

- Edge by detecting **zero-crossing** of bottom graph



Source: S Seitz, R Urtasun

Figure 2.1.10: (slide) f , the Laplacian-of-Gaussian operator $\frac{\partial^2}{\partial x^2}h$, and $f * \frac{\partial^2}{\partial x^2}h$: the edge is at the zero-crossing. Source: S. Seitz, R. Urtasun.

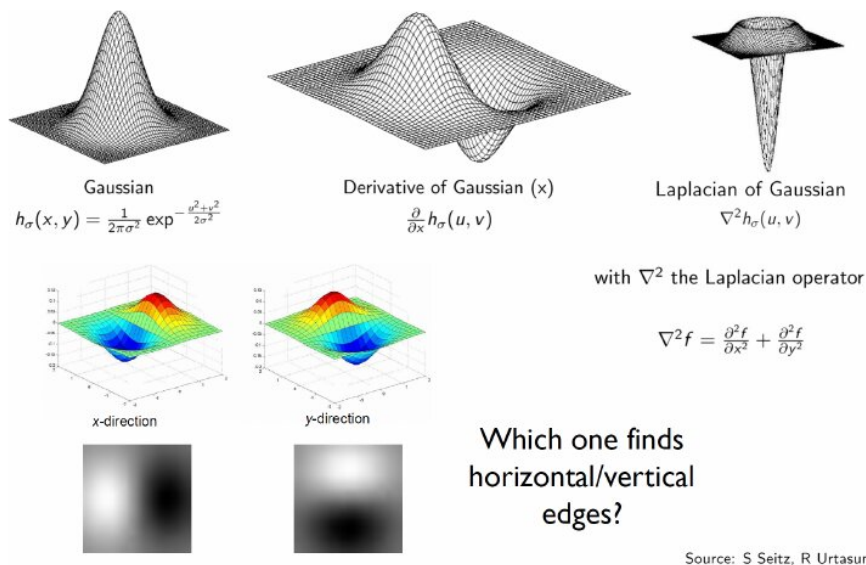


Figure 2.1.11: (slide) Gaussian, derivative of Gaussian (a *directional* operator) and Laplacian of Gaussian $\nabla^2 G$ (an *isotropic*, “Mexican hat” operator). “Which one finds horizontal/vertical edges?” — the derivative of Gaussian in x/y ; the LoG has no preferred direction.

Derivation: Laplacian of Gaussian in closed form

$\nabla^2 = \frac{\partial^2}{\partial x^2} + \frac{\partial^2}{\partial y^2}$. From $\partial_x G = -\frac{x}{\sigma^2} G$: $\partial_{xx} G = -\frac{1}{\sigma^2} G + \frac{x^2}{\sigma^4} G = \frac{x^2 - \sigma^2}{\sigma^4} G$. Adding the y term:

$$\nabla^2 G_\sigma(x, y) = \frac{x^2 + y^2 - 2\sigma^2}{\sigma^4} G_\sigma(x, y)$$

Negative inside the circle $r = \sqrt{2}\sigma$, positive outside — an inverted Mexican hat (often drawn negated). Zero-crossing radius $r_0 = \sqrt{2}\sigma$.

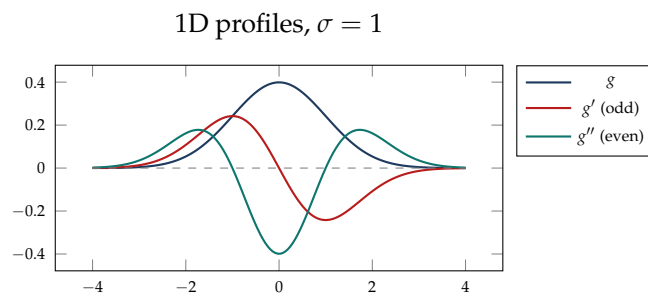
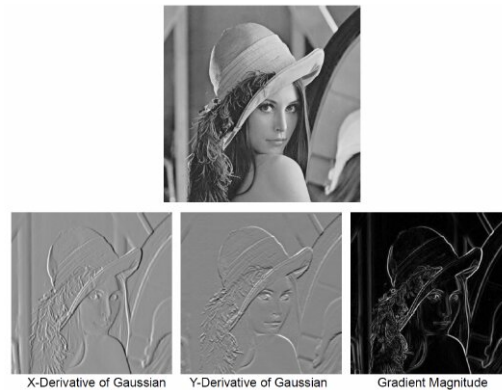


Figure 2.1.12: Gaussian and its derivatives. g' is odd (a smoothed $[-1, 0, 1]$); g'' is even with two zero-crossings at $\pm\sigma$ (in 1D).

2.1.8 Computing Gradients in Practice



Source: S Seitz, R Urtasun

Figure 2.1.13: (slide) x -derivative of Gaussian, y -derivative of Gaussian, and gradient magnitude of the Lena image. Source: S. Seitz, R. Urtasun.

2.1.9 Properties of a Good Edge Detector

- Criteria for a good edge detector?
 - **Good detection:** find all real edges, ignoring noise or other artifacts
 - **Good localization:** detect edges as close as possible to true edges
 - **Single response:** return one point only for each true edge point

Source: K. Grauman

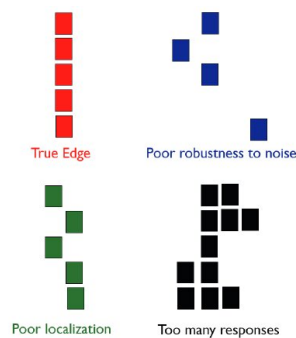


Figure 2.1.14: (slide) Canny's three criteria illustrated. Source: K. Grauman.

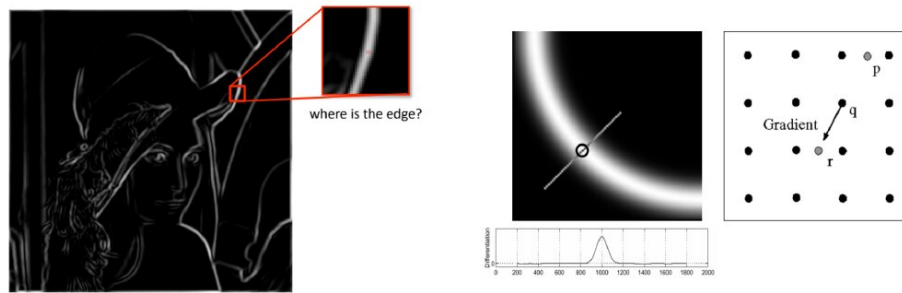
1. **Good detection:** find all real edges, ignoring noise and other artefacts (few false positives and false negatives).
2. **Good localisation:** detected edges as close as possible to the true edges.
3. **Single response:** return one point only for each true edge point (no thick or double edges).

Intuition: The fundamental trade-off

More smoothing (larger σ) $\Rightarrow$ better *detection* (noise suppressed) but worse *localisation* (edges blur and shift, nearby edges merge). Canny showed the optimal 1D step-edge detector under his criteria is well approximated by the **first derivative of a Gaussian**.

2.1.10 Non-Maximum Suppression (NMS)

After gradient computation, edges are several pixels thick (the “ridge” of $\|\nabla f\|$). **Thin them:** keep a pixel only if it is a **local maximum along the gradient direction**.



- Check if pixel is local maximum along gradient direction:
 - Could require checking interpolated pixels p and r

Source: N Snavely, R Urtasun

Figure 2.1.15: (slide) At pixel q , compare $\|\nabla f\|$ with neighbours p and r along the gradient; p, r may fall between pixels and need interpolation. Source: N. Snavely, R. Urtasun.

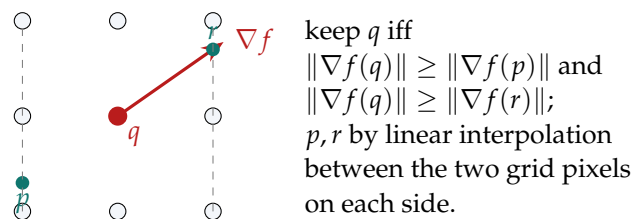
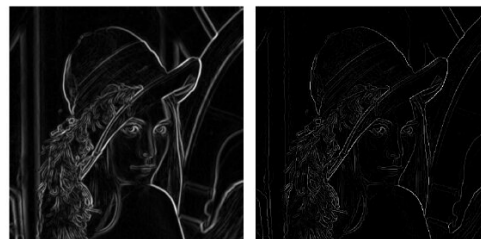


Figure 2.1.16: NMS geometry. A cheaper variant quantises θ into 4 directions ($0^\circ, 45^\circ, 90^\circ, 135^\circ$) and compares with the two grid neighbours in that direction.



Before and after non-maxima suppression

Source: Derek Hoiem

Figure 2.1.17: (slide) Before and after non-maximum suppression: thick ridges become 1-pixel-wide curves. Source: D. Hoiem.

2.1.11 Hysteresis Thresholding

Check for **well-connected** edges. A single threshold either breaks weak parts of real edges (too high) or keeps noise (too low). **Hysteresis:** use a *high* threshold to *start* edge curves and a *low* threshold to *continue* them.

- $\text{gradient} > \text{High} \Rightarrow$ **edge pixel** (strong);
- $\text{gradient} < \text{Low} \Rightarrow$ **non-edge pixel**;
- $\text{Low} \leq \text{gradient} \leq \text{High} \Rightarrow$ edge pixel **iff** it is connected to an edge pixel directly or via other pixels between Low and High (weak, kept only if linked).

- Check for well-connected edges. How?
 - Use **hysteresis**: use a *high* threshold to start edge curves and a *low* threshold to continue them.
- How does it work?
 - If gradient at pixel $> \text{'High'}$ $\Rightarrow$ **'edge pixel'**
 - If gradient at pixel $< \text{'Low'}$ $\Rightarrow$ **'non-edge pixel'**
 - If gradient at pixel $\geq \text{'Low'}$ and $\leq \text{'High'}$ $\Rightarrow$ **'edge pixel'** iff it is connected to an 'edge pixel' directly or via pixels between 'Low' and 'High'



Source: S Seitz, R Urtasun

Figure 2.1.18: (slide) Hysteresis on a gradient-magnitude profile. Source: S. Seitz, R. Urtasun.

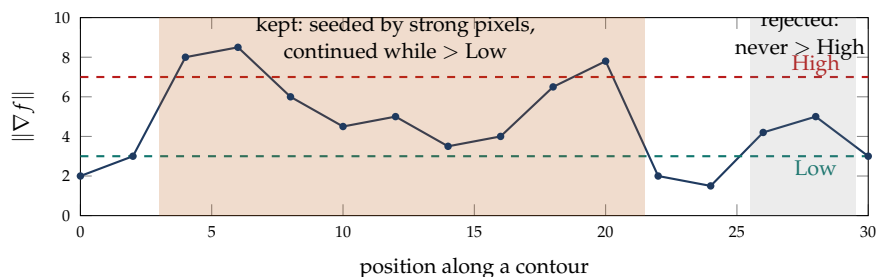


Figure 2.1.19: Hysteresis in 1D: the segment from 4 to 21 is kept because it contains strong pixels and never dips below Low; the segment near 26–29 is between Low and High but is not connected to any strong pixel, so it is discarded.

Derivation: Hysteresis as connected components

Let $S = \{p : \|\nabla f(p)\| > T_H\}$ and $W = \{p : T_L \leq \|\nabla f(p)\| \leq T_H\}$. Build the graph on $S \cup W$ with 8-connectivity. Final edges = union of connected components that contain at least one pixel of S . (Implement with BFS/flood fill from each strong pixel.) Canny recommended $T_H : T_L$ between 2:1 and 3:1.

2.1.12 The Canny Edge Detector (1986)

Probably the most widely used edge detector in computer vision.

Key Idea: Algorithm

1. Filter the image with the **derivative of Gaussian** (both x and y).
2. Find **magnitude and orientation** of the gradient.
3. **Non-maximum suppression**.
4. **Linking and thresholding (hysteresis)**: two thresholds, low and high; high starts edge curves, low continues them.

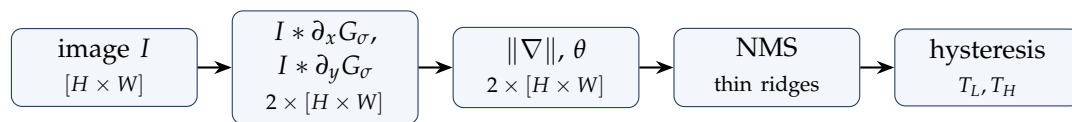
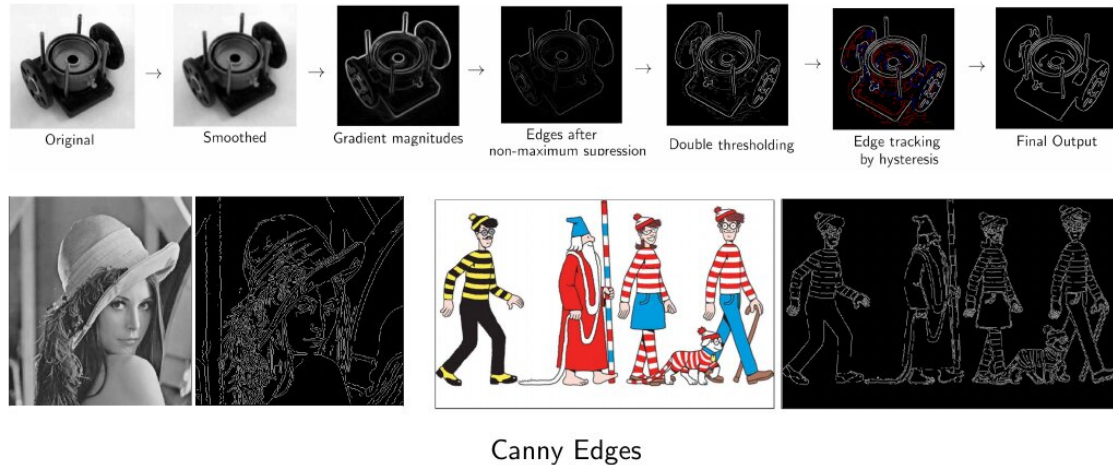


Figure 2.1.20: Canny pipeline with tensor shapes.



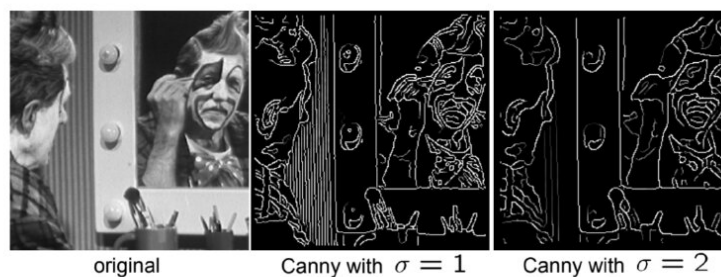
Source: Prem Kalra, R Urtasun, S Fidler

Figure 2.1.21: (slide) Canny pipeline and examples: original, smoothed, gradient magnitude, edges after NMS, double thresholding, edge tracking by hysteresis, final output. Source: P. Kalra, R. Urtasun, S. Fidler.

2.1.12.1 Effect of σ

The choice of σ (Gaussian spread/size) depends on the desired behaviour: **large σ detects large-scale edges; small σ detects fine edges.**

- The choice of σ (Gaussian kernel spread/size) depends on desired behavior
 - large σ detects large-scale edges
 - small σ detects fine edges



Source: S Seitz, R Urtasun

Figure 2.1.22: (slide) Canny with $\sigma = 1$ vs. $\sigma = 2$. Source: S. Seitz, R. Urtasun.

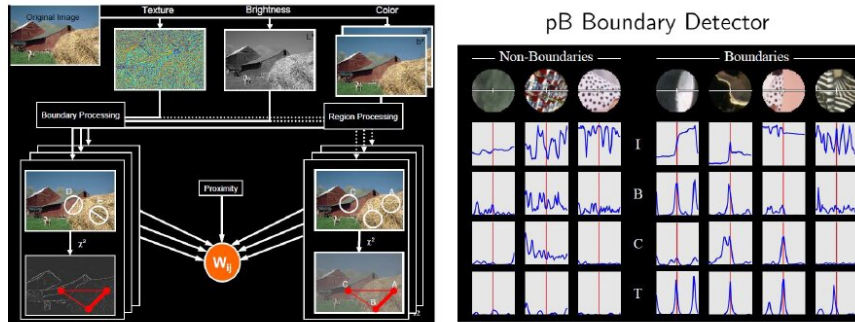
2.1.13 More Recent Methods in Edge Detection

2.1.13.1 Pb: learning boundaries from brightness, colour and texture (Martin, Fowlkes & Malik, 2004)

At every pixel and orientation, compare the two halves of a disc: compute histograms of *brightness*, *colour* and *texture* (textons) in each half and measure their difference (e.g. χ^2 distance). Combine

these cues with a classifier (logistic regression) trained on **human-annotated boundaries** (Berkeley Segmentation Dataset) to output a *probability of boundary* $P_b(x, y, \theta)$.

Learning to Detect Natural Image Boundaries Using Local Brightness, Color, and Texture Cues
(Martin et al, 2004)



Source: Derek Hoiem

Figure 2.1.23: (slide) The Pb boundary detector: oriented half-disc gradients of brightness, colour and texture are combined into a boundary probability. Source: D. Hoiem.

$$\chi^2(g, h) = \frac{1}{2} \sum_i \frac{(g_i - h_i)^2}{g_i + h_i} \quad (\text{histogram distance between the two half-discs}).$$

2.1.13.2 Structured Forests for fast edge detection (Dollár & Zitnick, 2013)

Goal: quickly predict whether each pixel is an edge. **Insights:** predictions can be learned from training data; predictions for nearby pixels should *not* be independent. **Solution:** train *structured* random forests that split data into patches with similar boundaries; predict boundaries at the **patch level** and aggregate (average votes).

1. Extract overlapping 32×32 patches at three scales.
2. Features: pixel values and pairwise differences in feature maps (LUV colour, gradient magnitude, oriented gradient).
3. Predict T boundary maps for the central 16×16 region using T trained decision trees.
4. Average predictions for each pixel across all patches.

Structured Forests for Fast Edge Detection (Dollár et al, 2013)

• Algorithm

- ① Extract overlapping 32×32 patches at three scales
- ② Features are pixel values and pairwise differences in feature maps (LUV color, gradient magnitude, oriented gradient)
- ③ Predict T boundary maps in the central 16×16 region using T trained decision trees
- ④ Average predictions for each pixel across all patches

Source: Derek Hoiem

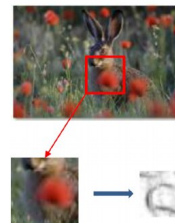


Figure 2.1.24: (slide) Structured forest: a patch is mapped to a 16×16 boundary map; overlapping predictions are averaged. Source: D. Hoiem.

2.1.13.3 Crisp boundaries via Pointwise Mutual Information (Isola et al., 2014)

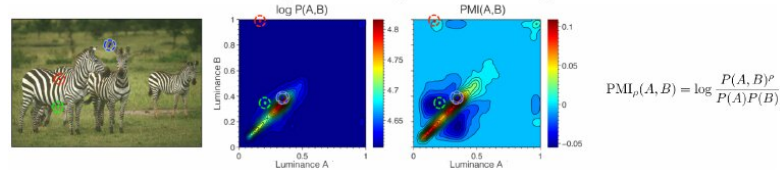
Idea: pixel combinations that are *unlikely to occur together* are edges. For features A, B of neighbouring pixels,

$$\text{PMI}_\rho(A, B) = \log \frac{P(A, B)^\rho}{P(A)P(B)},$$

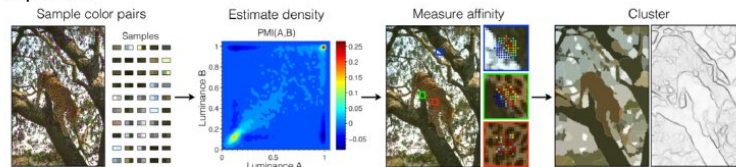
estimated from the image itself. High PMI $\Rightarrow$ the colours co-occur often (same region); low PMI $\Rightarrow$ boundary. Pipeline: sample colour pairs $\rightarrow$ estimate density $\rightarrow$ PMI affinity $\rightarrow$ spectral clustering $\rightarrow$ boundaries.

Crisp Boundary Detection using Pointwise Mutual Information (Isola et al, 2014)

- Pixel combinations that are unlikely to be together are edges



- Algorithm Pipeline:



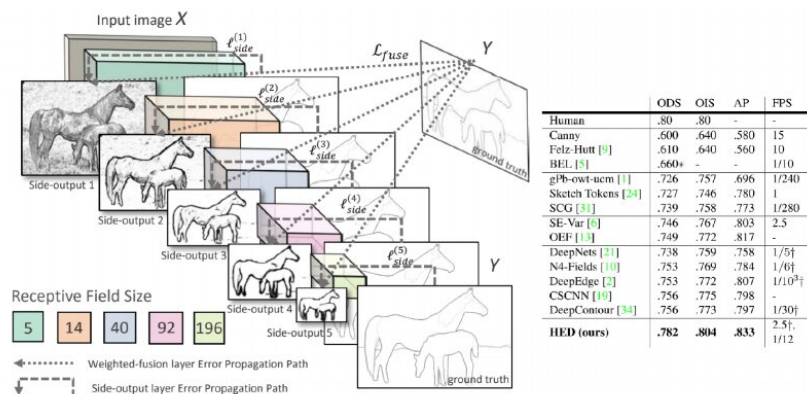
Source: Derek Hoiem

Figure 2.1.25: (slide) PMI-based crisp boundary detection pipeline. Source: D. Hoiem.

2.1.13.4 Holistically-Nested Edge Detection, HED (Xie & Tu, 2015)

A deep CNN (VGG-16 backbone) with **side outputs** after each stage: each side output predicts an edge map at its own scale, each is **deeply supervised** with the ground-truth edges, and a learned **fusion layer** combines them. Early layers give fine, noisy edges; deep layers give coarse, semantic boundaries; the fusion is multi-scale by construction. It set the state of the art on BSDS500 (ODS F ≈ 0.78 vs. Canny ≈ 0.60).

Holistically Nested Edge Detection (Xie et al, 2015)



Source: Derek Hoiem

Figure 2.1.26: (slide) HED architecture with side outputs and weighted fusion, and its comparison table. Source: D. Hoiem.

Intuition: From Canny to HED — the arc of the whole course

Canny: hand-designed filter + hand-designed post-processing. Pb / structured forests: hand-designed features, *learned* combination. HED: everything learned end-to-end, multi-scale by design. The same arc repeats for corners/descriptors (SIFT → learned features) and segmentation.

2.1.14 Homework

Reading: Szeliski Ch. 2. **Question:** How do you go from Canny edges to straight lines? (Answered at the start of Module 2.2.)

Module Summary: §2.1

Edges = rapid intensity change caused by normal, depth, reflectance or illumination discontinuities. Derivatives via finite-difference filters ($[-1, 1]$, Prewitt, Sobel, Roberts); gradient ∇f , direction $\theta = \text{atan2}(f_y, f_x)$ (edge normal), magnitude $\|\nabla f\|$. Noise is amplified by differentiation $\Rightarrow$ smooth first; by the derivative theorem use derivative-of-Gaussian kernels in one pass. Second-derivative (LoG) edges are zero-crossings. Canny: DoG filtering $\rightarrow$ magnitude/orientation $\rightarrow$ NMS $\rightarrow$ hysteresis; σ sets the scale. Learned detectors (Pb, structured forests, PMI, HED) beat Canny on real boundaries.

From Edges to Blobs and Corners

2.2.1 Review: From Canny Edges to Straight Lines

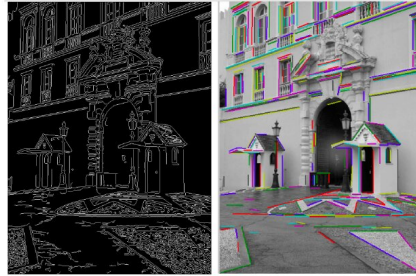
The homework question of §2.1, answered:

1. Compute Canny edges.
2. Compute $\nabla_x f$, $\nabla_y f$ (gradients in x, y) and $\theta = \tan^{-1} \frac{\nabla_y f}{\nabla_x f}$.
3. Assign each edge pixel to one of **8 directions**. For each direction d , obtain “**edgelets**”: connected components of edge pixels whose direction is in $\{d-1, d, d+1\}$ (tolerance of one bin).
4. For each edgelet compute its orientation and **straightness** using the eigenvalues/eigenvectors of the **second-moment matrix** of its pixel coordinates:

$$M = \begin{bmatrix} \sum (x - \mu_x)^2 & \sum (x - \mu_x)(y - \mu_y) \\ \sum (x - \mu_x)(y - \mu_y) & \sum (y - \mu_y)^2 \end{bmatrix}_{[2 \times 2]}, \quad [\mathbf{v}, \lambda] = \text{eig}(M),$$

$$\theta = \tan^{-1}(v_1/v_0) \quad (\mathbf{v} = \text{eigenvector of the larger eigenvalue}), \quad \text{straightness} = \lambda_2/\lambda_1.$$

5. Threshold straightness appropriately and store the line segments.



Credit: Derek Hoiem

Figure 2.2.1: (slide) Line segments recovered from Canny edges, coloured by orientation. Credit: D. Hoiem.

Derivation: Why eigenvectors of the scatter matrix give line direction

M is $(n \times n)$ the covariance of the edgelet's pixel positions. For a unit direction $\mathbf{u}$, the spread of the points along $\mathbf{u}$ is $\sum_i (\mathbf{u}^\top (\mathbf{p}_i - \boldsymbol{\mu}))^2 = \mathbf{u}^\top M \mathbf{u}$. Maximising this subject to $\|\mathbf{u}\| = 1$ (Lagrangian $\mathbf{u}^\top M \mathbf{u} - \lambda(\mathbf{u}^\top \mathbf{u} - 1)$) gives $M \mathbf{u} = \lambda \mathbf{u}$: the **largest-eigenvalue eigenvector** is the line direction (this is PCA). The smaller eigenvalue is the spread *perpendicular* to the line. For a perfectly straight segment $\lambda_{\min} = 0$, so straightness $\lambda_{\min}/\lambda_{\max} \rightarrow 0$ (small = straight; the slide's ratio λ_2/λ_1 with $\lambda_1 \geq \lambda_2$).

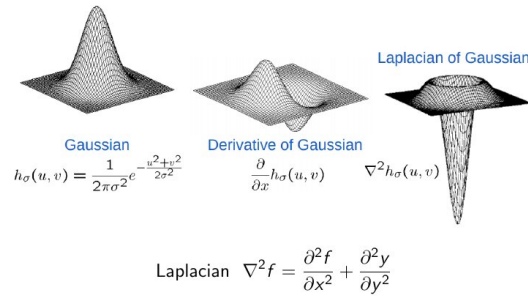
2.2.2 Going Further: the Laplacian of Gaussian

What if we take the **Laplacian** of the Gaussian-smoothed image?

$$\text{Laplacian: } \nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}.$$

(The slide has a typo, $\frac{\partial^2 y}{\partial y^2}$; it should be $\frac{\partial^2 f}{\partial y^2}$.)

• What if we took the Laplacian of Gaussian?



Credit: S Seitz, K Grauman

Figure 2.2.2: (slide) Gaussian $\rightarrow$ derivative of Gaussian $\rightarrow$ Laplacian of Gaussian $\nabla^2 h_\sigma$. Credit: S. Seitz, K. Grauman.

2.2.2.1 Discrete Laplacian: how is the 3×3 filter obtained?

Derivation: From second differences to the kernel

Second derivative = difference of first differences:

$$\frac{\partial^2 f}{\partial x^2} \approx [f(x+1, y) - f(x, y)] - [f(x, y) - f(x-1, y)] = f(x+1, y) + f(x-1, y) - 2f(x, y),$$

$$\frac{\partial^2 f}{\partial y^2} \approx f(x, y+1) + f(x, y-1) - 2f(x, y).$$

Sum: $\nabla^2 f \approx f(x+1, y) + f(x-1, y) + f(x, y+1) + f(x, y-1) - 4f(x, y)$, i.e. the kernel

$$\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

Its coefficients sum to 0 (flat regions give 0). Including diagonals gives the 8-neighbour variant $[1, 1, 1; 1, -8, 1; 1, 1, 1]$. (The slide calls this a “LoG” filter; strictly it is the discrete *Laplacian*, which becomes a LoG when applied to a Gaussian-smoothed image.)



Original Image



Laplacian



Laplacian of Gaussian

What else can LoG do?

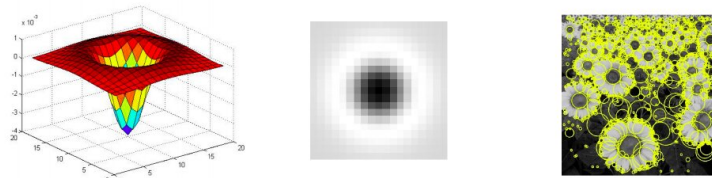
Credit: K Grauman

Figure 2.2.3: (slide) Original, Laplacian (noisy) and Laplacian of Gaussian (cleaner). “What else can LoG do?” Credit: K. Grauman.

2.2.3 LoG as a Blob Detector

Recall: convolution compares a little “picture” of what you want to find (the kernel) with every local region. The LoG kernel is **circularly symmetric** — it looks like a bright/dark disc surrounded by a ring — so its response is extremal at the centres of **blobs** whose size matches the kernel.

- Recall that convolution with a filter can be viewed as comparing a little “picture” of what you want to find against all local regions in the image.



- Observing the LoG filter matrix reveals that it is circularly symmetric. Thus it can be used for blob detection!

Credit: S Lazebnik

Figure 2.2.4: (slide) LoG surface, the LoG kernel as an image, and blobs (sunflowers) it detects. Credit: S. Lazebnik.

Derivation: Which blob size does a LoG respond to most?

For a binary disc of radius r , the (scale-normalised, see §2.3) response $\sigma^2 \nabla^2 G_\sigma * \text{disc}$ at the centre is extremal when the zero-crossing circle of the LoG aligns with the disc boundary, i.e. $r = \sqrt{2} \sigma$ (in 2D). So a blob of radius r is best detected at $\sigma^* = r / \sqrt{2}$ — the basis of automatic scale selection.

2.2.4 From Blobs to Corners

- In the following image, what are some interesting features to choose?



Credit: K Grauman, R Urtasun

Figure 2.2.5: (slide) “What are some interesting features to choose?” Credit: K. Grauman, R. Urtasun.

Look for image regions that are unusual. How to define “unusual”?

- Textureless patches** are nearly impossible to localise.
- Patches with **large contrast changes** (gradients) are easier to localise.
- But straight line segments at a single orientation suffer from the **aperture problem**: it is only possible to align the patches along the direction *normal* to the edge.
- Gradients in at least **two (significantly) different orientations** are the easiest — e.g. **corners**.

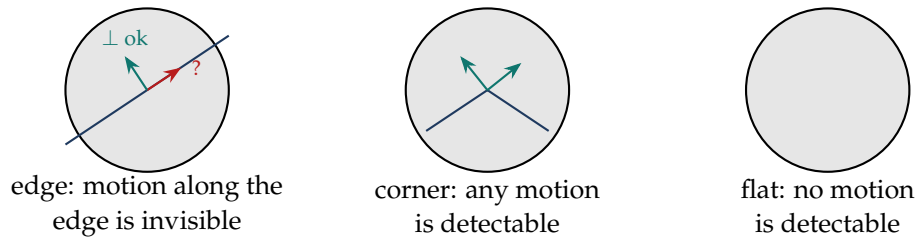
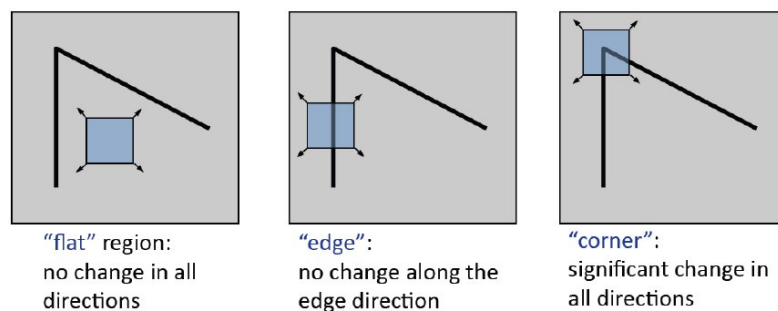


Figure 2.2.6: The aperture problem: seen through a small window, a straight edge’s displacement can only be measured perpendicular to it.

2.2.4.1 Shifting a small window

Consider a small window of pixels. How does the window’s content change when you shift it?

- Consider a small window of pixels. How does the window change when you shift it?



Credit: S Seitz, D Frolova, D Simakova, R Urtasun

Figure 2.2.7: (slide) “Flat”: no change in any direction; “edge”: no change along the edge direction; “corner”: significant change in all directions. Credit: S. Seitz, D. Frolova, D. Simakova, R. Urtasun.

2.2.5 Autocorrelation

How do we quantify “significant” change? The **autocorrelation function**: the (weighted) sum of squared differences between pixel intensities under a small displacement $\Delta \mathbf{u}$ of the patch:

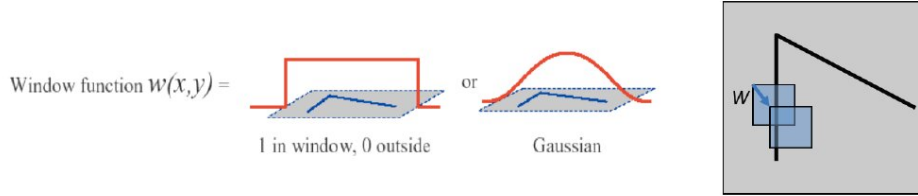
$$E_{AC}(\Delta \mathbf{u}) = \sum_{x,y} w(\mathbf{p}_i) [I(\mathbf{p}_i + \Delta \mathbf{u}) - I(\mathbf{p}_i)]^2 \quad \mathbf{p}_i = (x, y), \quad \Delta \mathbf{u} = (\Delta u, \Delta v)^\top.$$

w is a window function: a box (1 inside, 0 outside) or, better, a Gaussian (rotationally symmetric, smooth).

- In the previous slide, how to quantify the "significant" change of the window?
- Answer: **Autocorrelation function**. Compute the sum of squared differences between pixel intensities with respect to small variations in the image patch position.

$$E_{AC}(\Delta \mathbf{u}) = \sum_{x,y} w(\mathbf{p}_i) [I(\mathbf{p}_i + \delta \mathbf{u}) - I(\mathbf{p}_i)]^2$$

where $\mathbf{p}_i = (x, y)$, a particular position on the image.



Credit: R Urtasun

Figure 2.2.8: (slide) Window function $w(x, y)$: box (1 in window, 0 outside) or Gaussian. Credit: R. Urtasun.

Derivation: Taylor expansion to a quadratic form

With the first-order Taylor expansion $I(\mathbf{p}_i + \Delta \mathbf{u}) \approx I(\mathbf{p}_i) + \nabla I(\mathbf{p}_i) \Delta \mathbf{u}$, where $\nabla I(\mathbf{p}_i) = \left(\frac{\partial I}{\partial x}, \frac{\partial I}{\partial y} \right) (\mathbf{p}_i)$ $[1 \times 2]$:

$$\begin{aligned} E_{AC}(\Delta \mathbf{u}) &= \sum w(\mathbf{p}_i) [I(\mathbf{p}_i + \Delta \mathbf{u}) - I(\mathbf{p}_i)]^2 \approx \sum w(\mathbf{p}_i) [I(\mathbf{p}_i) + \nabla I(\mathbf{p}_i) \Delta \mathbf{u} - I(\mathbf{p}_i)]^2 \\ &= \sum w(\mathbf{p}_i) [\nabla I(\mathbf{p}_i) \Delta \mathbf{u}]^2 = \sum w(\mathbf{p}_i) \Delta \mathbf{u}^\top \nabla I^\top \nabla I \Delta \mathbf{u} = \boxed{\Delta \mathbf{u}^\top A \Delta \mathbf{u}} \end{aligned}$$

using $(\mathbf{a}\mathbf{b})^2 = \mathbf{b}^\top \mathbf{a}^\top \mathbf{a}\mathbf{b}$ for a row vector $\mathbf{a}$ and column vector $\mathbf{b}$.

Key Idea: The second-moment (structure) matrix

$$A = \sum_u \sum_v w(u, v) \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix} = w * \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix} \quad [2 \times 2 \text{ per pixel}]$$

The weighted sums are discrete convolutions with the window w : compute three images I_x^2 , I_y^2 , $I_x I_y$ and blur each with w . A is symmetric positive semi-definite, so

$$A = U \begin{bmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{bmatrix} U^\top, \quad A \mathbf{u}_i = \lambda_i \mathbf{u}_i, \quad \lambda_i \geq 0.$$

The **eigenvalues** reveal the amount of intensity change in the two principal orthogonal gradient directions in the window.

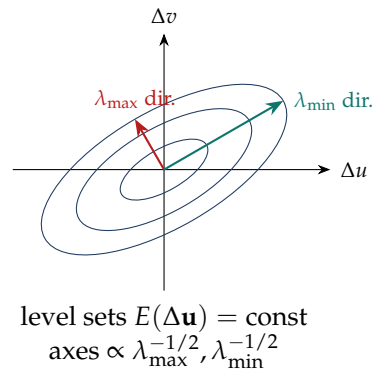
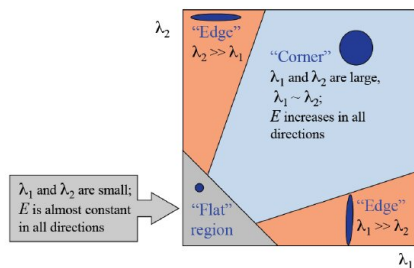


Figure 2.2.9: $E(\Delta \mathbf{u}) = \Delta \mathbf{u}^\top A \Delta \mathbf{u}$ is a bowl; its elliptical level sets are short in the direction of fastest change ($\lambda_{\max}$) and long in the slowest ($\lambda_{\min}$).

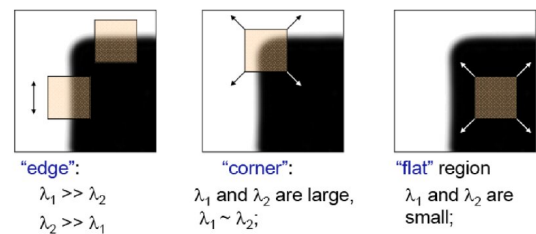
2.2.6 Classifying Points by the Eigenvalues

- How do the eigenvalues determine if an image point is a corner?



Credit: N Snavely, R Urtasun

(slide) The (λ_1, λ_2) plane: corner, edge, flat regions.
Credit: N. Snavely, R. Urtasun.



Credit: K Grauman, R Urtasun

(slide) Edge: $\lambda_1 \gg \lambda_2$; corner: both large, $\lambda_1 \sim \lambda_2$; flat: both small. Credit: K. Grauman, R. Urtasun.

Region	Eigenvalues	$E(\Delta \mathbf{u})$
Flat	λ_1, λ_2 both small	almost constant in all directions
Edge	$\lambda_1 \gg \lambda_2$ (or vice versa)	increases only across the edge
Corner	λ_1, λ_2 both large, $\lambda_1 \sim \lambda_2$	increases in all directions

2.2.7 Harris Corner Detector

Key Idea: Algorithm

1. Compute gradients I_x, I_y at each point in the image.
2. Compute A for each image window (products $I_x^2, I_y^2, I_x I_y$ blurred by w).
3. Compute the **cornerness** score (no eigen-decomposition needed):

$$M_c = \lambda_1 \lambda_2 - \kappa (\lambda_1 + \lambda_2)^2 = \det(A) - \kappa \text{trace}^2(A) \quad \kappa \approx 0.04\text{--}0.06.$$

4. Find points whose window gives a large cornerness ($M_c > \text{threshold}$).
5. Take points of local maxima: perform **non-maximum suppression**.

Derivation: Homework: $\text{trace} = \sum \lambda_i$ and $\det = \prod \lambda_i$

The characteristic polynomial is $p(\lambda) = \det(A - \lambda I) = \prod_{i=1}^n (\lambda_i - \lambda)$ (it has the eigenvalues as roots, leading coefficient $(-1)^n$).

- Set $\lambda = 0$: $\det A = \prod_i \lambda_i$.
- Compare the coefficient of λ^{n-1} . On the right: $(-1)^{n-1} \sum_i \lambda_i$. On the left, expanding $\det(A - \lambda I)$, only the product of the diagonal entries $\prod_i (a_{ii} - \lambda)$ contributes a λ^{n-1} term: $(-1)^{n-1} \sum_i a_{ii}$. Hence $\text{tr } A = \sum_i \lambda_i$.

For 2×2 : $\det \begin{bmatrix} a - \lambda & b \\ b & c - \lambda \end{bmatrix} = \lambda^2 - (a + c)\lambda + (ac - b^2)$, and by Vieta $\lambda_1 + \lambda_2 = a + c$, $\lambda_1 \lambda_2 = ac - b^2$. So $M_c = (ac - b^2) - \kappa(a + c)^2$ with $a = \sum w I_x^2$, $b = \sum w I_x I_y$, $c = \sum w I_y^2$ — cheap.

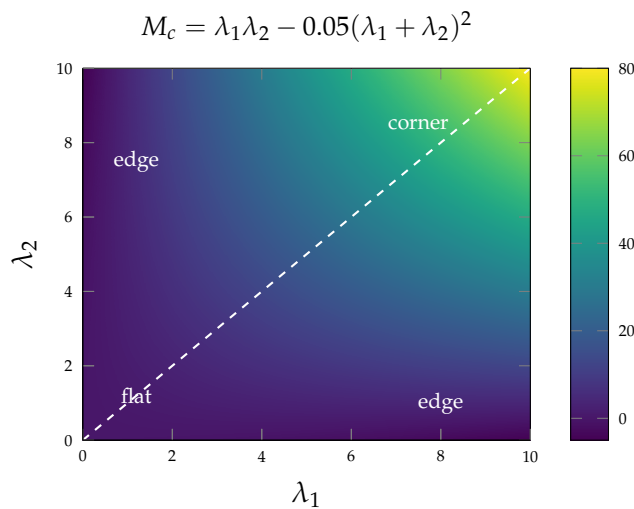


Figure 2.2.10: Heat map of the Harris response. $M_c > 0$ and large near the diagonal (corner); $M_c < 0$ along the axes (edges, one eigenvalue dominant); $|M_c|$ small near the origin (flat).

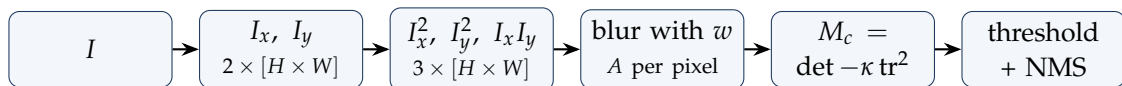


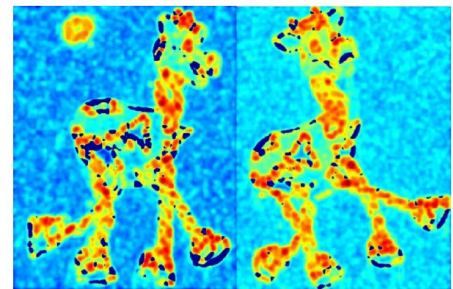
Figure 2.2.11: Harris pipeline with shapes.

2.2.7.1 Harris in action



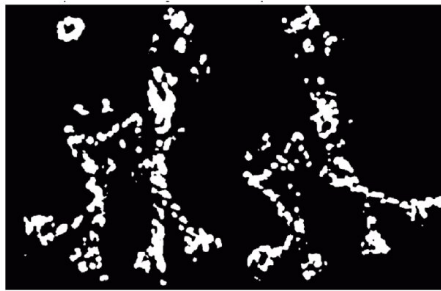
Credit: K Grauman, R Urtasun

(slide) Input image pair.



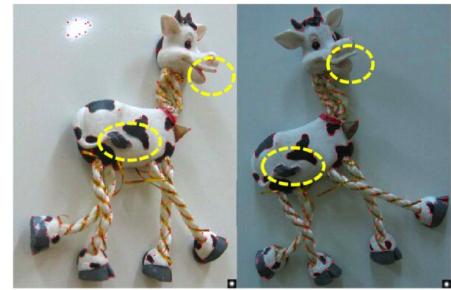
Credit: K Grauman, R Urtasun

(slide) Cornerness M_c (red = high).



Credit: K Grauman, R Urtasun

(slide) Thresholded high response.



Credit: K Grauman, R Urtasun

(slide) After NMS: detected corners, consistent across the two views. Credit: K. Grauman, R. Urtasun.

2.2.8 Harris Detector Variants

- **Harris & Stephens (1988):** $\det(A) - \alpha \text{trace}(A)^2 = \lambda_0 \lambda_1 - \alpha(\lambda_0 + \lambda_1)^2$. Rotationally invariant and down-weights edge-like features where $\lambda_1 \gg \lambda_0$.
- **Shi-Tomasi / Triggs (2004):** $\lambda_0 - \alpha \lambda_1$ (Shi-Tomasi uses simply $\lambda_{\min}$): directly demand the *smaller* eigenvalue be large.
- **Brown et al. (2005):** harmonic mean $\frac{\det(A)}{\text{trace}(A)} = \frac{\lambda_0 \lambda_1}{\lambda_0 + \lambda_1}$, which is smoother when $\lambda_0 \approx \lambda_1$.

Intuition: All variants say the same thing

Each score is large only if the *smaller* eigenvalue is large. E.g. harmonic mean $\frac{\lambda_0 \lambda_1}{\lambda_0 + \lambda_1} \approx \lambda_0$ when $\lambda_1 \gg \lambda_0$ — it is dominated by the weaker direction.

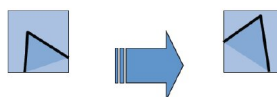
2.2.9 Properties of the Harris Detector

2.2.9.1 Rotation invariance — yes

- Rotation-invariant?

$$\mathbf{A} = w * \begin{bmatrix} I_x^2 & I_x I_y \\ I_y I_x & I_y^2 \end{bmatrix} = \mathbf{U} \begin{bmatrix} \lambda_0 & 0 \\ 0 & \lambda_1 \end{bmatrix} \mathbf{U}^T \quad \text{with} \quad \mathbf{A} \mathbf{u}_i = \lambda_i \mathbf{u}_i$$

- Relative cornerness remains the same!



Credit: N Snavely, R Urtasun

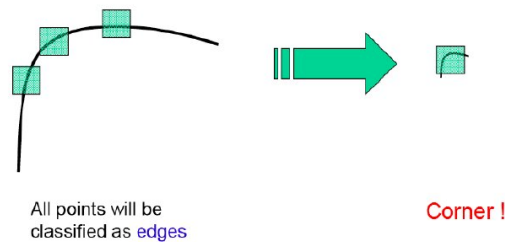
Figure 2.2.12: (slide) The ellipse rotates but its shape (eigenvalues) stays the same: relative cornerness is unchanged. Credit: N. Snavely, R. Urtasun.

Derivation: Why

Rotating the image by R rotates gradients: $\nabla I' = R \nabla I$, so $A' = R A R^T$. Similar matrices have the same eigenvalues: $\det(A' - \lambda I) = \det(R(A - \lambda I)R^T) = \det(A - \lambda I)$. Hence M_c is unchanged (with a rotationally symmetric Gaussian window).

2.2.9.2 Scale invariance — no

- Scale-invariant?



Credit: R Urtasun

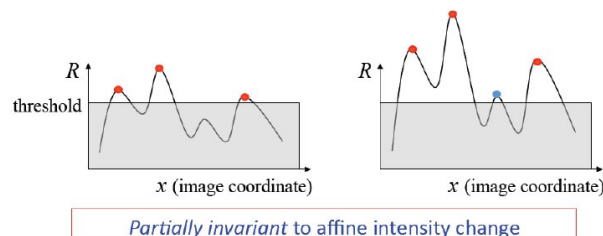
Figure 2.2.13: (slide) At a fine scale every small window on a smooth curve looks like an edge; zoomed out, the same curve is a corner. Credit: R. Urtasun.

Harris uses a fixed window: a corner that is large relative to the window looks like a set of edges. We need **scale selection** (§2.3).

2.2.9.3 Photometric change: partially invariant to affine intensity change

- Additive shift $I \rightarrow I + b$: only derivatives are used $\Rightarrow$ **invariant**.
- Scaling $I \rightarrow aI$: $I_x \rightarrow aI_x$ so $A \rightarrow a^2A$ and $M_c \rightarrow a^4M_c$. Positions of local maxima are unchanged, but a *fixed threshold* admits more/fewer corners $\Rightarrow$ **partially invariant**.

- Photometric change: Affine intensity change $I = aI + b$?
- Only derivatives are used, so it's invariant to shift $I = I + b$.
- What about intensity scale?



Credit: K Grauman, R Urtasun

Figure 2.2.14: (slide) Scaling intensity scales $R = M_c$; with a fixed threshold, some corners appear/disappear. Credit: K. Grauman, R. Urtasun.

2.2.10 Homework

Reading: Szeliski Ch. 2. Linear-algebra review: show $\text{trace} = \sum \lambda_i$ and $\det = \prod \lambda_i$ (done above).

Module Summary: §2.2

Canny $\rightarrow$ lines: bin edge directions, connected edgelets, PCA of pixel coordinates (orientation = top eigenvector, straightness = $\lambda_{\min}/\lambda_{\max}$). Laplacian kernel $[0, 1, 0; 1, -4, 1; 0, 1, 0]$ from second differences; LoG is circularly symmetric $\Rightarrow$ blob detector (best at $r = \sqrt{2}\sigma$). Good features have gradients in two directions (aperture problem). Autocorrelation $E(\Delta \mathbf{u}) \approx \Delta \mathbf{u}^\top A \Delta \mathbf{u}$ with

$A = w * [I_x^2, I_x I_y; I_x I_y, I_y^2]$; eigenvalues classify flat/edge/corner. Harris $M_c = \det A - \kappa \operatorname{tr}^2 A$, then threshold + NMS. Variants: Shi–Tomasi/Triggs, harmonic mean. Rotation-invariant, not scale-invariant, invariant to $I + b$ but not aI .

Scale Space, Image Pyramids and Filter Banks

2.3.1 Review: What Harris Gives Us

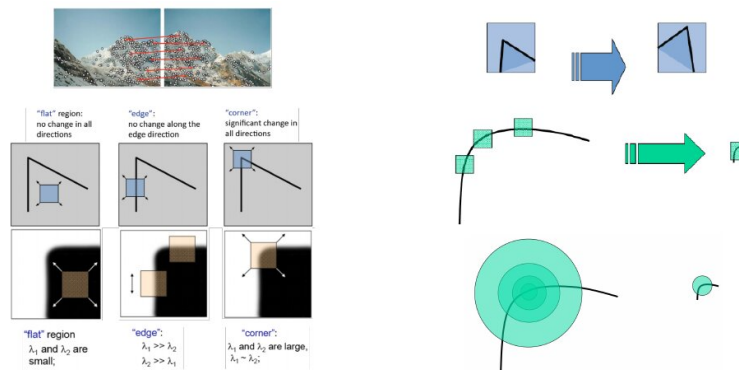


Figure 2.3.1: (slide) Review: Harris corners (flat / edge / corner windows) are rotation-invariant, but the same curve is a corner at a coarse scale and a set of edges at a fine scale — **not scale-invariant**.

2.3.2 Scale-Invariant Interest Point Detection

Question: how can we *independently* select interest points in each image such that detections are **repeatable across different scales**?

- **Option 1:** extract features at a variety of scales (e.g. multiple resolutions in a pyramid) and match features at the same level. *When does this work?* Only when the scale change between images happens to match a pyramid level — and it multiplies the number of features.
- **Better:** extract features that are **stable in both location and scale**: find the scale that gives a **local maximum of a function f in both position and scale**.

- How can we independently select interest points in each image, such that detections are repeatable across different scales?
 - Extract features at a variety of scales, e.g., by using multiple resolutions in a pyramid, and then matching features at the same level.
 - When does this work?
 - More efficient to extract features stable in both location and scale.
 - Find scale that gives local maxima of a function f in both position and scale.

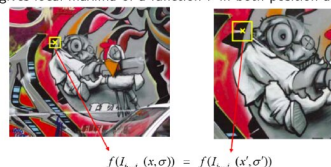
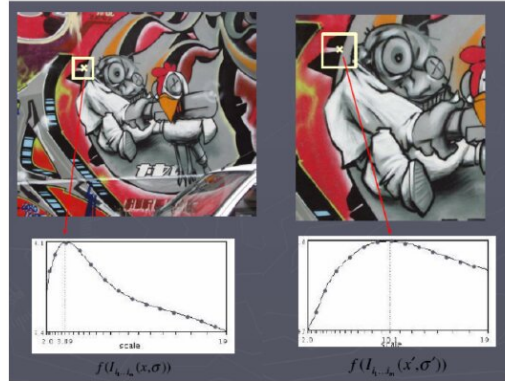


Figure 2.3.2: (slide) Choose the region size independently in each image so that $f(I_{i_1...i_m}(x, \sigma)) = f(I_{i_1...i_m}(x', \sigma'))$: corresponding regions have the same content.

2.3.3 Automatic Scale Selection

For a fixed point, evaluate a function f on windows of increasing size (the **scale signature**). The scale at which f peaks is the point's **characteristic scale**. If the second image is the first scaled by s , the peak moves from σ to $s\sigma$ — so the two regions cover the same content.

Function responses for increasing scale (scale signature).



Credit: R Urtasun

Figure 2.3.3: (slide) Scale signatures of corresponding points in two images: the peaks occur at scales in the ratio of the image scale factor. Credit: R. Urtasun.

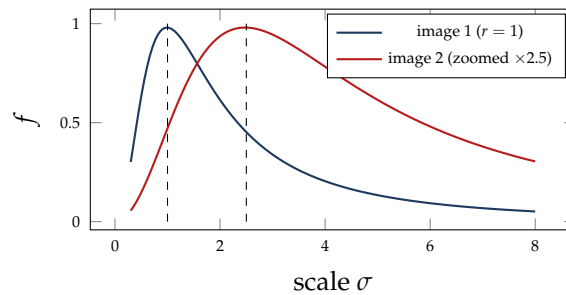


Figure 2.3.4: Characteristic scale: the signature of a zoomed image is the same curve stretched along σ , so σ_2^*/σ_1^* equals the zoom factor.

Key Idea: Which f ? The scale-normalised Laplacian

A good f is a *blob/corner detector evaluated at scale σ* , most commonly the **scale-normalised LoG**

$$f(x, y, \sigma) = \sigma^2 |\nabla^2 G_\sigma * I|(x, y).$$

The factor σ^2 is essential.

Derivation: Why multiply by σ^2

Let $L(\mathbf{x}, \sigma) = G_\sigma * I$ and let the second image be a zoom: $I'(\mathbf{x}') = I(\mathbf{x}'/s)$, i.e. $\mathbf{x}' = s\mathbf{x}$. Blurring a zoomed image with a zoomed kernel gives the zoomed result: $L'(s\mathbf{x}, s\sigma) = L(\mathbf{x}, \sigma)$ (change variables in the convolution integral; the Gaussian's $\frac{1}{2\pi\sigma^2}$ normalisation absorbs the Jacobian s^2). Derivatives, however, pick up factors of $1/s$: $\partial_{x'} = \frac{1}{s}\partial_x$, so

$$\nabla'^2 L'(s\mathbf{x}, s\sigma) = \frac{1}{s^2} \nabla^2 L(\mathbf{x}, \sigma).$$

The raw LoG response is *not* equal at corresponding points — it shrinks by s^2 . Multiplying by the scale squared fixes it:

$$(s\sigma)^2 \nabla'^2 L'(s\mathbf{x}, s\sigma) = \sigma^2 \nabla^2 L(\mathbf{x}, \sigma).$$

So $\sigma^2 \nabla^2 L$ takes identical values at $(\mathbf{x}, \sigma)$ and $(s\mathbf{x}, s\sigma)$, and its extrema move exactly with the zoom. (For an n -th derivative, normalise by σ^n .) Without normalisation, responses decay with σ and the “best” scale would always be the smallest.

2.3.3.1 Implementation with a pyramid

Instead of computing f for larger and larger windows, keep the window **fixed** and apply it to a **Gaussian pyramid** (the image at shrinking resolutions). Sometimes in-between levels are needed (e.g. a $\frac{3}{4}$ -size image) for finer scale sampling.

Instead of computing f for larger and larger windows, we can implement using a fixed window size with a Gaussian pyramid.

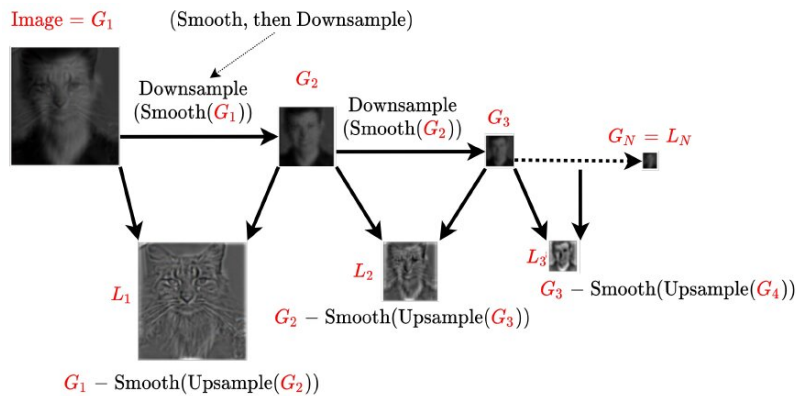


Sometimes need to create in-between levels, e.g., a $\frac{3}{4}$ -size image.

Credit: R. Urtasun

Figure 2.3.5: (slide) Fixed window, shrinking image. Credit: R. Urtasun.

2.3.4 Gaussian and Laplacian Pyramids



Credit: Derek Hoiem

Figure 2.3.6: (slide) Gaussian and Laplacian pyramids. Credit: D. Hoiem.

Key Idea: Construction

Gaussian pyramid: $G_0 = I$; $G_{i+1} = \text{Downsample}(\text{Smooth}(G_i))$ (“smooth, then down-sample”).

Laplacian pyramid: $L_i = G_i - \text{Smooth}(\text{Upsample}(G_{i+1}))$; the top level is $L_N = G_N$.

Sizes: G_i $[\frac{H}{2^i} \times \frac{W}{2^i}]$; L_i has the same size as G_i .

Derivation: Perfect reconstruction

Rearranging, $G_i = L_i + \text{Smooth}(\text{Upsample}(G_{i+1}))$. Starting from $G_N = L_N$ and applying this recursively down to $i = 0$ recovers $I = G_0$ *exactly* (the same Smooth/Upsample operator must be used in analysis and synthesis). Each L_i is a **band-pass** image: G_i contains frequencies up to its Nyquist limit; subtracting the upsampled coarser level removes the lower half, leaving roughly one octave. Hence the name: the Laplacian pyramid is approximately a stack of *difference-of-Gaussian* (i.e. LoG-like) filtered images.

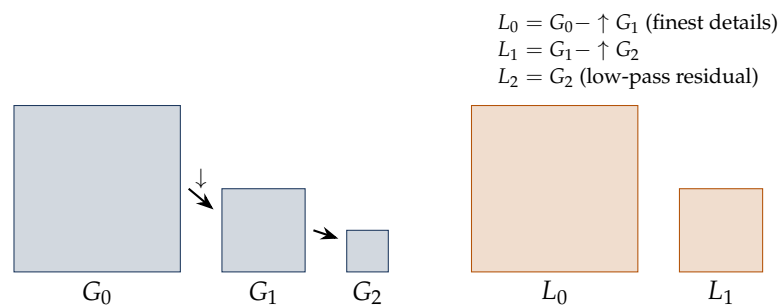


Figure 2.3.7: Gaussian pyramid (blurred copies) vs. Laplacian pyramid (band-pass details). Total pixels $\approx \frac{4}{3}$ of the original.

2.3.5 Uses of Image Pyramids

- **Compression** (Burt & Adelson 1983): Laplacian levels are sparse (mostly near zero) and quantise well.
- **Object detection:** scale search (run a fixed-size detector at every level); multi-scale features.
- **Detecting stable interest points** (SIFT, next module).
- **Registration — coarse-to-fine image registration:**
 1. Compute the Gaussian pyramid.
 2. Align at the coarsest level (small image $\Rightarrow$ a large motion becomes a small one; brute-force search is cheap).
 3. Successively align with finer levels, using the upsampled estimate as initialisation, *searching a smaller range*.

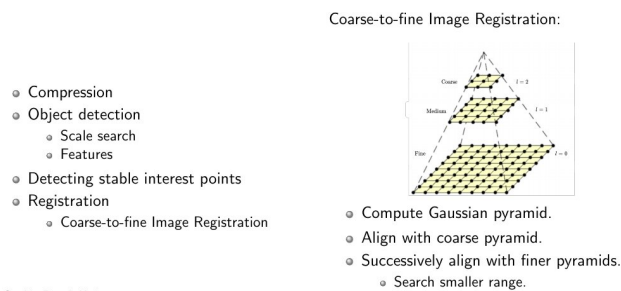


Figure 2.3.8: (slide) Coarse-to-fine registration through the pyramid. Credit: D. Hoiem.

Derivation: Why coarse-to-fine saves work

A displacement of d pixels at full resolution is $d/2^L$ pixels at level L . Exhaustive search over $\pm d$ costs $O(d^2)$ per pixel at full resolution but only $O((d/2^L)^2)$ at the top, plus $O(1)$ refinement (± 1 pixel) at each of the L finer levels. E.g. $d = 32$, $L = 4$: $65^2 = 4225$ candidates vs. $5^2 + 4 \times 3^2 = 61$.

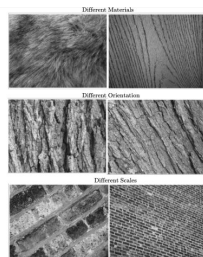
2.3.6 Texture in Images

Textures are regular or stochastic patterns caused by bumps, grooves and/or markings. They give information about the **spatial arrangement** of colours or intensities.

Textures:

- Regular or stochastic patterns caused by bumps, grooves and/or markings.
- Gives us information about spatial arrangement of colors or intensities in an image.

Credit: Derek Hoiem

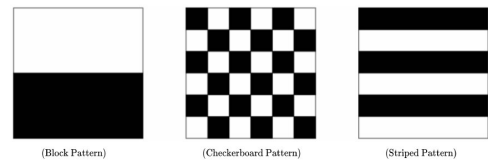


(slide) Regular and stochastic textures.

Credit: D. Hoiem.

Conveys more information that can be exploited to match regions of interest in images.

Histogram conveys 50% white pixels and 50% black pixels



Drastically different textures

Credit: Linda G Shapiro

(slide) Block, checkerboard and striped patterns: all 50% white/50% black (identical histograms), yet drastically different textures. Credit: L. Shapiro.

Intuition

Histograms (Week 1) are blind to spatial arrangement. Texture *is* arrangement. So we need descriptors built from *local filter responses*.

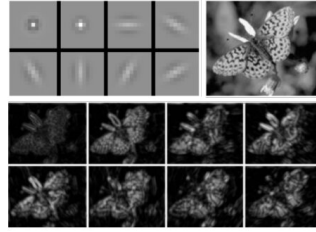
How to represent textures? Compute responses of blobs and edges at various **orientations** and **scales**. Ways to process:

- Record simple **statistics** (mean, std.) of absolute filter responses over a region.
- Take the **vector of filter responses at each pixel** and **cluster** them (the cluster centres are “textons”).

2.3.7 Filter Banks

An array of **band-pass filters** that separates the input signal into multiple components, each carrying a single frequency sub-band of the original signal. Process the image with each filter and keep the responses (or squared/absolute responses).

- An array of bandpass filters that separates the input signal into multiple components, each one carrying a single frequency sub-band of the original signal.
- Process image with each filter and keep responses (or squared/abs responses).



Credit: Derek Hoiem

Figure 2.3.9: (slide) A filter bank (edges and bars at several orientations, spots at several scales) and the responses on an image. Credit: D. Hoiem.

Key Idea: Filter bank $\Rightarrow$ feature vector per pixel

With K filters $F_1, \dots, F_K$ the image becomes a stack $[|I * F_1|, \dots, |I * F_K|]$ $[H \times W \times K]$. Each pixel now has a K -dimensional texture descriptor. *This is exactly the output of a convolutional layer with K channels* — a CNN learns its filter bank.

2.3.8 Gabor Filters

- A special class of **band-pass** filters (allow a certain band of frequencies and reject others).
- A Gabor filter = a **sinusoid of a particular frequency and orientation, modulated by a Gaussian**.

- Special classes of bandpass filters (i.e., they allow a certain 'band' of frequencies and reject the others).
- A Gabor filter can be viewed as a sinusoidal signal of particular frequency and orientation, modulated by a Gaussian wave.

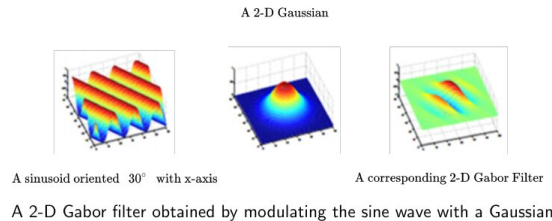


Figure 2.3.10: (slide) A sinusoid oriented at $30^\circ \times$ a 2D Gaussian = a 2D Gabor filter.

Key Idea: 2D Gabor filter

$$g(x, y; \lambda, \theta, \psi, \sigma, \gamma) = \exp\left(-\frac{x'^2 + \gamma^2 y'^2}{2\sigma^2}\right) \exp\left(i\left(2\pi \frac{x'}{\lambda} + \psi\right)\right),$$

$$x' = x \cos \theta + y \sin \theta, \quad y' = -x \sin \theta + y \cos \theta.$$

- θ orientation of the *normal* to the parallel stripes
- λ wavelength of the sinusoid (frequency $1/\lambda$)
- ψ phase offset (0: even/cosine; $\pi/2$: odd/sine)
- σ std. of the Gaussian envelope
- γ spatial aspect ratio (ellipticity of the support)

The real part (cosine) is an even “bar” detector; the imaginary part (sine) is an odd “edge” detector. (x', y') is the coordinate frame rotated by θ .

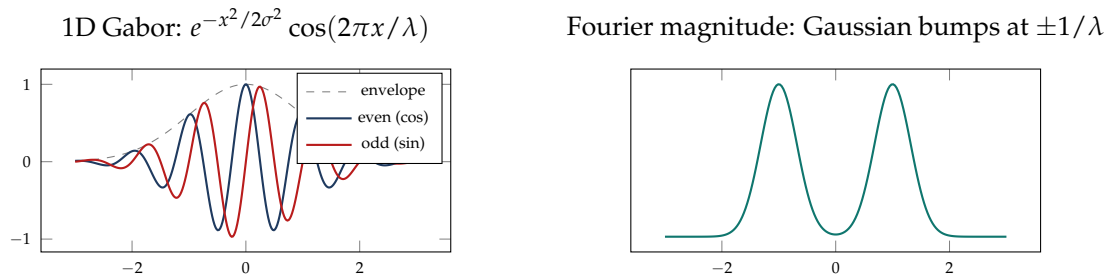
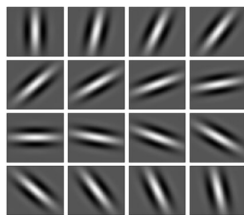


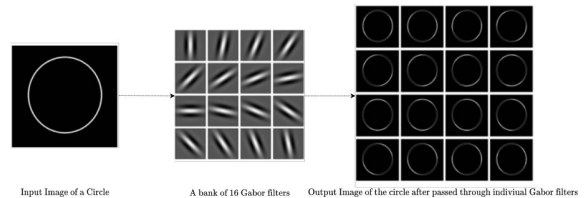
Figure 2.3.11: Why a Gabor is band-pass: multiplying by $\cos(2\pi x/\lambda)$ shifts the Gaussian's spectrum to $\pm 1/\lambda$ (modulation theorem). σ sets the bandwidth, λ the centre frequency, θ the direction in the 2D frequency plane.

2.3.8.1 Gabor filter banks



Bank of 16 Gabor filters at an orientation of 11.250 (i.e. if the first filter is at 00.00, then the second will be at 11.25, the third will be at 22.50, and so on.)

(slide) A bank of 16 Gabor filters at orientations $0^\circ, 11.25^\circ, 22.5^\circ, \dots$ (16 steps of 11.25° cover 180°).



(slide) A circle filtered by a bank of 16 Gabors: each filter responds only to the part of the boundary at its orientation.

Intuition: Gabors and biology

Gabor functions are excellent models of V1 simple-cell receptive fields (Module 2.7), and the first-layer filters learned by CNNs such as AlexNet look strikingly like Gabors. They achieve the minimum possible joint uncertainty in space and frequency.

2.3.9 Steerable Filters

Steerable filters are oriented filters that can be expressed as a **linear combination of a set of basis filters**. For an isotropic Gaussian $G(x, y) = e^{-(x^2+y^2)}$,

$$G_1^\theta = G_1^{0^\circ} \cos \theta + G_1^{90^\circ} \sin \theta$$

where G_1^θ is the first derivative of G in direction θ .

Derivation: Proof

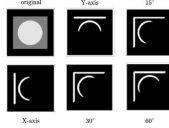
The directional derivative in direction $\mathbf{u} = (\cos \theta, \sin \theta)$ is $\nabla G \cdot \mathbf{u} = G_x \cos \theta + G_y \sin \theta$, and $G_1^{0^\circ} = G_x$, $G_1^{90^\circ} = G_y$. By linearity of convolution, $I * G_1^\theta = \cos \theta (I * G_x) + \sin \theta (I * G_y)$: **filter twice, then get the response at any orientation for free**. Maximising over θ : $\theta^* = \text{atan2}(I * G_y, I * G_x)$ and the max response is $\sqrt{(I * G_x)^2 + (I * G_y)^2}$ — the gradient magnitude. Higher derivatives need more basis filters (n -th derivative: $n + 1$ bases).

Steerable Filters are a class of oriented filters that can be expressed as a linear combination of a set of basis filters.

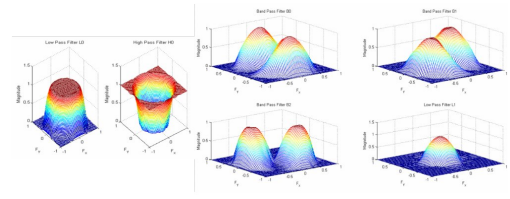
- For an isotropic Gaussian filter, $G(x, y) = e^{-(x^2+y^2)}$,

$$G_1^\theta = G_1^0 \cos(\theta) + G_1^{90} \sin(\theta)$$

where G_1^θ is the first derivative of G at angle θ .



(slide) A disc filtered with $G_1^{0^\circ}$, $G_1^{90^\circ}$ and a synthesised $G_1^{30^\circ}$.



(slide) Steerable basis filters and synthesised oriented filters.

2.3.10 Homework Questions (with answers)

Readings

- Chapter 2, Szaliski, *Computer Vision: Algorithms and Applications*

Questions

- Why is camouflage attire effective? How?
- How is texture different from noise?
- Will scale-invariant filters be effective in matching pictures containing Matryoshka (or Russian nesting) dolls?



Figure 2.3.12: (slide) Homework questions.

- Why is camouflage attire effective?** Its texture matches the background's filter-response statistics (similar orientations, scales and colours), and its blotches break up the wearer's outline so there are no long, consistent edges to group into a human silhouette.
- How is texture different from noise?** Texture has *structure*: repeated or statistically stationary patterns with spatial correlation (energy concentrated in particular orientation/frequency bands). i.i.d. noise is uncorrelated between pixels with a flat spectrum.
- Will scale-invariant filters work on Matryoshka dolls?** They will fire at many scales with nearly identical local appearance (each doll is a scaled copy), so matching becomes *ambiguous*: a keypoint on the big doll may match the corresponding point on a smaller doll. Scale invariance is a feature for repeatability but a liability when the scene itself is self-similar across scales.

Module Summary: §2.3

For scale invariance, find extrema in position *and* scale of a scale-normalised response ($\sigma^2 \nabla^2 G$); the characteristic scale follows the image zoom. Implement with a Gaussian pyramid ($G_{i+1} = \downarrow \text{Smooth}(G_i)$); Laplacian pyramid $L_i = G_i - \text{Smooth}(\uparrow G_{i+1})$ is band-pass and exactly invertible. Uses: compression, multi-scale detection, interest points, coarse-to-fine registration. Texture = spatial arrangement, captured by filter banks (statistics or clustered responses). Gabor = Gaussian $\times$ sinusoid (params $\lambda, \theta, \psi, \sigma, \gamma$), band-pass and V1-like. Steerable: $G_1^\theta = \cos \theta G_1^0 + \sin \theta G_1^{90}$.

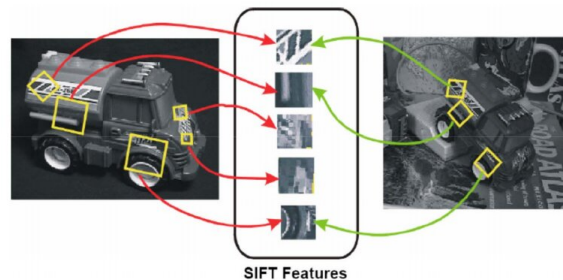
Feature Detectors: SIFT and Variants

2.4.1 SIFT: Scale-Invariant Feature Transform

David G. Lowe, *Distinctive Image Features from Scale-Invariant Keypoints*, IJCV 2004 (over 50,000 citations).

- Transforms image data into **scale-invariant coordinates** relative to local features.
- Fundamental to many core vision problems: recognition, motion tracking, multi-view geometry (stitching, SfM).
- Image content is transformed into local feature coordinates that are invariant to **translation, rotation, scale** (and robust to shear/affine distortion and illumination).

Image content is transformed into local feature coordinates that are invariant to translation, rotation, scale, shear.



Credit: Mubashir Shah, University of Central Florida

Figure 2.4.1: (slide) SIFT features: each keypoint carries a location, scale and orientation, defining a local frame. Credit: M. Shah, UCF.

Key Idea: The four stages

1. **Scale-space extrema detection:** detect interesting points (invariant to scale and orientation) using the **DoG**.
2. **Keypoint localisation:** determine location and scale at each candidate and select them based on **stability**.
3. **Orientation estimation:** use local gradients to assign an orientation to each keypoint; preserve orientation, scale and location for each feature.
4. **Keypoint descriptor:** extract local gradients at the selected scale around the keypoint and form a representation invariant to local shape and illumination distortion.

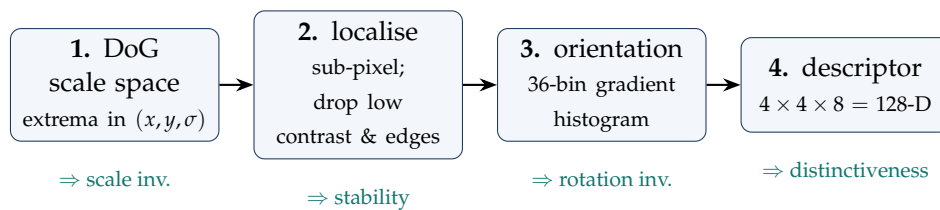


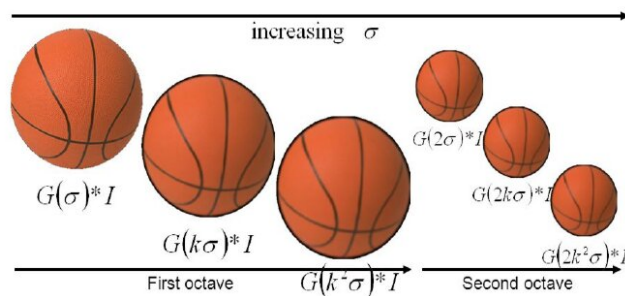
Figure 2.4.2: SIFT at a glance: stages 1–3 make the *detector* (where/how big/which way), stage 4 is the *descriptor* (what it looks like).

2.4.2 Step 1: Scale-Space Extrema Detection

2.4.2.1 Constructing the scale space

Blur the image with Gaussians of increasing σ : $\hat{I}(x, y, \sigma) = G(x, y, \sigma) * I(x, y)$. Scales are grouped into **octaves**: within an octave σ is multiplied by a constant k each step; after σ doubles, the image is downsampled by 2 and the next octave begins.

Constructing Scale Space



Credit: Ofir Pele

Figure 2.4.3: (slide) Constructing scale space: $G(\sigma) * I, G(k\sigma) * I, G(k^2\sigma) * I, \dots$; first and second octave. Credit: O. Pele.

2.4.2.2 Difference of Gaussians (DoG)

$$D(x, y, \sigma) = \hat{I}(x, y, k\sigma) - \hat{I}(x, y, \sigma) \quad \hat{I}(x, y, \sigma) = G(x, y, \sigma) * I(x, y).$$

Pitfall / Careful

The slide writes $D = \hat{I}(\sigma) - \hat{I}(k\sigma)$, which is just $-1 \times$ Lowe's definition; since we look for both maxima *and* minima, the sign does not matter.

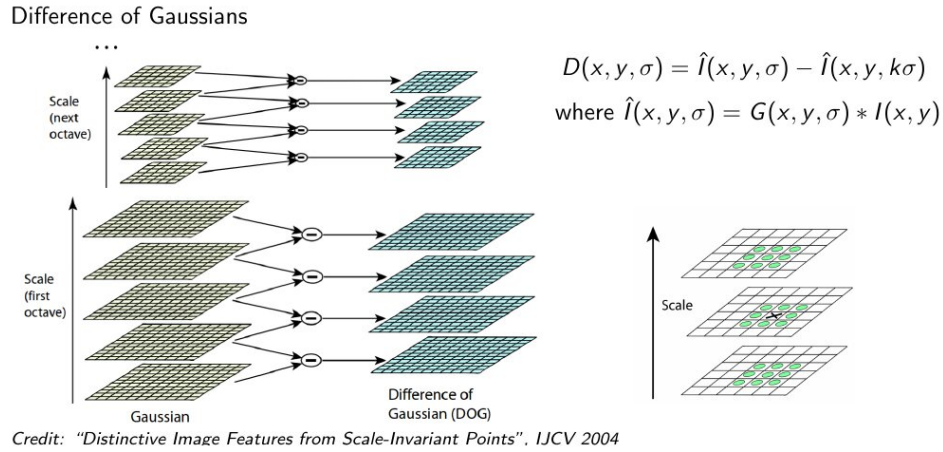


Figure 2.4.4: (slide) Adjacent Gaussian-blurred images are subtracted to give the DoG stack, octave by octave. Credit: Lowe, IJCV 2004.

Derivation: DoG approximates the scale-normalised LoG

The Gaussian satisfies the heat (diffusion) equation $\frac{\partial G}{\partial \sigma} = \sigma \nabla^2 G$ (check: differentiate $\frac{1}{2\pi\sigma^2} e^{-r^2/2\sigma^2}$ w.r.t. σ and compare with $\nabla^2 G = \frac{r^2 - 2\sigma^2}{\sigma^4} G$). By a finite difference in σ :

$$\sigma \nabla^2 G = \frac{\partial G}{\partial \sigma} \approx \frac{G(k\sigma) - G(\sigma)}{k\sigma - \sigma} \implies \boxed{G(k\sigma) - G(\sigma) \approx (k-1) \sigma^2 \nabla^2 G}.$$

So the DoG is (up to the *constant* factor $k-1$) exactly the scale-normalised LoG of §2.3 — scale invariance for free, and each DoG costs only one subtraction.

Key Idea: Sampling parameters (Lowe 2004)

s intervals per octave $\Rightarrow k = 2^{1/s}$ (Lowe uses $s = 3$). Need $s + 3$ Gaussian images per octave to get $s + 2$ DoGs, so that extrema can be searched in s DoG layers (each needs a neighbour above and below). Initial $\sigma_0 = 1.6$; the image is first upsampled $\times 2$ to get more keypoints.

2.4.2.3 Finding extrema

Compare a pixel X with its **26 neighbours**: 8 in its own DoG image, 9 in the scale above and 9 in the scale below. Select X if it is larger (or smaller) than all 26.

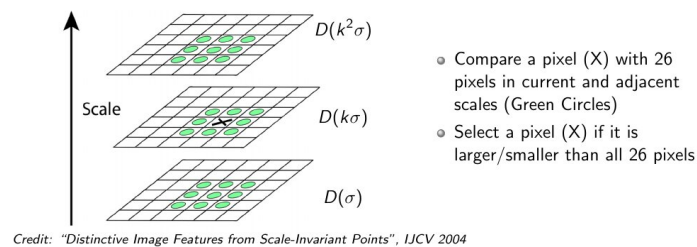


Figure 2.4.5: (slide) The 26-neighbour test across $D(\sigma)$, $D(k\sigma)$, $D(k^2\sigma)$. Credit: Lowe, IJCV 2004.

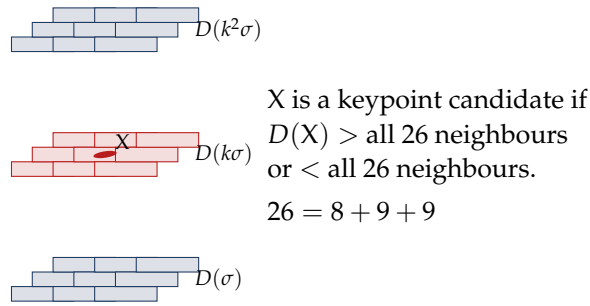
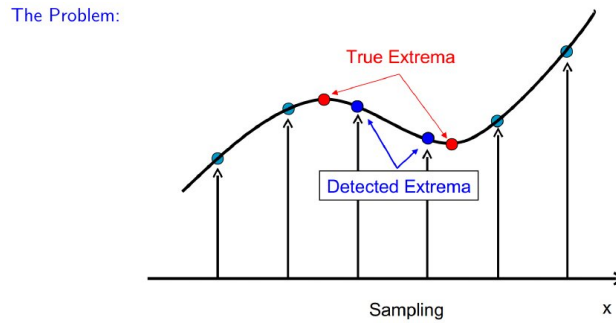


Figure 2.4.6: Extremum test in the $3 \times 3 \times 3$ neighbourhood of (x, y, σ) .

2.4.3 Step 2: Keypoint Localisation

The problem: extrema are detected on a discrete grid (pixels, and a few scales per octave). The true extremum lies *between* samples.



Credit: Ofir Pele

Figure 2.4.7: (slide) Detected (sampled) extremum vs. true extremum. Credit: O. Pele.

The solution: fit a quadratic via a second-order Taylor expansion of the scale-space function around the sample $\mathbf{s}_0 = (x_0, y_0, \sigma_0)^\top$, with offset $\Delta\mathbf{s} = (\delta x, \delta y, \delta\sigma)^\top$:

$$D(\mathbf{s}_0 + \Delta\mathbf{s}) = D(\mathbf{s}_0) + \frac{\partial D}{\partial \mathbf{s}} \Big|_{\mathbf{s}_0}^\top \Delta\mathbf{s} + \frac{1}{2} \Delta\mathbf{s}^\top \frac{\partial^2 D}{\partial^2 \mathbf{s}} \Big|_{\mathbf{s}_0} \Delta\mathbf{s}.$$

Derivation: The sub-pixel/sub-scale offset

Set the derivative w.r.t. $\Delta\mathbf{s}$ to zero: $\frac{\partial D}{\partial \mathbf{s}} + \frac{\partial^2 D}{\partial^2 \mathbf{s}} \Delta\mathbf{s} = \mathbf{0}$, so

$$\hat{\mathbf{s}} = - \left(\frac{\partial^2 D}{\partial^2 \mathbf{s}} \Big|_{\mathbf{s}_0} \right)^{-1} \frac{\partial D}{\partial \mathbf{s}} \Big|_{\mathbf{s}_0} \quad \frac{\partial D}{\partial \mathbf{s}} \text{ [3]}, \quad \frac{\partial^2 D}{\partial^2 \mathbf{s}} \text{ [3x3]} \text{ (Hessian in } x, y, \sigma).$$

This is one Newton step. All derivatives are finite differences of neighbouring samples. If any component of $\hat{\mathbf{s}}$ exceeds 0.5, the extremum is closer to a neighbouring sample: move there and repeat. Substituting back gives the interpolated value $D(\hat{\mathbf{s}}) = D(\mathbf{s}_0) + \frac{1}{2} \frac{\partial D}{\partial \mathbf{s}}^\top \hat{\mathbf{s}}$ (because $\frac{1}{2} \hat{\mathbf{s}}^\top H \hat{\mathbf{s}} = -\frac{1}{2} \mathbf{g}^\top \hat{\mathbf{s}}$).

Next, reject **low-contrast points** and points that lie on **edges**.

2.4.3.1 Low-contrast elimination

Reject the keypoint if $|D(\hat{\mathbf{s}})| < 0.03$ (image values normalised to $[0, 1]$). Low-contrast extrema are dominated by noise and are not repeatable.

2.4.3.2 Edge elimination

The DoG responds strongly along edges, but a point on an edge is poorly localised along the edge (aperture problem again). Reject points with a strong response in *one direction only*. **How?** Similar to Harris, but SIFT uses the 2×2 **Hessian** of D (principal curvatures):

$$H = \begin{bmatrix} D_{xx} & D_{xy} \\ D_{xy} & D_{yy} \end{bmatrix}, \quad \alpha = \lambda_{\max}, \beta = \lambda_{\min},$$

$$\text{Tr}(H) = D_{xx} + D_{yy} = \alpha + \beta, \quad \text{Det}(H) = D_{xx}D_{yy} - D_{xy}^2 = \alpha\beta.$$

Derivation: The ratio test

Let $r = \alpha/\beta \geq 1$. Then

$$\frac{\text{Tr}(H)^2}{\text{Det}(H)} = \frac{(\alpha + \beta)^2}{\alpha\beta} = \frac{(r\beta + \beta)^2}{r\beta^2} = \frac{(r + 1)^2}{r}$$

which depends only on the *ratio* of eigenvalues. $\frac{d}{dr} \frac{(r+1)^2}{r} = \frac{(r+1)(r-1)}{r^2} \geq 0$ for $r \geq 1$, so it is **minimal** (= 4) at $r = 1$ (equal curvatures, blob/corner-like) and increases with r (edge-like). Reject the keypoint if

$$\frac{\text{Tr}(H)^2}{\text{Det}(H)} > \frac{(r_{\text{th}} + 1)^2}{r_{\text{th}}}, \quad r_{\text{th}} = 10 \Rightarrow \text{threshold} = \frac{121}{10} = 12.1.$$

Also reject if $\text{Det}(H) < 0$ (curvatures of opposite sign: a saddle, not an extremum). No eigen-decomposition needed.

2.4.3.3 The effect of each filter



From 233x189 original image to 832 DoG Extrema

Credit: Mubarak Shah, University of Central Florida

(slide) 233×189 image $\rightarrow$ 832
DoG extrema.



From 729 keypoints to 536 keypoints after testing ratio based on Hessian

Credit: Mubarak Shah, University of Central Florida

(slide) $832 \rightarrow 729$ after contrast
threshold $\rightarrow$ 536 after Hessian
ratio test. Credit: M. Shah.

2.4.4 Step 3: Orientation Estimation

Why? To achieve **rotation invariance**: the descriptor will be computed in a frame rotated to this orientation.

Use the keypoint's scale to choose the correct blurred image $\hat{I}(x, y) = G(x, y, \sigma) * I(x, y)$, then compute gradient magnitude and orientation with finite differences:

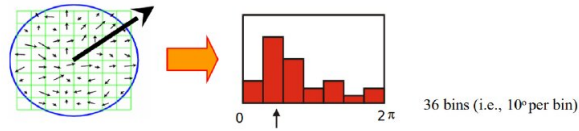
$$m(x, y) = \sqrt{(\hat{I}(x+1, y) - \hat{I}(x-1, y))^2 + (\hat{I}(x, y+1) - \hat{I}(x, y-1))^2},$$

$$\theta(x, y) = \tan^{-1} \frac{\hat{I}(x, y+1) - \hat{I}(x, y-1)}{\hat{I}(x+1, y) - \hat{I}(x-1, y)}.$$

- Build a **histogram of gradient directions** (36 bins of 10° covering 360°) in a region around the keypoint at the selected scale.

- Each sample is weighted by its **gradient magnitude** and by a **Gaussian** with σ equal to **1.5** $\times$ **the keypoint's scale**.
- Select the **peak** as the keypoint's direction.
- Introduce **additional keypoints** at the same location (and scale) for any other peak within **80%** of the maximum (about 15% of keypoints get multiple orientations; they contribute significantly to matching stability).

- Create histogram of gradient directions, within a region around the keypoint, at selected scale:



- Histogram entries are weighted by:
 - gradient magnitude, and
 - a Gaussian function with σ equal to 1.5 times scale of the keypoint
- Select **the peak as direction of keypoint**
- Introduce additional key points at same location if another peak is within 80% of max peak of histogram with different direction

Credit: Svetlana Lazebnik, UIUC

Figure 2.4.8: (slide) Orientation histogram (36 bins) and its dominant peak. Credit: S. Lazebnik.

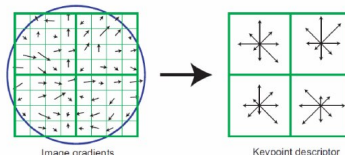
Key Idea: Peak refinement (Lowe)

Fit a parabola to the peak bin and its two neighbours (h_{-1}, h_0, h_{+1}); the peak offset in bins is $\delta = \frac{h_{-1} - h_{+1}}{2(h_{-1} - 2h_0 + h_{+1})}$, giving an orientation finer than 10° .

2.4.5 Step 4: Keypoint Descriptor

- Compute the gradient at each pixel in a 16×16 window around the keypoint, at the Gaussian-pyramid level where it was detected, *in coordinates rotated to the keypoint orientation*.
- Down-weight gradients by a **Gaussian fall-off** (blue circle; σ = half the window width) to reduce the influence of distant gradients (and of small registration errors).
- In each 4×4 quadrant, compute a **gradient-orientation histogram with 8 bins**.

- Compute gradient at each pixel in a 16×16 window around the detected keypoint, using the appropriate level as detected.



- Downweight gradients by a Gaussian fall-off function (blue circle) to reduce the influence of gradients far from the center.
- In each 4×4 quadrant, compute a gradient orientation histogram using 8 orientation histogram bins.

Credit: Raquel Urtasun, Szeliski

Figure 2.4.9: (slide) Image gradients in a 16×16 window (shown 8×8 for clarity) $\rightarrow$ 4×4 grid of 8-bin histograms (shown 2×2). Credit: R. Urtasun, Szeliski.

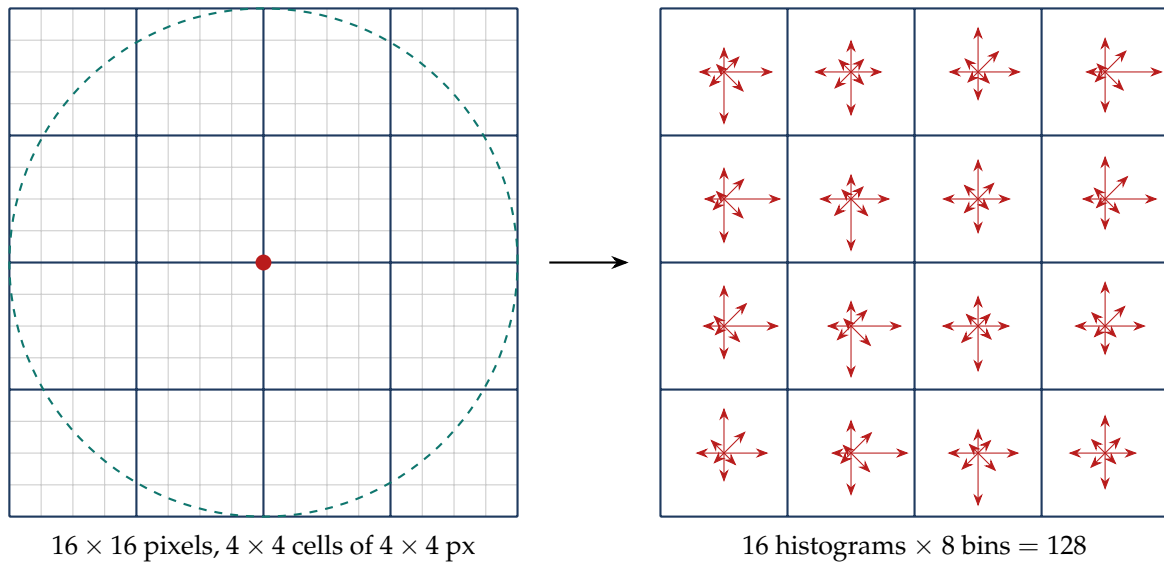


Figure 2.4.10: The SIFT descriptor layout: $\mathbf{d}$ [4·4·8=128].

2.4.5.1 Normalisation

1. The 128 non-negative values form the **raw** SIFT vector.
2. To reduce the effects of **contrast or gain** (multiplicative changes; additive ones are already removed by using gradients), **normalise to unit length**.
3. To be robust to other photometric variations (non-linear illumination, saturation, 3D lighting effects), **clip values at 0.2** and **renormalise** to unit length.

Derivation: Why each step gives the invariance it claims

Gain $I \rightarrow aI$ scales every gradient by a : $\mathbf{d} \rightarrow a\mathbf{d}$, and $\frac{a\mathbf{d}}{\|a\mathbf{d}\|} = \frac{\mathbf{d}}{\|\mathbf{d}\|}$. Offset $I \rightarrow I + b$: gradients unchanged. Clipping at 0.2 limits the influence of any single large gradient magnitude (e.g. a specular highlight), putting more emphasis on the *distribution of orientations* than on magnitudes.

Key Idea: Details worth knowing (Lowe 2004)

Each gradient sample contributes to its neighbouring spatial cells and orientation bins by **trilinear interpolation**, avoiding abrupt changes when a sample shifts across a bin boundary. Matching uses nearest neighbour in Euclidean distance with the **ratio test**: accept if $\frac{d_1}{d_2} < 0.8$ (distance to the nearest over the second-nearest descriptor), which eliminates about 90% of false matches while discarding under 5% of correct ones.

2.4.6 SIFT in Practice

Extraordinarily **robust** feature detection:

- Changes in viewpoint: up to about **60° out-of-plane rotation**.
- Changes in illumination: sometimes even day vs. night.
- Fast and efficient — can run in real time.

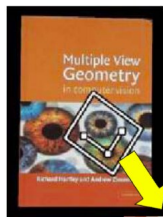
- Extraordinarily robust feature detection
- Changes in viewpoint: up to about 60 degree out of plane rotation
- Changes in illumination: sometimes even day vs night (below)
- Fast and efficient – can run in real-time



Credit: Raquel Urtasun, Szeliski



(slide) Illumination robustness examples. Credit: R. Urtasun, Szeliski.



e.g. scale,
translation,
rotation

Credit: Raquel Urtasun, Tinne Tuytelaars

(slide) Invariance to geometric transformations (scale, translation, rotation).



Mars Rover images with SIFT feature matches

Credit: Raquel Urtasun, N Snavely

(slide) Mars Rover images: hard for humans ("look for tiny squares...?"), SIFT finds the matches. Credit: R. Urtasun, N. Snavely.

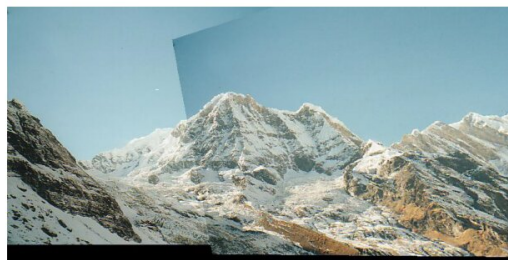


Credit: Raquel Urtasun, Tinne Tuytelaars

(slide) Invariance to photometric transformations. Credit: R. Urtasun, T. Tuytelaars.

2.4.6.1 Application: image stitching

1. Detect feature points in both images.
2. Find corresponding pairs of feature points (descriptor matching + ratio test).
3. Use the pairs to align the images (estimate a homography, robustly with RANSAC).



- Detect feature points in both images.
- Find corresponding pairs of feature points.
- Use the pairs to align the images.

Credit: Raquel Urtasun

Figure 2.4.11: (slide) Panorama from matched SIFT features. Credit: R. Urtasun.

2.4.7 SURF: Speeded-Up Robust Features

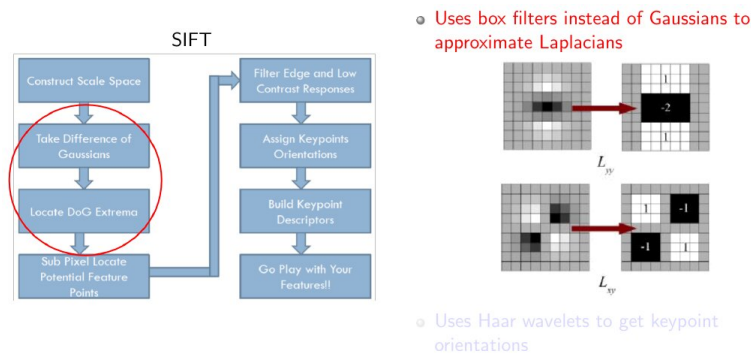


Figure 2.4.12: (slide) SURF modifies SIFT's pipeline: box filters approximate the Gaussian second derivatives (D_{yy} , D_{xy}).

- Uses **box filters instead of Gaussians** to approximate the Laplacian (Hessian) — computed in constant time with an **integral image**.
- Uses **Haar wavelets** to get keypoint orientations; Haar wavelets are simple ± 1 box filters giving gradients in x and y .
- Good at handling **blur and rotation**; **not as good as SIFT** on invariance to **illumination and viewpoint** changes.
- About **3× faster** than SIFT.

Derivation: Integral image: any box sum in 4 lookups

Define $S(x, y) = \sum_{x' \leq x, y' \leq y} I(x', y')$, computed in one pass via $S(x, y) = I(x, y) + S(x - 1, y) + S(x, y - 1) - S(x - 1, y - 1)$. Then the sum over the rectangle $(x_0, x_1] \times (y_0, y_1]$ is

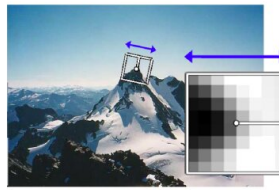
$$S(x_1, y_1) - S(x_0, y_1) - S(x_1, y_0) + S(x_0, y_0),$$

independent of the box size. So SURF's filters cost the same at every scale — the scale space is built by *enlarging the filter*, not shrinking the image. SURF's detector uses the Hessian determinant $\det H_{\text{approx}} = D_{xx}D_{yy} - (0.9D_{xy})^2$.

2.4.8 MOPS: Multi-Scale Oriented Patches

Making the descriptor rotation-invariant in the simplest way: **rotate the patch according to its dominant gradient orientation**, putting patches into a canonical orientation; then sample a small (8×8) patch from a coarser level (a 40×40 window, spacing 5), normalise intensities (bias/gain) and compare patches directly.

- Multiscale **O**riented **P**atch**S** descriptor
- Rotate patch according to its dominant gradient orientation.
- This puts the patches into a canonical orientation



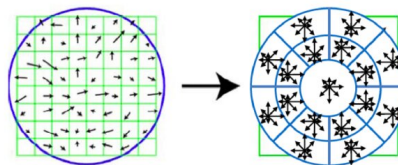
Credit: Matthew Brown, Kristen Grauman, Raquel Urtasun

Figure 2.4.13: (slide) MOPS: patch rotated to its dominant orientation. Credit: M. Brown, K. Grauman, R. Urtasun.

2.4.9 GLOH: Gradient Location-Orientation Histogram

A variant of SIFT that uses a **log-polar** binning structure instead of four quadrants: **17 spatial bins** (1 centre + 2 rings $\times$ 8 angular sectors) and **16 orientation bins** $\Rightarrow 17 \times 16 = 272$ -D histogram, projected onto **128-D** with PCA trained on a large dataset.

- Variant of SIFT that uses a log-polar binning structure instead of four quadrants.
- Uses 17 spatial bins and 16 orientation bins.
- The 272D histogram is then projected onto a 128D descriptor using PCA trained on a large dataset.



Credit: Matthew Brown, Kristen Grauman, Raquel Urtasun

Figure 2.4.14: (slide) GLOH: log-polar spatial bins. Credit: M. Brown, K. Grauman, R. Urtasun.

Method	Detector	Descriptor	Notes
Harris	$\det A - \kappa \operatorname{tr}^2 A$	(none)	rotation-inv., not scale-inv.
SIFT	DoG extrema in (x, y, σ)	128-D gradient histograms	scale+rotation inv., robust to illumination & $\sim 60^\circ$ viewpoint
SURF	Hessian det. with box filters	64-D Haar responses	$\sim 3\times$ faster; weaker on illumination/viewpoint
MOPS	multi-scale Harris	8×8 normalised oriented patch	simple, fast; less distinctive
GLOH	(as SIFT)	272-D log-polar $\rightarrow$ 128-D (PCA)	often more distinctive than SIFT

2.4.10 Homework Questions (with answers)

1. **Which descriptor performs better, SIFT or MOPS?** Generally **SIFT**: histograms of gradient orientations over 4×4 cells tolerate small misregistrations and local deformations, while MOPS compares raw (normalised) intensities of a small patch, which is sensitive to misalignment, blur and non-linear lighting. MOPS is faster and simpler.
2. **Why is SIFT better than the Harris detector?** Harris only *detects* points and is *not scale-invariant*; SIFT detects in scale space (scale-invariant), assigns orientation (rotation-invariant), refines location, filters unstable points, and provides a distinctive, illumination-robust *descriptor* for matching.

Module Summary: §2.4

SIFT = (1) extrema of DoG $D = G(k\sigma) * I - G(\sigma) * I \approx (k-1)\sigma^2 \nabla^2 G * I$ over 26 neighbours; (2) sub-pixel/scale refinement $\hat{s} = -H^{-1} \nabla D$, reject $|D(\hat{s})| < 0.03$ and edges with $\text{Tr}^2 / \text{Det} > (r+1)^2/r$ ($r = 10$); (3) orientation from a 36-bin magnitude- and Gaussian(1.5σ)-weighted histogram, extra keypoints for peaks $\geq 80\%$; (4) 16×16 window $\rightarrow 4 \times 4$ cells $\times 8$ bins = 128-D, unit-normalise, clip 0.2, renormalise. Variants: SURF (box filters/integral image/Haar, $3 \times$ faster), MOPS (oriented patches), GLOH (log-polar, $272 \rightarrow 128$ by PCA).

Image Segmentation

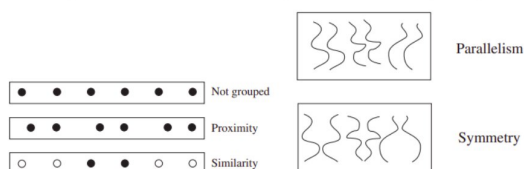
Segmentation = partitioning an image into coherent regions (pixels that “belong together”). It is a *grouping* problem, so we start with how humans group.

2.5.1 Human Vision: Gestalt and Grouping

- **Gestalt theory** (early 20th century): “*The whole is greater than the sum of its parts.*”
- It emphasised **grouping** as an important part of understanding human vision.
- The Müller-Lyer illusion illustrates the belief: we see the line *together with* its arrowheads as a group, not as independent components.

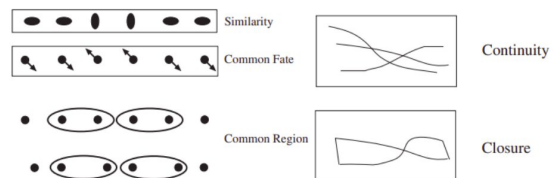
2.5.1.1 Gestalt grouping factors

• Gestalt theory proposes various factors in images which can lead to grouping:



Credit: David Forsyth

(slide) Not grouped, proximity, similarity; parallelism, symmetry.



Credit: David Forsyth

(slide) Similarity, common fate, common region; continuity, closure. Credit: D. Forsyth.

Proximity	Nearby elements are grouped.
Similarity	Elements with similar colour/shape/size are grouped.
Common fate	Elements moving together are grouped (key cue in video).
Common region	Elements inside the same enclosed region are grouped.
Parallelism	Parallel curves are grouped.
Symmetry	Symmetric pairs of curves are grouped.
Continuity	Smooth continuations are preferred over abrupt turns.
Closure	Curves that (nearly) close are grouped into a figure.

Pitfall / Careful: Limitation

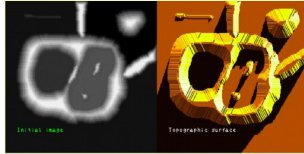
Gestalt theory is fairly *descriptive*: a loose set of rules explaining why some elements are grouped. The rules are **insufficiently defined to be used directly as algorithms**. Modern methods instead encode similarity (proximity + appearance) in affinity/graph weights. Further reading: Forsyth & Ponce §15.1.

2.5.2 Watershed Segmentation (1979)

- An early method; segments an image into several “**catchment basins**” or regions.
- Any grayscale image can be interpreted as a **3D topographic surface** (intensity = height).

- The image is segmented into regions where **rainwater would flow into the same lake**.
- Algorithm: **flood the landscape from local minima**, and **prevent merging** of water from different minima (build dams) $\Rightarrow$ partition into catchment basins separated by **watershed lines**.

- An early method for image segmentation (1979)
- Segments an image into several "catchment basins" or "regions"
- Any grayscale image can be interpreted as a 3D topological surface

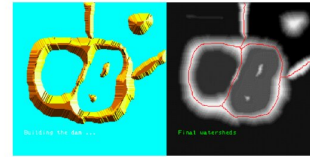


- Image can be segmented into regions where rainwater would flow into the same lake

Credit: S. Beucher

(slide) Grayscale image as a topographic surface.

- Flood the landscape from local minima and prevent merging of water from different minima



- Results in partitioning the image into **catchment basins** and **watershed lines**

Credit: S. Beucher

(slide) Flooding from minima; watershed lines where basins meet. Credit: S. Beucher.

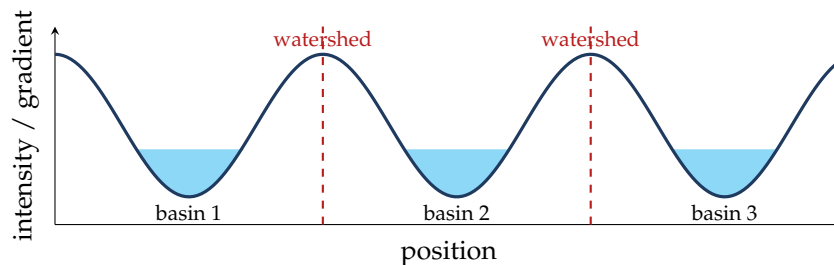
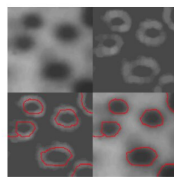


Figure 2.5.1: 1D watershed: water rising from each minimum fills a basin; where two basins would meet (at the ridges), a dam — the watershed line — is built.

- Generally applied on the **gradient image** rather than on the image itself: region interiors have low gradient (basins), boundaries high gradient (ridges).

Generally applied on image gradients instead of applying directly on images



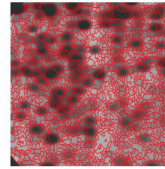
(Top left) Original image; (Top right) Gradient image; (Bottom left) Watersheds of gradient image; (Bottom right) Final segmentation output

Credit: S. Beucher

Figure 2.5.2: (slide) Original; gradient image; watersheds of the gradient; final segmentation. Credit: S. Beucher.

- In practice watershed often leads to **over-segmentation** (every tiny noise minimum becomes a basin).
- Hence usually used in an **interactive** system (*marker-controlled watershed*): the user marks "centres" of each component, and flooding is done only from those markers.

- In practice, often leads to over-segmentation due to noise and irregularities in image
- Hence usually used as part of an interactive system, where user marks "centers" of each component, on which flooding is done



Further Reading

- Chapter 5.2.1, Szeliski, *Computer Vision: Algorithms and Applications*

Figure 2.5.3: (slide) Over-segmentation from noise and irregularities. Further reading: Szeliski §5.2.1.

2.5.3 Region Splitting and Merging

- **Region splitting** methods split the image into successively finer regions (e.g. normalised cuts below; quadtree splitting).
- **Region merging** methods successively merge pixels into groups based on heuristics such as colour differences.
- Result: **superpixels** — small, compact, homogeneous regions, generally used as a **pre-processing step** for higher-level methods.

- **Region splitting methods** involve splitting the image into successively finer regions.
 - We'll discuss one such method in the upcoming slides
- **Region merging methods** successively merge pixels into groups based on various heuristics such as color differences
 - Figure on right shows an image segmented into such *superpixels*
 - Generally used as preprocessing step to higher-level segmentation algorithms

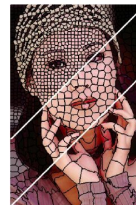


Image Credit: Achanta et al. SLIC Superpixels

Figure 2.5.4: (slide) SLIC superpixels. Image credit: Achanta et al.

2.5.4 Graph-Based Segmentation (Felzenszwalb & Huttenlocher, 2004)

A **region-merging** method that uses *relative* dissimilarities between regions to decide which to merge.

- Image = graph $G = (V, E)$: pixels are vertices, edges connect adjacent pixels (e.g. N_8 neighbours).
- Edge weight $w(e)$, $e = (v_1, v_2)$: a pixel-to-pixel **dissimilarity**, e.g. $|I(v_1) - I(v_2)|$ or colour distance.

- Felzenszwalb and Huttenlocher (2004) proposed a graph-based segmentation algorithm which uses *relative dissimilarities* between regions to decide which ones to merge (region-merging method)
- An image = graph $G = (V, E)$ where pixels form vertices V and edges E lie between adjacent pixels



- A pixel-to-pixel dissimilarity metric $w(e)$ is defined where edge $e = (v_1, v_2)$ and v_1, v_2 are two pixels. This measures, for instance, intensity differences between N_8 neighbors.

Figure 2.5.5: (slide) Pixels as graph nodes.

Key Idea: Definitions

Internal difference of region C — largest edge weight in its **minimum spanning tree**:

$$\text{Int}(C) = \max_{e \in \text{MST}(C)} w(e).$$

Minimum internal difference of two adjacent regions ($\tau(C)$ a region penalty, typically $\tau(C) = k/|C|$):

$$\text{MInt}(C_1, C_2) = \min (\text{Int}(C_1) + \tau(C_1), \text{Int}(C_2) + \tau(C_2)).$$

Difference between two adjacent regions — the minimum-weight edge connecting them:

$$\text{Dif}(C_1, C_2) = \min_{e=(v_1, v_2): v_1 \in C_1, v_2 \in C_2} w(e).$$

Predicate:

$$D(C_1, C_2) = \begin{cases} \text{true} & \text{if } \text{Dif}(C_1, C_2) > \text{MInt}(C_1, C_2) \quad (\text{boundary: keep separate}) \\ \text{false} & \text{otherwise} \quad (\text{merge}) \end{cases}$$

$\Rightarrow$ the algorithm **merges two regions whose difference is smaller than their minimum internal difference**.

◦ A predicate $D(C_1, C_2)$ for any two regions C_1 and C_2 is defined as:

$$D(C_1, C_2) = \begin{cases} \text{true,} & \text{if } \text{Dif}(C_1, C_2) > \text{MInt}(C_1, C_2) \\ \text{false,} & \text{otherwise} \end{cases}$$

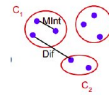


Figure 2.5.6: (slide) Comparing the between-region edge with the internal MST edges.

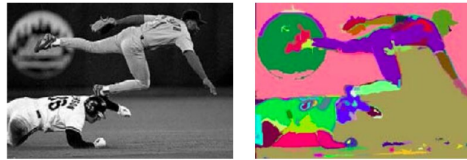
Intuition: Why “relative” dissimilarity works

A textured region (grass) has large internal differences, so it tolerates large between-region differences before declaring a boundary; a smooth region (sky) has tiny internal differences, so even a small jump is a boundary. The threshold *adapts to each region’s own variability*. $\tau = k/|C|$ makes small regions easy to merge (when $|C| = 1$, $\text{Int} = 0$ and only τ allows merging); larger $k \Rightarrow$ larger segments.

Key Idea: The algorithm (Kruskal-like, $O(|E| \log |E|)$)

1. Sort edges by increasing weight. Start with each pixel as its own region.
2. For each edge $e = (v_1, v_2)$ in order: if v_1, v_2 are in different regions C_1, C_2 and $w(e) \leq \text{MInt}(C_1, C_2)$, merge them (union-find); the new $\text{Int} = w(e)$ because edges come in increasing order.

Because edges are processed in increasing order, the first edge between two regions is $\text{Dif}(C_1, C_2)$.

Graph-based merging segmentation using N_8 pixel neighborhood

Further Reading

- Chapter 5.2.4, Szeliski, *Computer Vision: Algorithms and Applications*

Figure 2.5.7: (slide) Graph-based merging segmentation using an N_8 pixel neighbourhood. Further reading: Szeliski §5.2.4.

2.5.5 Probabilistic Aggregation (Alpert et al., 2007)

A probabilistic **bottom-up** merging algorithm aggregating two cues: **grey-level similarity** and **texture similarity**.

- Initially each pixel is a region; assign a **merging likelihood** p_{ij} (from intensity and texture similarity) to each pair of neighbouring regions.
- Given graph $G^{[s-1]} = (V^{[s-1]}, E^{[s-1]})$, construct $G^{[s]}$ by selecting a subset of **seed nodes** $C \subset V^{[s-1]}$; merge nodes that are **strongly coupled** to regions in C :

$$\frac{\sum_{j \in C} p_{ij}}{\sum_{j \in V} p_{ij}} > \text{threshold (usually 0.2)}.$$

- Once a segmentation is found at a coarser level, assignments are propagated to their finer-level “children”, followed by further coarsening.

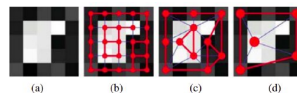


Figure 5.15 Coarse to fine node aggregation in segmentation by weighted aggregation (SWA) (Sharon, Galun, Sharon *et al.* 2006) © 2006 Macmillan Publishers Ltd [Nature]: (a) original gray-level pixel grid; (b) inter-pixel couplings, where thicker lines indicate stronger couplings; (c) after one level of coarsening, where each original pixel is strongly coupled to one of the coarse-level nodes; (d) after two levels of coarsening.

Further Reading

- Chapter 5.2.5, Szeliski, *Computer Vision: Algorithms and Applications*

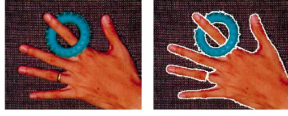
Image Credit: Szeliski

Figure 2.5.8: (slide) Coarse-to-fine node aggregation (Szeliski Fig. 5.15): original grey-level pixel grid, inter-pixel couplings (thicker = stronger), after one level of coarsening, after multiple levels. Further reading: Szeliski §5.2.5.

2.5.6 Mean-Shift Segmentation

- A **mode-finding** technique based on **non-parametric density estimation**.
- Feature vectors of each pixel (e.g. colour) are samples from an unknown probability distribution. Estimate the pdf non-parametrically and find its **modes**.
- Segment pixel-wise: all pixels that **climb to the same mode** form one segment.

- A mode-finding technique based on non-parametric density estimation
- Feature vectors of each pixel in the image are assumed to be samples from an unknown probability distribution

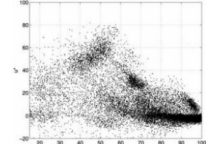


- We estimate p.d.f. using non-parametric estimation and find its *modes*.
- Image is segmented pixel-wise by considering every set of pixels which climb to the same mode as a consistent segment.

Image Credit: Szeliski

(slide) Mean-shift segmentation result. Image credit: Szeliski.

- Consider an example image below on the left. The graph on the right shows the distribution of L^*u^* features of each pixel (in the $L^*u^*v^*/CIELUV$ space¹)



- Our aim is to obtain modes of distribution on the right, without actually explicitly computing the density function! How to do this?

Image Credit: Comaniciu and Meer

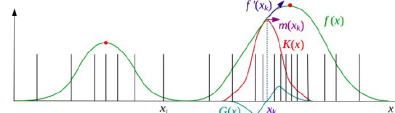
¹<https://en.wikipedia.org/wiki/CIELUV>

(slide) An image and its pixels' L^*u^* features (CIELUV): the clusters are density modes. Credit: Comaniciu & Meer.

Aim: find the modes of the distribution *without explicitly computing the density function*.

2.5.6.1 Kernel density estimate

- 1D visualization as an example, to illustrate the mode finding approach.



- Estimate the density function by convolving the data with kernel of width h , where k is the kernel function:

$$\hat{f}(x) = \sum_i k\left(\frac{\|x - x_i\|^2}{h^2}\right)$$

Image Credit: Szeliski

Figure 2.5.9: (slide) 1D illustration: the density $f(x)$ is a sum of kernels centred on the samples x_i ; mean shift climbs to its peaks. Image credit: Szeliski.

Estimate the density by convolving the data with a kernel of width h (k = kernel profile):

$$\hat{f}(x) = \sum_i k\left(\frac{\|x - x_i\|^2}{h^2}\right).$$

To find modes, mean shift uses **gradient ascent with multiple restarts**: pick a guess y_0 (e.g. a data point x_i), compute $\nabla f(y_0)$ and step uphill.

Derivation: Homework: gradient of the KDE and the mean-shift vector

Let $g(r) = -k'(r)$ and $G(x - x_i) = g\left(\frac{\|x - x_i\|^2}{h^2}\right)$. By the chain rule, $\nabla_x \frac{\|x - x_i\|^2}{h^2} = \frac{2(x - x_i)}{h^2}$, so

$$\nabla f(x) = \sum_i k'(\cdot) \frac{2(x - x_i)}{h^2} = \frac{2}{h^2} \sum_i (x_i - x) g\left(\frac{\|x - x_i\|^2}{h^2}\right).$$

(The slide drops the constant $2/h^2$, which does not affect the direction.) Split the sum:

$$\sum_i (x_i - x) G(x - x_i) = \sum_i x_i G(x - x_i) - x \sum_i G(x - x_i) = \left[\sum_i G(x - x_i) \right] \underbrace{\left(\frac{\sum_i x_i G(x - x_i)}{\sum_i G(x - x_i)} - x \right)}_{\mathbf{m}(x)}.$$

Hence

$$\nabla f(x) \propto \left[\sum_i G(x - x_i) \right] \mathbf{m}(x), \quad \mathbf{m}(x) = \frac{\sum_i x_i G(x - x_i)}{\sum_i G(x - x_i)} - x.$$

The scalar in brackets is positive, so the **mean-shift vector** $\mathbf{m}(x)$ points along the gradient: it is the difference between x and the G -weighted mean of its neighbours.

Key Idea: Mean-shift iteration

$$\mathbf{y}_{k+1} = \mathbf{y}_k + \mathbf{m}(\mathbf{y}_k) = \frac{\sum_i \mathbf{x}_i G(\mathbf{y}_k - \mathbf{x}_i)}{\sum_i G(\mathbf{y}_k - \mathbf{x}_i)}$$

Replace the current estimate by its *locally weighted mean*; repeat until $\|\mathbf{m}\| < \epsilon$. No step size to tune: the step adapts (large far from a mode, small near it). For the Epanechnikov profile $k(r) = 1 - r$ ($r \leq 1$), g is a flat window and the update is simply the **mean of the points within radius h** . For a Gaussian profile $k(r) = e^{-r^2/2}$, g is again Gaussian.

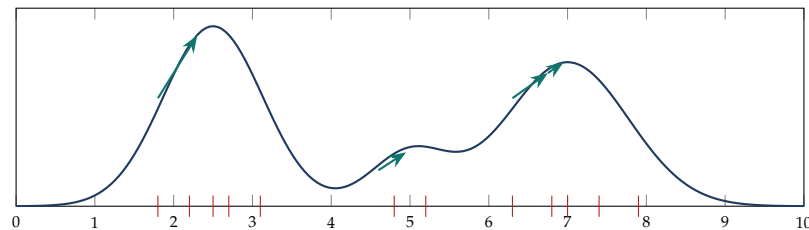


Figure 2.5.10: 1D mean shift: samples (red ticks), KDE (blue), and iterates (teal) climbing to the nearest mode. Points ending at the same mode form one cluster/segment.

2.5.6.2 Practical notes

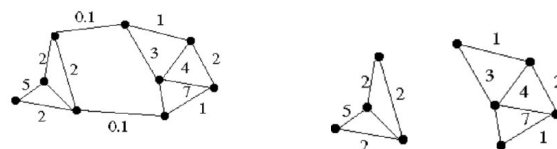
- Relies on selecting a suitable **kernel width h** (too small $\Rightarrow$ many tiny modes; too large $\Rightarrow$ merged segments).
- Purely colour-based features ignore layout; better results use feature vectors with **both colour and location**, e.g. (x, y, L^*, u^*, v^*) with separate bandwidths h_s (spatial) and h_r (range) — note the similarity to the bilateral filter.

Reading: Szeliski §5.3.2.

2.5.7 Normalised Cuts for Segmentation

A **region-splitting** method: a graph of pixels is successively split into parts. Edge weights w_{ij} measure **similarity** (affinity) between pixels. Split the graph in two by finding and deleting a **cut-set with minimum total weight** — a **min-cut**.

- **Region-splitting method** where a graph representing pixels in an image is successively split into parts
- Edge weights between pixels in graph measure their similarity



- Graph split into two parts by finding and deleting a **cut-set** with minimum sum of weights i.e., a **min-cut**

Image Credit: K Shafique

Figure 2.5.11: (slide) A weighted graph and its min-cut. Image credit: K. Shafique.

Key Idea: Cut and normalised cut

$$\text{cut}(A, B) = \sum_{i \in A, j \in B} w_{ij}, \quad A \cup B = V, A \cap B = \emptyset.$$

Min-cut alone yields trivial solutions such as **isolating a single pixel** (few edges to cut). The **normalised cut** fixes this:

$$\text{Ncut}(A, B) = \frac{\text{cut}(A, B)}{\text{assoc}(A, V)} + \frac{\text{cut}(A, B)}{\text{assoc}(B, V)}$$

where $\text{assoc}(A, V) = \text{assoc}(A, A) + \text{assoc}(A, B)$ is the sum of all weights attached to vertices in A , with $\text{assoc}(A, B) = \sum_{i \in A, j \in B} w_{ij}$.

Derivation: Why Ncut penalises tiny segments

If $A = \{p\}$ is a single pixel, $\text{assoc}(A, V) = \sum_j w_{pj} = \text{cut}(A, B)$ (all of p 's edges are cut), so the first term equals 1 — the maximum possible for that term. Balanced partitions with few cut edges make both ratios small. Ncut also relates to *normalised association*: $\text{Nassoc} = \frac{\text{assoc}(A, A)}{\text{assoc}(A, V)} + \frac{\text{assoc}(B, B)}{\text{assoc}(B, V)} = 2 - \text{Ncut}$, so minimising Ncut simultaneously *maximises within-group association*.

Key Idea: Solving it (Shi & Malik, 2000)

Minimising Ncut exactly is **NP-complete**. With W the affinity matrix and $D = \text{diag}(d_i)$, $d_i = \sum_j w_{ij}$, relax the indicator vector to real values:

$$\min_{\mathbf{y}} \frac{\mathbf{y}^\top (D - W) \mathbf{y}}{\mathbf{y}^\top D \mathbf{y}} \text{ s.t. } \mathbf{y}^\top D \mathbf{1} = 0 \Rightarrow (D - W) \mathbf{y} = \lambda D \mathbf{y}.$$

Take the eigenvector with the **second smallest** eigenvalue and threshold it to split; recurse on each part. ($D - W$ is the graph Laplacian.)

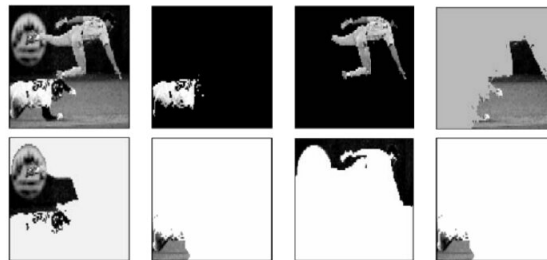


Image components returned by Normalized cuts algorithm

Image Credit: Shi and Malik

Figure 2.5.12: (slide) Image components returned by normalised cuts. Image credit: Shi & Malik.

2.5.8 Other Methods

- ***k*-means clustering** of pixel features (colour, or colour+position): fast, but needs k and assumes roughly spherical clusters.

- **Markov Random Fields and Conditional Random Fields:** energy $E(\mathbf{l}) = \sum_i \psi_i(l_i) + \sum_{(i,j)} \psi_{ij}(l_i, l_j)$ with a data term and a pairwise smoothness term, minimised e.g. by graph cuts.
- Many more (Szeliski Ch. 5).

2.5.9 Do We Still Need These?

With deep learning, do we still need classical segmentation? **Yes, to an extent.** These methods inspired early deep-learning methods for detection and semantic segmentation. The first **R-CNN** used region proposals from **CPMC** (Constrained Parametric Min-Cuts) — a min-cut segmentation — and later *Selective Search* (hierarchical grouping of Felzenszwalb–Huttenlocher superpixels).

- With the advent of deep learning based methods, do we still need these methods?
- Yes, to an extent. These classical segmentation methods actually inspired early versions of deep learning based methods for object detection and semantic segmentation (we will see this later)
- First R-CNN work (object detection method) used a version of min-cut segmentation method known as CPMC (Constrained Parametric Min Cuts) to generate region proposals for foreground segments

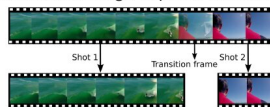


Figure 2.5.13: (slide) CPMC foreground segment used as a region proposal.

2.5.10 Beyond Images: Segmentation of Video

- **Shot boundary detection:** divide a video into shots, each taken from a single continuous camera sequence (detect abrupt changes, e.g. in colour histograms between frames, or gradual transitions).
- **Motion segmentation:** detect and isolate motion (a person running, a car moving) — the Gestalt cue of *common fate*.

- **Shot boundary detection** - a key problem in video segmentation: Divide a video into collection of *shots*, each taken from a single sequence of camera



- Another interesting problem is **motion segmentation**, where the aim is to detect and isolate motion in the video. Examples: a person running, a car moving, etc.

Further Information

- Chapter 15.2, 17.1.4, Forsyth, *Computer Vision: A Modern Approach*

Image Credit: M Gygli

Figure 2.5.14: (slide) Shot boundary detection. Image credit: M. Gygli. Further: Forsyth §15.2, §17.1.4.

Module Summary: §2.5

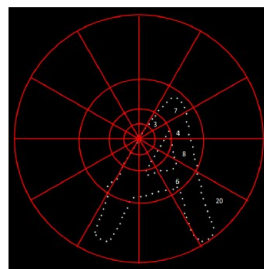
Gestalt cues (proximity, similarity, common fate/region, parallelism, symmetry, continuity, closure) are descriptive, not algorithmic. Watershed floods the gradient image from minima (over-segments; use markers). Felzenszwalb–Huttenlocher: merge if $\text{Dif}(C_1, C_2) \leq \min(\text{Int}(C_i) + k/|C_i|)$, Kruskal-style. Probabilistic aggregation: coupling ratio > 0.2 to seeds, coarse-to-fine. Mean shift: $\mathbf{y} \leftarrow \frac{\sum \mathbf{x}_i G(\mathbf{y} - \mathbf{x}_i)}{\sum G(\mathbf{y} - \mathbf{x}_i)}$ climbs KDE modes ($\nabla f \propto \mathbf{m}$). Normalised cuts: $N_{\text{cut}} = \frac{\text{cut}}{\text{assoc}(A, V)} + \frac{\text{cut}}{\text{assoc}(B, V)}$, NP-complete, relaxed to $(D - W)\mathbf{y} = \lambda D\mathbf{y}$. Classical segmentation seeded R-CNN proposals (CPMC).

Other Feature Spaces

SIFT describes *local appearance* around a keypoint. This module covers other hand-crafted feature spaces: a *shape* descriptor (shape context), a *region* detector (MSER), dense *gradient* descriptors (HOG, PHOG) and a *texture* code (LBP).

2.6.1 Shape Context (Belongie, Malik & Puzicha, 2002)

- **Log-polar representation:** a 2D coordinate system parametrised by the *logarithm of the distance* from the origin and the *angle*.
- For each point sampled on the edge (contour) of a shape, **count the number of other contour points in each log-polar bin** around it.
- Log spacing gives **more precision for nearby points** and **more flexibility for farther points**.
- It is **translation-invariant** (everything is relative to the point) and can be made **scale-invariant** (normalise radial distances by the mean pairwise distance).

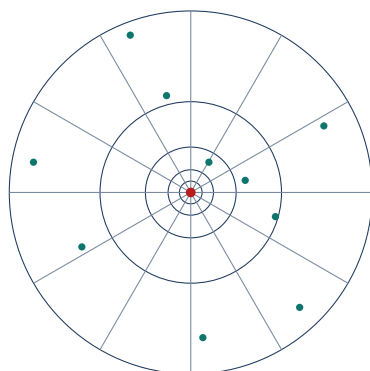


Log-polar binning over 12×5 bins

- **Log-polar representation:** Co-ordinate system in two dimensions parametrized by logarithmic distance from origin and angle.
- For each point taken on edge of shape, count number of points in each log-polar bin.
- More precision for nearby points, more flexibility for farther points.
- It is translation- and scale-invariant

Credit: Derek Hoiem, UIUC

Figure 2.6.1: (slide) Log-polar binning over 12×5 bins centred on a contour point. Credit: D. Hoiem.



5 radial bins: $\log r$ spacing
12 angular bins of 30°
 $\Rightarrow h_i [60]$ counts per point

Figure 2.6.2: Shape context of the red point: a 60-bin histogram of where the other contour points (teal) lie. Inner bins are small (precise), outer bins large (tolerant).

2.6.1.1 Matching with shape contexts

Two points i (shape 1) and j (shape 2) are said to be in **correspondence** if they minimise the χ^2 cost

$$C_{ij} = \frac{1}{2} \sum_{k=1}^K \frac{[h_i(k) - h_j(k)]^2}{h_i(k) + h_j(k)}$$

where h_i, h_j are the (normalised) log-polar histograms of the two points and $K = 12 \times 5 = 60$ is the number of bins.

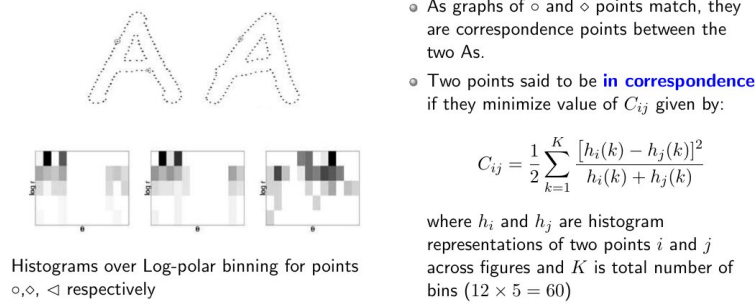


Figure 2.6.3: (slide) Two letter A's: histograms at corresponding points ($\circ, \diamond$) look alike; a non-corresponding point ($\triangle$) does not.

Derivation: Why χ^2 rather than Euclidean

The denominator $h_i(k) + h_j(k)$ normalises each bin's squared difference by its expected magnitude (for Poisson-like counts, variance $\approx$ mean). A difference of 2 in a bin that holds 100 points is insignificant, but in a bin holding 3 it is large. Bins where both are 0 are skipped (0/0). $C_{ij} \in [0, 1]$ for normalised histograms.

- **One-to-one point matching** is done sequentially w.r.t. the correspondence points (Belongie et al. solve it as a bipartite assignment problem, e.g. Hungarian algorithm, minimising $\sum C_{i,\pi(i)}$).
- A total matching **distance below a threshold** implies a shape match.



- One-to-one point matching is done sequentially w.r.t. correspondence point
- Distance less than a threshold implies shape match

Figure 2.6.4: (slide) Sequential one-to-one matching along the contour.

2.6.2 MSER: Maximally Stable Extremal Regions (Matas et al., 2004)

- A **blob/region detector** based on the idea of watershed segmentation.
- Identify regions that **stay nearly the same through a wide range of grey-level thresholds**.
- **Sweep a threshold** of intensity from black to white, performing simple luminance thresholding.
- Extract **connected components** ("extremal regions": every pixel inside is brighter (or darker) than every pixel on the boundary).
- **Region descriptors** (e.g. fitted ellipses, SIFT on the normalised region) serve as features.

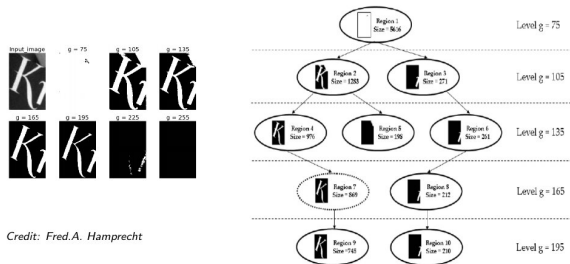
2.6.2.1 Methodology and notation

- Thresholding at grey level g produces regions (blobs). As g decreases, new regions appear or regions from higher g **coalesce** $\Rightarrow$ a **component tree**.
- Regions at level g : $R_1^g, \dots, R_n^g$, $|R_i^g|$ = number of pixels.
- Stability:

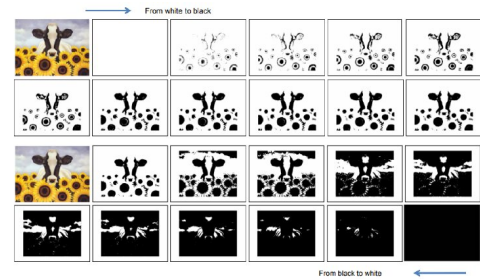
$$\Psi(R_i^g) = \frac{|R_j^{g-\Delta}| - |R_k^{g+\Delta}|}{|R_i^g|}$$

where Δ is a manually chosen intensity buffer and R_j, R_k are the parent and child regions of R_i^g at levels $g - \Delta$ and $g + \Delta$ in the tree.

- MSERs**: regions where Ψ is a local minimum and below a user-defined threshold.



(slide) Thresholding at different grey levels and the resulting component tree. Credit: F. Hamprecht.

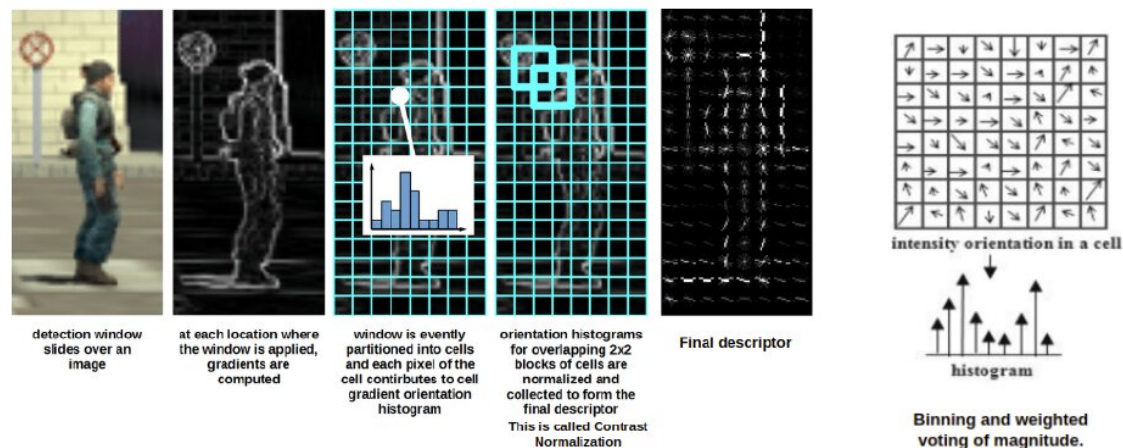


(slide) Sweeping the threshold from dark to bright. Credit: A. Del Bimbo.

Intuition: Why “maximally stable” regions are good features

A region whose area barely changes over a range 2Δ of thresholds has a sharp, high-contrast boundary — it will be found again under lighting changes (monotonic intensity changes do not alter the tree), and because it is defined by connectivity, not by a fixed window, it adapts to the region’s shape: MSERs are **affine-covariant**. They are also very fast (near-linear time with union-find).

2.6.3 Histogram of Oriented Gradients (HOG) (Dalal & Triggs, 2005)



Credit: Michał Olejniczak, Marek Kraft

Figure 2.6.5: (slide) HOG pipeline: input → gradients → cell histograms → block normalisation → descriptor, visualised as oriented strokes. Credit: M. Olejniczak, M. Kraft.

Key Idea: The HOG recipe (pedestrian detection)

1. **Gradients** with the simple centred kernel $[-1, 0, 1]$ (no smoothing works best); magnitude and *unsigned* orientation $\theta \in [0^\circ, 180^\circ)$.
2. **Cells** of 8×8 pixels: a 9-bin orientation histogram (20° each), votes weighted by magnitude (with bilinear interpolation between bins).
3. **Blocks** of 2×2 cells (16×16 px), overlapping with stride 8: concatenate 4 cell histograms (36-D) and **contrast-normalise** (L2-Hys: L2 normalise, clip at 0.2, renormalise).
4. Concatenate all block vectors over the detection window; classify with a linear SVM.

Derivation: HOG dimension for a 64×128 window

Cells: $\frac{64}{8} \times \frac{128}{8} = 8 \times 16$. Blocks (2×2 cells, stride 1 cell): $(8 - 1) \times (16 - 1) = 7 \times 15 = 105$.
Each block: 4 cells $\times$ 9 bins = 36. Total $105 \times 36 = \boxed{3780}$ dimensions.

Intuition: HOG vs. SIFT

Same ingredient (magnitude-weighted orientation histograms over cells, normalised) but applied **densely on a grid** over a whole window, with overlapping normalisation blocks, rather than sparsely at keypoints with rotation/scale normalisation. HOG is the dense template for “what does a pedestrian’s silhouette look like”.

2.6.4 Pyramidal HOG (PHOG) (Bosch, Zisserman & Munoz, 2007)

- Divide the image into $2^l \times 2^l$ cells at each pyramid level l .
- Extract HOG descriptors (same dimension) over each cell.
- The final PHOG descriptor is the **concatenation** of the HOGs at all pyramid levels.

- Captures the **spatial relationship** of oriented gradients better than a single HOG.

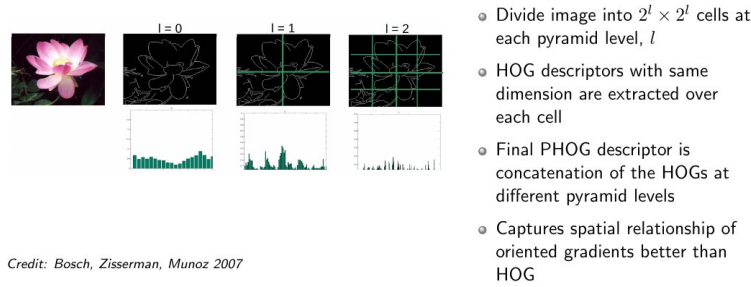


Figure 2.6.6: (slide) PHOG at levels $l = 0, 1, 2$ and the concatenated histogram. Credit: Bosch, Zisserman, Munoz 2007.

Derivation: PHOG length

With K orientation bins and levels $l = 0, \dots, L$: $K \sum_{l=0}^L 4^l = K \frac{4^{L+1}-1}{3}$. E.g. $K = 8, L = 2$: $8 \times (1 + 4 + 16) = 168$; $L = 3$: $8 \times 85 = 680$.

2.6.5 Local Binary Patterns (LBP) (Ojala et al.)

A texture descriptor: encode each pixel by **thresholding its neighbours at the centre value** and reading the bits as a number.

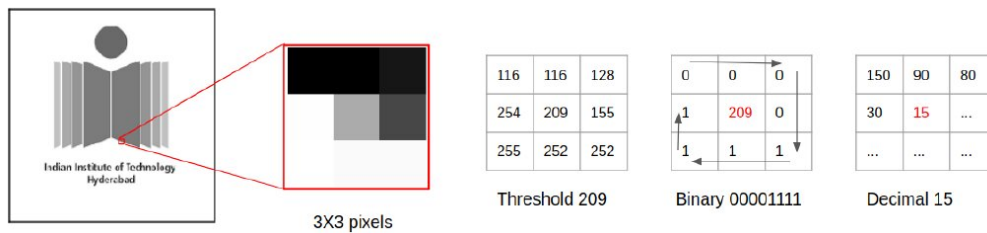


Figure 2.6.7: (slide) LBP on a 3×3 patch: threshold 209 $\rightarrow$ binary 00001111 $\rightarrow$ decimal 15.

Derivation: Reading the slide's example

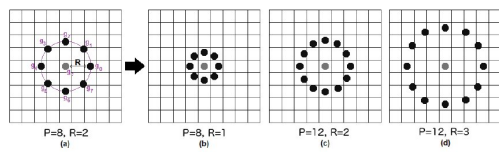
Patch $\begin{bmatrix} 116 & 116 & 128 \\ 254 & \mathbf{209} & 155 \\ 255 & 252 & 252 \end{bmatrix}$, centre $g_c = 209$. Bit = 1 if neighbour ≥ 209 : top row 0, 0, 0; right (155) 0; bottom-right (252) 1; bottom (252) 1; bottom-left (255) 1; left (254) 1. Reading clockwise from the top-left: $00001111_2 = 8 + 4 + 2 + 1 = 15$. In general

$$\text{LBP}_{P,R}(x_c, y_c) = \sum_{p=0}^{P-1} s(g_p - g_c) 2^p, \quad s(z) = \begin{cases} 1 & z \geq 0 \\ 0 & z < 0 \end{cases}$$

with P neighbours on a circle of radius R .

- Different radii R and numbers of neighbours P can be used (e.g. $(P, R) = (4, 1), (8, 1), (12, 1.5), (16, 2)$); **bilinear interpolation** is used when a sample point falls between pixels.
- Invariant to any **monotonic** grey-level change (only the sign of differences matters).

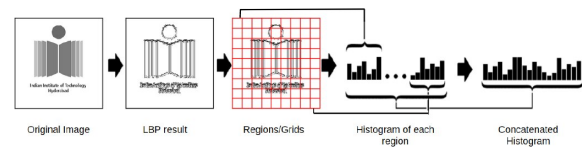
- Image descriptor: compute the LBP image, split into regions, take a **histogram of LBP codes per region** and **concatenate** (classic for face recognition).



- Different radii (R) and neighbours (P) can be considered for per-pixel binary encoding
- Bilinear interpolation done if point shares pixels

Credit: Ojala, Pietikainen, Harwood 2007

(slide) Circular neighbourhoods (P, R). Credit: Ojala, Pietikainen, Harwood.



(slide) Original $\rightarrow$ LBP image $\rightarrow$ regions $\rightarrow$ per-region histograms $\rightarrow$ concatenated histogram.

Key Idea: Extensions (beyond the slides)

Rotation invariance: take the minimum over all circular bit rotations. **Uniform patterns:** codes with at most 2 bit transitions ($0 \rightarrow 1$ or $1 \rightarrow 0$) around the circle (edges, corners, spots); for $P = 8$ there are 58 uniform codes + 1 bin for all others = 59 bins.

2.6.6 Comparison of Feature Detectors

Table 7.1 Overview of feature detectors.

Feature Detector	Corner	Blob	Region	Rotation invariant	Scale invariant	Affine invariant	Repeatability	Localization accuracy	Robustness	Efficiency
Harris	✓			✓			+++	+++	+++	++
Hessian		✓		✓			++	++	++	+
SUSAN	✓			✓			++	++	++	+++
Harris-Laplace	✓	(✓)		✓	✓		+++	+++	++	+
Hessian-Laplace	(✓)	✓		✓	✓		+++	+++	+++	+
DoG	(✓)	✓		✓	✓		++	++	++	++
SURF	(✓)	✓		✓	✓		++	++	++	+++
Harris-Affine	✓	(✓)		✓	✓	✓	+++	+++	++	++
Hessian-Affine	(✓)	✓		✓	✓	✓	+++	+++	+++	++
Salient Regions	(✓)	✓		✓	✓	(✓)	+	+	++	+
Edge-based	✓			✓	✓	✓	+++	+++	+	+
MSER			✓	✓	✓	✓	+++	+++	++	+++
Intensity-based			✓	✓	✓	✓	++	++	++	++
Superpixels			✓	✓	(✓)	(✓)	+	+	+	+

Credit: Tuytelaars, Mikolajczyk 2008

Figure 2.6.8: (slide) Overview of feature detectors (Tuytelaars & Mikolajczyk, 2008): corner/blob/region type, rotation/scale/affine invariance, repeatability, localisation accuracy, robustness, efficiency.

Take-aways from the table: Harris/Hessian are rotation- but not scale-invariant; Harris-/Hessian-Laplace, DoG and SURF add scale invariance; the -Affine variants, edge/intensity-based regions and MSER add affine invariance. MSER combines top repeatability and localisation with top efficiency; SURF is the most efficient of the scale-invariant blob detectors.

2.6.7 ...and Along Came Deep Learning



In the 60s, Marvin Minsky assigned a couple of undergrads to spend the summer programming a computer to use a camera to identify objects in a scene. He figured they'd have the problem solved by the end of the summer.

Over 45 years later, came along deep learning...

Credit: xkcd comics, <https://xkcd.com/1425/>

Figure 2.6.9: (slide) xkcd #1425 “Tasks”: in the 1960s Minsky assigned undergrads to solve vision in one summer; over 45 years later came deep learning. Credit: xkcd.com/1425.

Intuition: Why study hand-crafted features at all?

Every idea in this module reappears inside deep networks: filter banks $\rightarrow$ conv layers; histograms/pooling of responses $\rightarrow$ pooling; normalisation (SIFT/HOG) $\rightarrow$ batch/layer norm; pyramids (PHOG) $\rightarrow$ spatial pyramid pooling and FPNs; LBP-like local comparisons $\rightarrow$ learned local patterns. Knowing them tells you what CNN layers are learning.

Readings: Szeliski §4.1.1–4.1.2; Wikipedia pages on feature detectors, shape context, MSER, HOG.

Module Summary: §2.6

Shape context: 60-bin (12×5) log-polar histogram of contour points; match by χ^2 cost $C_{ij} = \frac{1}{2} \sum \frac{(h_i - h_j)^2}{h_i + h_j}$; translation/scale invariant. MSER: sweep thresholds, component tree, stability $\Psi = \frac{|R^{s-\Delta}| - |R^{s+\Delta}|}{|R^s|}$ minimal; affine-covariant, fast. HOG: $[-1, 0, 1]$ gradients, 8×8 cells $\times 9$ bins, 2×2 block normalisation; $64 \times 128 \Rightarrow 3780$ -D. PHOG: concatenate HOGs over $2^l \times 2^l$ grids. LBP: $\sum_p s(g_p - g_c)2^p$ (example: 15), (P, R) variants, regional histograms; invariant to monotonic intensity changes.

The Human Visual System

Pitfall / Careful: Source

Most of this lecture's slides are based on Lecture 8 of CS455: Computer Vision (Prof. Rajesh Rao, University of Washington, 2009); figures are from there unless otherwise cited.

This lecture explains *why* the operators of Weeks 1–2 (derivatives, centre-surround/LoG, oriented filters, hierarchies) are natural: the brain uses them.

2.7.1 The Visual Pathway

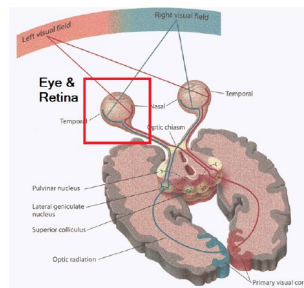


Image Source: Rafael Redondo [6]

Figure 2.7.1: (slide) Eye → optic nerve → optic chiasm → LGN (thalamus) → primary visual cortex (V1) at the back of the brain. Image source: R. Redondo.

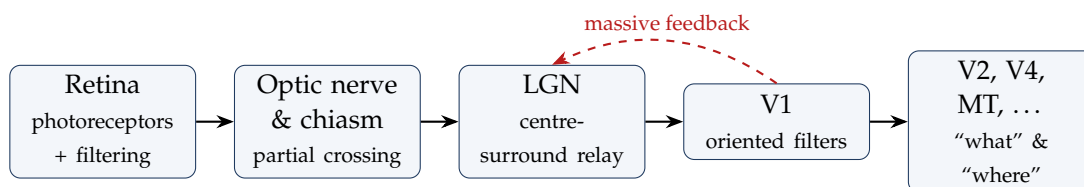


Figure 2.7.2: The main visual pathway.

2.7.2 Light Visible to the Human Eye

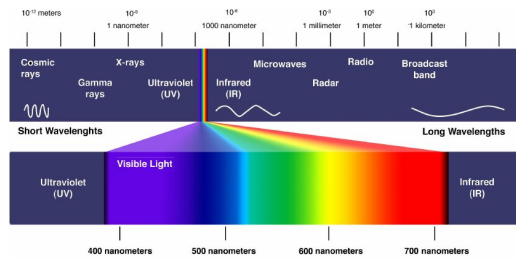
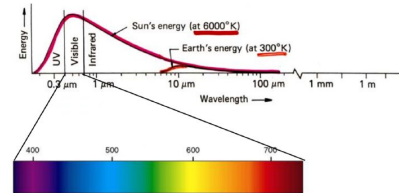


Image Source: www.astronomersgroup.org

(slide) The electromagnetic spectrum; visible light is a narrow band ($\approx 400\text{--}700\text{ nm}$). Source: astronomersgroup.org.

Our vision appears to be optimized for receiving the most abundant spectral radiance our star emits



(slide) The Sun's spectral radiance peaks in the visible band.

Our vision appears to be **optimised for receiving the most abundant spectral radiance our star emits** — the Sun's emission (about 5800 K blackbody) peaks near 500 nm, right in the visible band.

2.7.3 The Retina

The retina = photoreceptors + image filtering. It is not a passive sensor: several layers of neurons (bipolar, horizontal, amacrine, ganglion cells) already process the image before it leaves the eye.

The Retina = Photoreceptors + Image Filtering

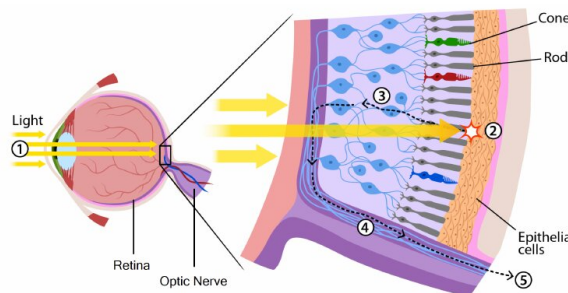


Image Source: mymacularjournal.com

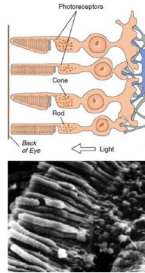
Figure 2.7.3: (slide) Structure of the retina: light passes through the neural layers to reach rods and cones at the back. Image source: mymacularjournal.com.

2.7.3.1 Photoreceptors

	Rods	Cones
Sensitive to	intensity, not colour	colour (three types)
Images	blurred (many rods pool into one ganglion cell)	sharp (concentrated in the fovea)
Light needed	very sensitive; night (scotopic) vision	need many photons; day (photopic) vision
Types	one (peak $\approx 498\text{ nm}$)	S/M/L, maximally sensitive to three different wavelengths

Two Types:

- **Rods:** Sensitive to intensity, but not color; form blurred images
- **Cones:** Color sensitive, form sharp images, require many photons. Three types, each maximally sensitive to one of three different wavelengths.



(slide) Rods and cones.

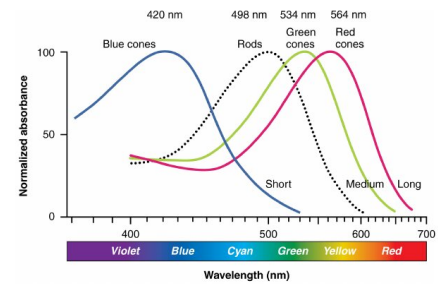


Image Source: Michael C (StackExchange)

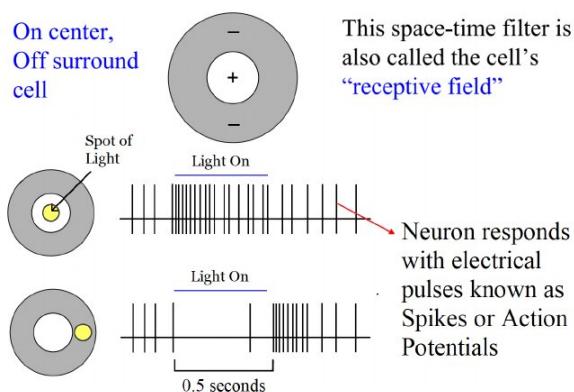
(slide) Normalised absorbance of rods and the three cone types (blue ≈ 420 , green ≈ 534 , red ≈ 564 nm). Source: Michael C (StackExchange).

2.7.4 Image Filtering in Space and Time in the Retina

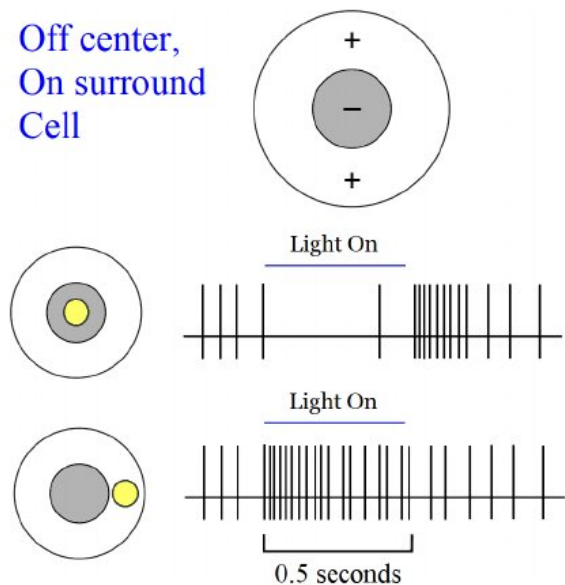
Retinal ganglion cells have **centre-surround receptive fields**:

- **On-centre, off-surround:** fires more when light falls on the centre, less when it falls on the surround.
- **Off-centre, on-surround:** the opposite.

The neuron's response is a train of electrical pulses (*spikes*, action potentials); the space-time filter it implements is called its **receptive field**.



(slide) On-centre/off-surround cell: spot in the centre $\Rightarrow$ many spikes; spot in the surround $\Rightarrow$ fewer.



(slide) Off-centre/on-surround cell: the reverse.

Key Idea: The retina takes spatial and temporal derivatives

Spatially, a centre-surround receptive field is well modelled by a **Laplacian of Gaussian** $\nabla^2 h_\sigma(u, v)$ (or a difference of Gaussians — a narrow excitatory centre minus a wide inhibitory surround). Temporally, the response is a **band-pass** (derivative-like) filter: cells respond to *changes*, not steady light.

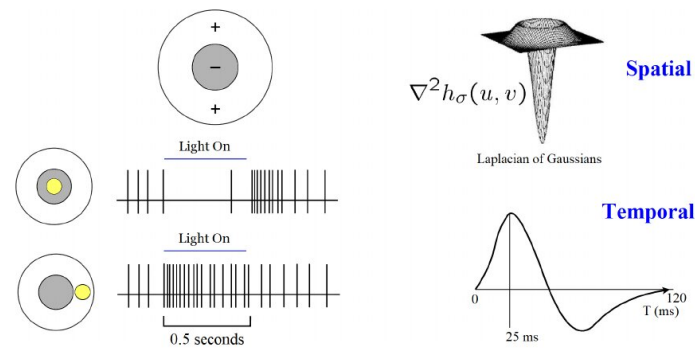


Figure 2.7.4: (slide) Spatial receptive field $\approx$ LoG; temporal impulse response $\approx$ a biphasic (derivative-like) filter over ~ 250 ms.

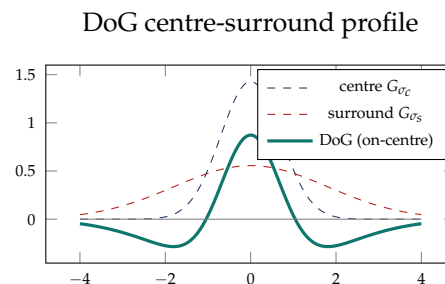


Figure 2.7.5: On-centre/off-surround = difference of a narrow and a wide Gaussian — the same DoG SIFT uses. It sums to zero, so uniform illumination produces no response: the retina transmits *contrast*, not absolute light.

2.7.4.1 Your retinal filters at work

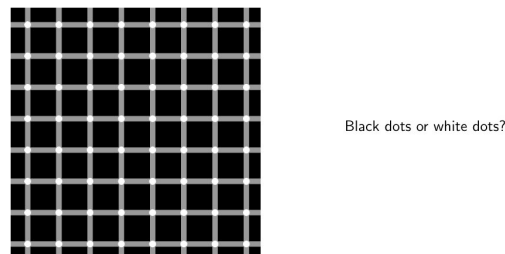


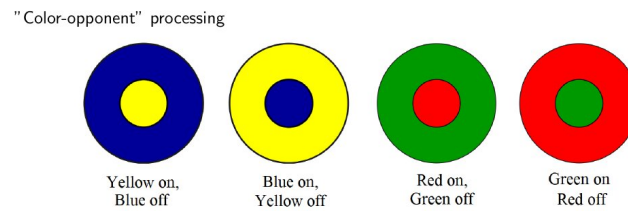
Figure 2.7.6: (slide) “Black dots or white dots?” The scintillating/Hermann grid.

Derivation: Why grey/black dots appear at intersections

Model a ganglion cell as on-centre DoG. On a white *street* between black squares, the surround covers white on two sides and black on two sides. At an *intersection*, the surround covers white on four sides $\Rightarrow$ more inhibition $\Rightarrow$ lower response $\Rightarrow$ the intersection looks darker. Foveal receptive fields are small, which is why the dot disappears where you look directly.

2.7.5 Colour-Opponent Processing

The retina also takes **derivatives in colour space**: “colour-opponent” cells respond to one colour in the centre and its opponent in the surround — *yellow-on/blue-off*, *blue-on/yellow-off*, *red-on/green-off*, *green-on/red-off*.



Visual consequence: **Negative afterimage** - An image is seen after a portion of the retina is exposed to an intense visual stimulus (colors complementary to those of stimulus)

Figure 2.7.7: (slide) Colour-opponent receptive fields and the negative afterimage.

Visual consequence — negative afterimage: after a part of the retina is exposed to an intense stimulus, an image in the **complementary colours** is seen (the adapted channel is fatigued, so its opponent dominates).

Intuition

Opponent channels (luminance, red–green, blue–yellow) decorrelate the highly correlated L/M/S cone signals — the same reason JPEG converts RGB to YCbCr (Week 1).

2.7.6 The Lateral Geniculate Nucleus (LGN)

- LGN receptive fields are similar to retinal ones (**centre-surround, on-off**).
- Once thought to be a simple relay, but it receives **massive feedback from the cortex** (more fibres come down from V1 than up from the retina).

2.7.7 Primary Visual Cortex (V1)

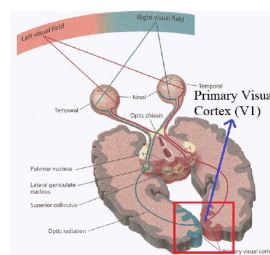


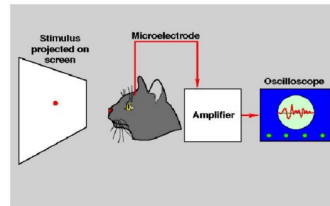
Image Source: Rafael Redondo [6]

Figure 2.7.8: (slide) Location of V1 (striate cortex). Image source: R. Redondo.

2.7.7.1 A tale of two receptive fields

Recall: Hubel and Wiesel first characterised V1 receptive fields by recording from a cat viewing stimuli on a screen (Nobel Prize, 1981).

Recall: David Hubel and Torsten Wiesel were the first to characterize V1 receptive fields by recording from a cat viewing stimuli on a screen



In 1981, they received a Nobel prize in physiology and medicine for their work

Figure 2.7.9: (slide) The Hubel–Wiesel experiment: stimulus on a screen, electrode in V1, amplified spikes.

- **Simple cells:** detect **oriented bars and edges at a specific location** (elongated excitatory/inhibitory sub-regions).
- **Complex cells:** sensitive to **orientation but invariant to position** (within their receptive field).

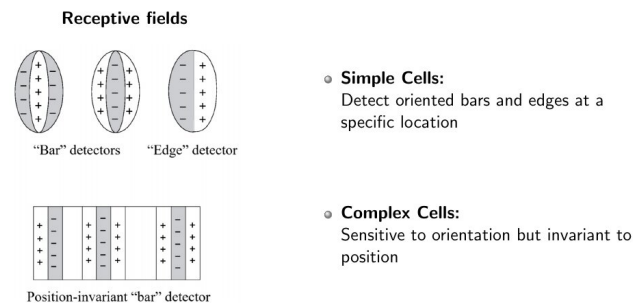


Figure 2.7.10: (slide) Simple-cell “bar” and “edge” detectors built from aligned LGN centre-surround inputs; a complex cell pools simple cells at different positions.

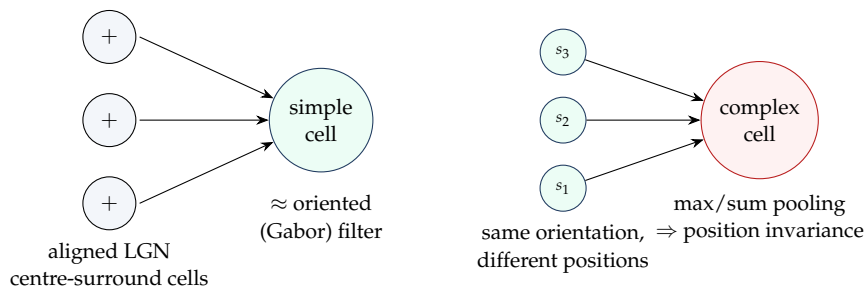
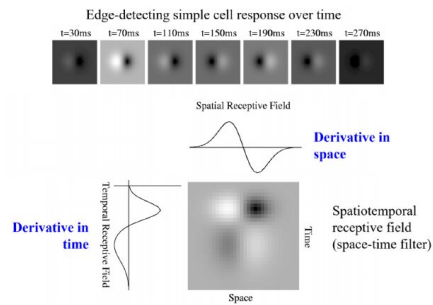


Figure 2.7.11: The Hubel–Wiesel model: simple cell = convolution with an oriented filter; complex cell = pooling over positions. This is precisely conv + pooling in a CNN (and S/C cells in the Neocognitron).

2.7.7.2 Cortical cells compute derivatives

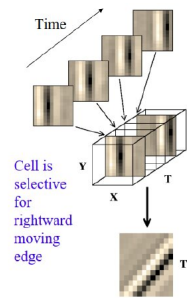
Spatial derivative is orientation-sensitive: a simple cell’s receptive field looks like an oriented derivative (of Gaussian) — a Gabor-like filter. Some cells also take derivatives in **time**: their space-time receptive field is **oriented in $x-t$ space**, which makes them **direction-selective** (they respond to an edge moving one way, not the other).

Spatial derivative is orientation-sensitive



(slide) Spatial and spatio-temporal receptive fields: derivative in space and derivative in time.

Oriented derivative in X-T space!



(slide) Direction selectivity = an oriented derivative in $X-T$ space: the cell prefers rightward-moving edges.

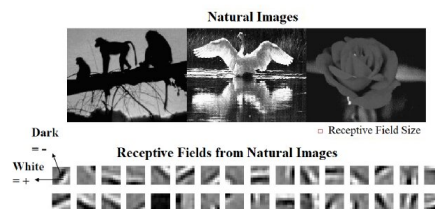
Derivation: Why an $x-t$ oriented filter detects motion

An edge moving at velocity v traces the line $x = vt$ in the $x-t$ plane, i.e. the space-time image is $f(x - vt)$. A filter oriented along that line (a derivative perpendicular to it) responds maximally; the mirror-image filter (tuned to $x = -vt$) does not. Motion detection = orientation detection in space-time.

2.7.7.3 Oriented filters and natural images

Goal: learn *independent* filters whose linear combination best represents natural images (sparse coding). The **optimal set of such filters is oriented and localised** — Gabor-like, like V1 simple cells (Olshausen & Field 1996; Rao & Ballard 1999).

- **Goal:** Learn independent filters whose linear combination best represents natural images
- Optimal set of such filters are oriented and localized to specific regions of image



See Olshausen and Field 1996, Rao and Ballard 1999 for more details

Figure 2.7.12: (slide) Receptive fields learned from natural images: oriented, localised, band-pass.

Key Idea: Sparse coding objective (Olshausen & Field)

Find basis filters ϕ_i and sparse coefficients a_i minimising $\|\mathbf{I} - \sum_i a_i \phi_i\|^2 + \lambda \sum_i |a_i|$. The learned ϕ_i resemble Gabor filters — and so do the first-layer filters learned by CNNs trained on ImageNet: the same statistics of natural images lead to the same solution.

2.7.8 Dorsal and Ventral Pathways

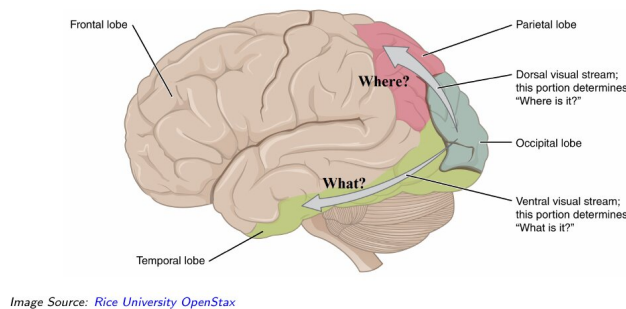


Figure 2.7.13: (slide) Dorsal stream (“where?”, to the parietal lobe) and ventral stream (“what?”, to the temporal lobe). Image source: Rice University OpenStax.

2.7.8.1 Ventral — the “what” (object) pathway

$$V1 \rightarrow V2 \rightarrow V4 \rightarrow TEO \rightarrow TE$$

Cells respond to **more and more complex stimuli** as we go higher: V1 bars/edges → V2 corners, contours, illusory contours → V4 shapes, curvature, colour → TEO/TE (inferotemporal) complex object parts, faces.

Object Pathway: V1 → V2 → V4 → TEO → TE

Cells respond to more and more complex stimuli as we go higher up

Example Receptive Fields

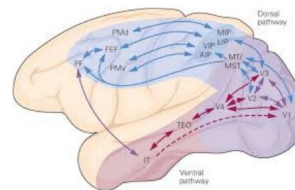
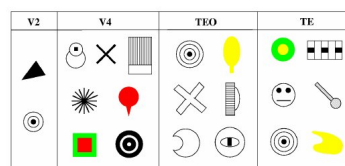


Figure 2.7.14: (slide) Example preferred stimuli at V2, V4, TEO, TE.

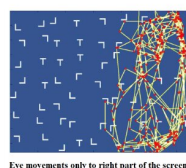
2.7.8.2 Dorsal — the “where” pathway

$$V1 \rightarrow V2 \rightarrow MT \rightarrow MST \rightarrow \text{posterior parietal cortex}$$

Cells respond to **more and more complex forms of motion and spatial relationships** (MT: direction/speed of motion; MST: optic flow patterns such as expansion and rotation). Damage to the right parietal cortex may cause **spatial hemi-neglect**: the patient behaves as if the left part of the visual world does not exist (e.g. draws only the right half of a clock).

V1 → V2 → MT → MST → Posterior Parietal Cortex

- Cells respond to more and more complex forms of motion and spatial relationships
- Damage to right parietal cortex may result in spatial hemi-neglect - patient behaves as if the left part of the visual world doesn't exist



Eye movements only to right part of the screen



Only right side of clock drawn

Image Source: Scholarpedia - Hemi-neglect

Figure 2.7.15: (slide) Hemi-neglect: eye movements only to the right side of the screen; a clock drawn with all numbers on the right. Image source: Scholarpedia.

2.7.9 The Visual Processing Hierarchy

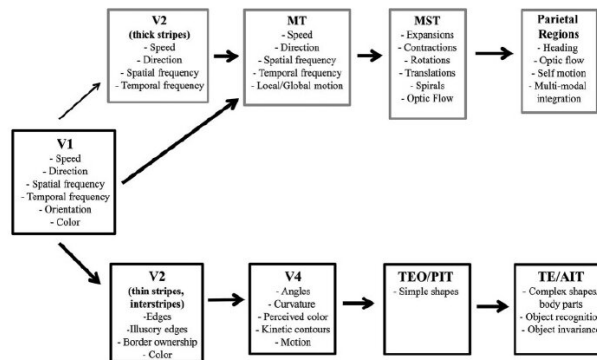


Image Source: Perry, Fallah 2014

Figure 2.7.16: (slide) The hierarchy of visual areas and the features they encode. Image source: Perry & Fallah 2014.

Biology	Classical CV (Weeks 1–2)	Deep learning
Retina/LGN centre-surround Colour-opponent cells	LoG / DoG, band-pass filtering YCbCr/Lab decorrelation	early conv filters, whitening learned colour-opponent first-layer filters
V1 simple cells	Gabor, derivative-of-Gaussian, steerable filters	conv layer 1
V1 complex cells	local pooling (SIFT/HOG his- tograms)	ReLU + max/avg pooling
V2 → V4 → IT hierarchy	Marr's primal → $2\frac{1}{2}$ -D → 3D	deep conv hierarchy (edges → parts → objects)
Dorsal stream (MT/MST)	optical flow, space-time filters	two-stream / 3D-conv video nets
Cortical feedback	coarse-to-fine, top-down priors	attention, recurrent/feedback nets

Readings: Majumder (UCI) lecture notes on visual perception; Fischler & Firschein, *Intelligence: The Eye, the Brain and the Computer*; Hubel, *Eye, Brain, and Vision*; Kaiser, *The Joy of Visual Perception*; Rao, CS455 Lecture 8.

Module Summary: §2.7

Pathway: retina → optic nerve/chiasm → LGN → V1 → higher areas; vision is tuned to the Sun's peak spectrum. Rods (intensity, blurry, sensitive) vs. cones (S/M/L colour, sharp). Retinal ganglion cells: centre-surround (LoG/DoG) spatial filters and derivative-like temporal filters (Hermann-grid dots); colour-opponent channels (negative afterimages). LGN: centre-surround relay with massive cortical feedback. V1: simple cells (oriented bars/edges at a location) and complex cells (orientation, position-invariant); spatial and space-time oriented derivatives (direction selectivity); sparse coding of natural images yields the same oriented filters. Ventral "what" (V1→V2→V4→TEO→TE) and dorsal "where" (V1→V2→MT→MST→parietal) streams; hemi-neglect.

Numerical Problems with Worked Solutions

All numbers below were verified by direct computation. Convention: x to the right, y down (image rows), kernels applied as cross-correlation unless stated.

A.1 Module 2.1: Edges

Problem 1 (Forward vs. central difference)

For $f(x) = x^3$, estimate $f'(2)$ with (a) the forward difference, (b) the central difference ($h = 1$). The true value is 12. Explain the errors.

Solution

(a) $f(3) - f(2) = 27 - 8 = 19$ (error 7). (b) $\frac{f(3) - f(1)}{2} = \frac{27 - 1}{2} = 13$ (error 1). Forward error $\approx \frac{h}{2} f''(2) = \frac{1}{2} \cdot 12 = 6$ (+higher-order terms = 7); central error $= \frac{h^2}{6} f''' = \frac{6}{6} = 1$ exactly. Central differences are $O(h^2)$ accurate.

Problem 2 (Sobel gradient)

Patch $P = \begin{bmatrix} 10 & 10 & 40 \\ 10 & 20 & 60 \\ 20 & 50 & 80 \end{bmatrix}$. Using unnormalised Sobel $M_x = [-1, 0, 1; -2, 0, 2; -1, 0, 1]$ and $M_y = [-1, -2, -1; 0, 0, 0; 1, 2, 1]$ (y down), compute G_x , G_y , $\|\nabla\|$ and θ .

Solution

$G_x = (40 - 10) + 2(60 - 10) + (80 - 20) = 30 + 100 + 60 = 190$.
 $G_y = (20 + 2 \cdot 50 + 80) - (10 + 2 \cdot 10 + 40) = 200 - 70 = 130$.
 $\|\nabla\| = \sqrt{190^2 + 130^2} = \sqrt{53000} \approx 230.2$; $\theta = \text{atan2}(130, 190) \approx 34.4^\circ$ (below the x -axis in image coordinates: intensity increases towards the bottom-right). The edge runs perpendicular, at $\approx 34.4^\circ + 90^\circ$. (Divide by 8 for the normalised Sobel.)

Problem 3 (Noise amplification)

Image noise is i.i.d. with variance σ^2 . Find the noise variance at the output of (a) $[-1, 1]$, (b) the unnormalised Sobel M_x , (c) $\frac{1}{8} M_x$.

Solution

Output variance $= \sigma^2 \sum H^2$. (a) $1 + 1 = 2 \Rightarrow 2\sigma^2$. (b) $1 + 4 + 1 + 1 + 4 + 1 = 12 \Rightarrow 12\sigma^2$. (c) $12/64 = 0.1875 \Rightarrow 0.1875\sigma^2$. To compare fairly, divide by the squared response to a unit ramp (signal gain): for $\frac{1}{8} M_x$ the gain is 1 per pixel, for $[-1, 1]$ also 1 — so normalised Sobel has

$\frac{0.1875}{2} \approx \frac{1}{10.7}$ the noise power of the plain difference: smoothing in y and central differencing both help.

Problem 4 (Hysteresis)

A 1D gradient-magnitude profile along a contour is $[2, 9, 5, 4, 6, 3, 5, 8, 2, 5, 4]$ with $T_L = 4$, $T_H = 7$. Which pixels are edges?

Solution

Strong (> 7): indices 1 (9), 7 (8). Weak ($4 \leq \cdot \leq 7$): 2 (5), 3 (4), 4 (6), 6 (5), 9 (5), 10 (4). Non-edges (< 4): 0, 5, 8. Grow from strong pixels through weak neighbours: from 1 $\rightarrow$ 2 $\rightarrow$ 3 $\rightarrow$ 4, stop at 5 (value 3). From 7 $\rightarrow$ 6 (left), stop at 8 on the right. Edges: $\{1, 2, 3, 4, 6, 7\}$. Pixels 9, 10 are weak but not connected to a strong pixel $\Rightarrow$ rejected.

Problem 5 (LoG zero-crossings and blob size)

(a) Where is $\nabla^2 G_\sigma$ zero in 2D? (b) Which σ best detects a dark blob of radius 10 px? (c) For the 1D second derivative of a Gaussian, where are the zero-crossings?

Solution

(a) $\nabla^2 G = \frac{r^2 - 2\sigma^2}{\sigma^4} G = 0 \Rightarrow r = \sqrt{2}\sigma$. (b) Match the zero-crossing circle to the blob: $\sigma = 10/\sqrt{2} \approx 7.07$ px. (c) $g''(x) = \frac{x^2 - \sigma^2}{\sigma^4} g(x) = 0 \Rightarrow x = \pm\sigma$.

A.2 Module 2.2: Corners, Blobs and Lines

Problem 6 (Discrete Laplacian)

Apply $[0, 1, 0; 1, -4, 1; 0, 1, 0]$ at the centre of the patch P of Problem 2.

Solution

$10 + 10 + 60 + 50 - 4 \cdot 20 = 130 - 80 = 50$. Positive $\Rightarrow$ the centre is darker than the average of its 4-neighbours (a local “valley” in intensity).

Problem 7 (Structure tensor from gradients)

In a window with uniform weights, the gradients at five pixels are $(I_x, I_y) = (1, 0), (2, 1), (0, 2), (-1, 1), (1, -1)$. Build A , find its eigenvalues and the Harris response with $\kappa = 0.05$.

Solution

$\sum I_x^2 = 1 + 4 + 0 + 1 + 1 = 7$, $\sum I_y^2 = 0 + 1 + 4 + 1 + 1 = 7$, $\sum I_x I_y = 0 + 2 + 0 - 1 - 1 = 0$.
 $A = \begin{bmatrix} 7 & 0 \\ 0 & 7 \end{bmatrix}$, $\lambda_1 = \lambda_2 = 7$. $M_c = 49 - 0.05 \cdot 14^2 = 49 - 9.8 = 39.2 > 0$: gradients in all directions — corner-like.

Problem 8 (Classify with Harris)

Classify windows with $A = \begin{bmatrix} a & b \\ b & c \end{bmatrix}$ using $\kappa = 0.04$: (i) (100, 0, 100), (ii) (200, 0, 2), (iii) (3, 1, 2), (iv) (80, 40, 60). Also give the harmonic-mean score $\det / \text{tr}$.

Solution

	det	tr	λ 's	M_c	det / tr	class
(i)	10000	200	100, 100	$10000 - 1600 = 8400$	50	corner
(ii)	400	202	200, 2	$400 - 1632.2 = -1232.2$	1.98	edge
(iii)	5	5	3.62, 1.38	$5 - 1 = 4$	1.0	flat
(iv)	3200	140	111.2, 28.8	$3200 - 784 = 2416$	22.9	corner (weaker)

Eigenvalues of (iv): $\frac{140 \pm \sqrt{140^2 - 4 \cdot 3200}}{2} = \frac{140 \pm 82.5}{2}$.

Problem 9 (Harris under intensity scaling)

An image is multiplied by 2 and offset by 30. What happens to A and M_c ? If a corner had $M_c = 500$ and the threshold is 1000, is it detected before/after?

Solution

Offset: no effect. Scaling by $a = 2$: $I_x, I_y \times 2 \Rightarrow A \times 4 \Rightarrow \det \times 16, \text{tr}^2 \times 16 \Rightarrow M_c \times 16 = 8000$. Not detected before ($500 < 1000$), detected after ($8000 > 1000$). Local maxima locations are unchanged; only thresholding changes — “partially invariant”.

Problem 10 (Straightness of an edgelet)

Edge pixels: (0, 0), (1, 1.1), (2, 1.9), (3, 3.05), (4, 4). Compute the scatter matrix, orientation and straightness.

Solution

Means $\mu_x = 2, \mu_y = 2.01$. $M \approx \begin{bmatrix} 10 & 9.95 \\ 9.95 & 9.92 \end{bmatrix}$, eigenvalues ≈ 19.91 and 0.011 . Top eigenvector $\approx (0.708, 0.706) \Rightarrow \theta \approx 44.9^\circ$. Straightness $\lambda_{\min} / \lambda_{\max} \approx 0.00055$ — essentially a perfect line at 45° .

A.3 Module 2.3: Scale Space, Pyramids, Filters**Problem 11 (Characteristic scale)**

A keypoint's scale-normalised LoG peaks at $\sigma = 3$ in image 1 and at $\sigma = 7.5$ at the corresponding point in image 2. What is the scale factor? What window radius should be used for a descriptor that spans 3σ in each?

Solution

$s = 7.5/3 = 2.5$ (image 2 is zoomed 2.5 $\times$). Radii 9 px and 22.5 px: both cover the same physical content.

Problem 12 (Pyramid sizes and coarse-to-fine cost)

(a) A 512×512 image: list the Gaussian pyramid sizes down to 32×32 and the total pixels.
 (b) Registration with displacement up to ± 32 px: compare exhaustive search at full resolution with a 4-level coarse-to-fine search (± 1 refinement per level).

Solution

(a) $512^2, 256^2, 128^2, 64^2, 32^2$: $262144 + 65536 + 16384 + 4096 + 1024 = 349184 \approx 1.33 \times$ the original. (b) Full: $(2 \cdot 32 + 1)^2 = 4225$ candidates/pixel. Coarse (level 4, $\times \frac{1}{16}$): $\pm 2 \Rightarrow 25$; plus 4 refinements of $3^2 = 9$: $25 + 36 = 61$ — about $69 \times$ fewer.

Problem 13 (Laplacian pyramid reconstruction)

1D signal $G_0 = [4, 6, 8, 6]$. Use “smooth+downsample” = average pairs, and “upsample+smooth” = repeat each sample. Compute G_1 , L_0 , and reconstruct.

Solution

$G_1 = [5, 7]$. Upsample: $[5, 5, 7, 7]$. $L_0 = G_0 - [5, 5, 7, 7] = [-1, 1, 1, -1]$; $L_1 = G_1$. Reconstruct: $\text{up}(G_1) + L_0 = [5, 5, 7, 7] + [-1, 1, 1, -1] = [4, 6, 8, 6] \checkmark$. L_0 holds the fine detail and is zero-mean.

Problem 14 (Steerable filter)

At a pixel, $I * G_x = 3$ and $I * G_y = 4$. What is the response of $G_1^{30^\circ}$? Which orientation gives the maximum and what is it?

Solution

$3 \cos 30^\circ + 4 \sin 30^\circ = 2.598 + 2 = 4.598$. Max at $\theta^* = \text{atan2}(4, 3) = 53.13^\circ$ with value $\sqrt{9 + 16} = 5$.

Problem 15 (Gabor bank design)

A bank should cover orientations every 22.5° and 3 wavelengths $\lambda = 4, 8, 16$ px. How many filters? What is θ for filter 5 of the orientation set? Why only $[0, 180^\circ)$?

Solution

$180/22.5 = 8$ orientations $\times 3 = 24$ filters. Filter 5 (starting at 0°): $4 \times 22.5 = 90^\circ$. A grating at θ and at $\theta + 180^\circ$ is the same set of stripes (only the phase/sign flips), so $[0, 180^\circ)$ suffices for even filters (or when using magnitudes of complex Gabors).

A.4 Module 2.4: SIFT

Problem 16 (Octave parameters)

With $s = 3$ intervals per octave and $\sigma_0 = 1.6$: find k , the σ 's of the Gaussian images in the first octave, the number of DoG images, and the constant in $\text{DoG} \approx (k-1)\sigma^2 \nabla^2 G$.

Solution

$k = 2^{1/3} \approx 1.26$. Need $s + 3 = 6$ Gaussian images: 1.6, 2.02, 2.54, 3.2, 4.03, 5.08. DoGs: 5 ($= s + 2$); extrema searched in the middle 3. $k - 1 \approx 0.26$ — a constant across scales, so it does not affect extremum locations.

Problem 17 (Sub-pixel localisation, 1D)

DoG samples along x : $D(-1) = 0.8$, $D(0) = 1.0$, $D(1) = 0.9$. Find the refined extremum and its value. Does it pass the 0.03 contrast test?

Solution

$D' \approx \frac{0.9-0.8}{2} = 0.05$, $D'' \approx 0.9 - 2(1.0) + 0.8 = -0.3$. $\hat{x} = -D'/D'' = 0.05/0.3 = 0.1667$ (less than 0.5 $\Rightarrow$ stay at this sample). $D(\hat{x}) = D(0) + \frac{1}{2}D'\hat{x} = 1.0 + 0.5 \cdot 0.05 \cdot 0.1667 = 1.0042$. $|D| > 0.03 \Rightarrow$ kept.

Problem 18 (Edge rejection)

With $r = 10$ (threshold 12.1), test: (a) $D_{xx} = 10$, $D_{yy} = 8$, $D_{xy} = 1$; (b) $D_{xx} = 20$, $D_{yy} = 1$, $D_{xy} = 0.5$; (c) $D_{xx} = 5$, $D_{yy} = -3$, $D_{xy} = 0$.

Solution

(a) $\text{Tr} = 18$, $\text{Det} = 80 - 1 = 79$, ratio $324/79 = 4.10 < 12.1$: **keep**. (b) $\text{Tr} = 21$, $\text{Det} = 20 - 0.25 = 19.75$, ratio $441/19.75 = 22.3 > 12.1$: **reject** (edge-like). (c) $\text{Det} = -15 < 0$: curvatures of opposite sign (saddle): **reject**.

Problem 19 (Orientation peak interpolation)

36-bin histogram (10° bins, bin i centred at $10i + 5^\circ$). The peak is bin 5 with value 50; bins 4 and 6 hold 30 and 40. Estimate the orientation. Would a secondary peak of 42 elsewhere spawn a new keypoint?

Solution

$\delta = \frac{h_{-1}-h_{+1}}{2(h_{-1}-2h_0+h_{+1})} = \frac{30-40}{2(30-100+40)} = \frac{-10}{-60} = 0.1667$ bins. Orientation = $55^\circ + 1.67^\circ \approx 56.7^\circ$. $42 \geq 0.8 \times 50 = 40 \Rightarrow$ yes, a second keypoint with that orientation.

Problem 20 (Descriptor normalisation)

A (toy, 5-D) raw descriptor is $[0.9, 0.3, 0.2, 0.1, 0.1]$. Apply SIFT's normalise-clip(0.2)-renormalise.

Solution

Norm = $\sqrt{0.81 + 0.09 + 0.04 + 0.01 + 0.01} = \sqrt{0.96} = 0.980$. Unit vector $\approx [0.919, 0.306, 0.204, 0.102, 0.102]$. Clip at 0.2: $[0.2, 0.2, 0.2, 0.102, 0.102]$, norm ≈ 0.375 . Renormalise: $[0.533, 0.533, 0.533, 0.272, 0.272]$. The dominant bin no longer dominates.

Problem 21 (Descriptor size and matching)

(a) Derive the SIFT length. (b) A GLOH variant with 3 rings of 8 sectors plus a centre and 16 orientations — raw length? (c) Nearest/second-nearest distances are 0.30/0.35 and 0.30/0.50. Which match passes Lowe's ratio test (0.8)?

Solution

(a) 4×4 cells $\times 8$ bins = 128. (b) $(3 \cdot 8 + 1) \times 16 = 25 \times 16 = 400$ (standard GLOH: $17 \times 16 = 272$). (c) $0.30/0.35 = 0.857 > 0.8$: reject (ambiguous); $0.30/0.50 = 0.6 < 0.8$: accept.

Problem 22 (Integral image)

$I = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 \end{bmatrix}$. Compute the integral image and the sum over rows 1–2, columns 1–3 (0-indexed) with 4 lookups.

Solution

$S = \begin{bmatrix} 1 & 3 & 6 & 10 \\ 6 & 14 & 24 & 36 \\ 15 & 33 & 54 & 78 \\ 28 & 60 & 96 & 136 \end{bmatrix}$. Sum = $S(2,3) - S(0,3) - S(2,0) + S(0,0) = 78 - 10 - 15 + 1 = 54$.
Direct: $6 + 7 + 8 + 10 + 11 + 12 = 54 \checkmark$.

A.5 Module 2.5: Segmentation**Problem 23 (Felzenszwalb–Huttenlocher in 1D)**

Pixels in a row: $[10, 12, 11, 30, 31, 29, 60]$, $w = |I_i - I_{i+1}|$, $\tau(C) = k/|C|$ with $k = 5$. Run the algorithm.

Solution

Edges sorted: $(1,2) : 1$, $(3,4) : 1$, $(0,1) : 2$, $(4,5) : 2$, $(2,3) : 19$, $(5,6) : 31$.

- $(1,2)$, $w = 1$: $\text{MInt} = \min(0 + 5, 0 + 5) = 5 \Rightarrow \text{merge}$; $\text{Int} = 1$. Same for $(3,4)$.
- $(0,1)$, $w = 2$: $\min(0 + 5, 1 + 5/2) = 3.5 \Rightarrow \text{merge } \{0, 1, 2\}$, $\text{Int} = 2$. Same for $(4,5) \Rightarrow \{3, 4, 5\}$.
- $(2,3)$, $w = 19$: $\min(2 + 5/3, 2 + 5/3) = 3.67 \Rightarrow \text{no merge}$. $(5,6)$, $w = 31$: $\min(3.67, 5) \Rightarrow \text{no merge}$.

Segments: $\{10, 12, 11\}, \{30, 31, 29\}, \{60\}$.

Problem 24 (Mean shift, flat kernel)

1D samples $\{1, 2, 2.5, 3, 7, 8, 8.5\}$, flat window of radius $h = 2$. Start at $y_0 = 1$ and at $y_0 = 7$.

Solution

$y_0 = 1$: points in $[-1, 3]$: $\{1, 2, 2.5, 3\}$, mean 2.125. From 2.125: window $[0.125, 4.125]$ has the same points $\Rightarrow$ converged at 2.125. $y_0 = 7$: window $[5, 9]$: $\{7, 8, 8.5\}$, mean 7.833; window $[5.83, 9.83]$: same $\Rightarrow$ converged at 7.833. Two modes $\Rightarrow$ two segments.

Problem 25 (Normalised cut)

Four nodes a, b, c, d with $w_{ab} = 3, w_{cd} = 3, w_{bc} = 1, w_{ad} = 0.5$. Compare $\text{Ncut}(\{a, b\}, \{c, d\})$ with $\text{Ncut}(\{a\}, \{b, c, d\})$ and the plain cuts.

Solution

Degrees: $d_a = 3.5, d_b = 4, d_c = 4, d_d = 3.5$. $\{a, b\}|\{c, d\}$: cut = $1 + 0.5 = 1.5$; assoc = 7.5 each; $\text{Ncut} = \frac{1.5}{7.5} \times 2 = 0.4$. $\{a\}|\{b, c, d\}$: cut = $3 + 0.5 = 3.5$; $\frac{3.5}{3.5} + \frac{3.5}{11.5} = 1.304$. The balanced split wins under both criteria here; but note that a pixel with very weak links would have a small plain cut while its Ncut stays near 1 — that is the point of normalising.

A.6 Module 2.6: Other Feature Spaces

Problem 26 (Shape context cost)

Two 5-bin histograms (toy) $h_i = [4, 2, 0, 1, 3], h_j = [3, 3, 1, 0, 3]$. Normalise and compute C_{ij} .

Solution

$\hat{h}_i = [0.4, 0.2, 0, 0.1, 0.3], \hat{h}_j = [0.3, 0.3, 0.1, 0, 0.3]$. Terms $\frac{(a-b)^2}{a+b}$: $\frac{0.01}{0.7} = 0.0143, \frac{0.01}{0.5} = 0.02, \frac{0.01}{0.1} = 0.1, \frac{0.01}{0.1} = 0.1, 0$. Sum 0.2343; $C_{ij} = 0.117$. Note that the two sparse bins dominate — exactly the χ^2 behaviour.

Problem 27 (MSER stability)

Region R^g has 1100 px; its parent at $g - \Delta$ has 1200 and child at $g + \Delta$ has 1000. Another region has 1000 px with parent 1500 and child 600. Which is more stable?

Solution

$\Psi_1 = \frac{1200-1000}{1100} = 0.182; \Psi_2 = \frac{1500-600}{1000} = 0.9$. Region 1 (smaller Ψ) is far more stable $\Rightarrow$ MSER candidate.

Problem 28 (HOG and PHOG dimensions)

(a) HOG for 64×64 and 128×128 windows (8-px cells, 2×2 blocks, stride 1 cell, 9 bins). (b) PHOG with 12 bins and $L = 2$.

Solution

(a) 64×64 : 8×8 cells, $7 \times 7 = 49$ blocks $\times 36 = 1764$. 128×128 : 16×16 cells, $15 \times 15 = 225$ blocks $\times 36 = 8100$. (b) $12 \times (1 + 4 + 16) = 252$.

Problem 29 (LBP code)

Compute the LBP (clockwise from top-left, bit = 1 if $\geq$ centre) of $\begin{bmatrix} 50 & 60 & 70 \\ 40 & \mathbf{55} & 80 \\ 30 & 55 & 90 \end{bmatrix}$. Is it a “uniform” pattern?

Solution

Sequence TL, T, TR, R, BR, B, BL, L = 50, 60, 70, 80, 90, 55, 30, 40 $\rightarrow$ bits 0, 1, 1, 1, 1, 1, 0, 0 $\Rightarrow 01111100_2 = 124$. Circular transitions: $0 \rightarrow 1$ and $1 \rightarrow 0$ (and $0 \rightarrow 0$ wrap) = 2 $\Rightarrow$ uniform (an edge-like pattern). Adding 20 to every pixel gives the same code.

A.7 Module 2.7: HVS**Problem 30 (DoG centre-surround response)**

A 1D on-centre cell is modelled as $h = G_{\sigma_c=1} - G_{\sigma_s=3}$ (both unit-area). What is its response to (a) uniform light of intensity 100, (b) a thin bright line at its centre of total energy 1 on a dark background?

Solution

(a) $\int h = 1 - 1 = 0 \Rightarrow$ response 0: uniform illumination is ignored (only contrast is signalled).
 (b) Response = $h(0) = \frac{1}{\sqrt{2\pi}} \left(1 - \frac{1}{3}\right) = 0.399 \times 0.667 \approx 0.266 > 0$ (excitation).

PRACTICE B

Multiple-Choice Self-Test

Answer each question before reading the explanation underneath.

B.1 Module 2.1: Edge Detection

Q1. Which is *not* one of the causes of edges listed in the lecture?

- (a) Surface normal discontinuity
- (b) Depth discontinuity
- (c) Illumination discontinuity
- (d) Sensor bit-depth discontinuity

Answer: (d). The four are surface normal, depth, surface colour/appearance, and illumination discontinuities.

Q2. Along a scanline, an ideal step edge corresponds to:

- (a) A zero of the first derivative
- (b) An extremum of the first derivative
- (c) A maximum of the intensity
- (d) A minimum of the second derivative only

Answer: (b). Edges are extrema of f' (and zero-crossings of f'').

Q3. The central difference $\frac{f(x+1)-f(x-1)}{2}$ has truncation error:

- (a) $O(h^2)$
- (b) $O(1)$
- (c) $O(h)$
- (d) $O(h^3)$

Answer: (a). Even Taylor terms cancel; the forward difference is only $O(h)$.

Q4. The Sobel x -kernel is best described as:

- (a) Central difference in x , binomial smoothing in y
- (b) A 2×2 diagonal difference
- (c) A Laplacian
- (d) Box smoothing in x , difference in y

Answer: (a). $[1, 2, 1]^T [-1, 0, 1]$; Roberts is the 2×2 diagonal one.

Q5. The gradient direction at an edge pixel is:

- (a) Undefined
- (b) Along the edge
- (c) Always horizontal
- (d) Perpendicular to the edge, pointing to increasing intensity

Answer: (d). $\theta = \text{atan2}(f_y, f_x)$ is the edge *normal*.

Q6. Why smooth before differentiating?

- (a) Differentiation amplifies high-frequency noise
- (b) To make the image smaller
- (c) Smoothing is required for colour images only
- (d) To increase contrast

Answer: (a). $\frac{d}{dx} \leftrightarrow j\omega$ boosts high frequencies where noise lives.

Q7. The derivative theorem of convolution states:

- (a) $\frac{d}{dx}(f * g) = 0$
- (b) $\frac{d}{dx}(f * g) = f * \frac{dg}{dx}$
- (c) $\frac{d}{dx}(f * g) = f + g$
- (d) $\frac{d}{dx}(f * g) = \frac{df}{dx} \cdot \frac{dg}{dx}$

Answer: (b). Saves an operation: convolve once with the derivative of Gaussian.

Q8. With a second-derivative (LoG) operator, edges are located at:

- (a) Saddle points
- (b) Maxima
- (c) Minima
- (d) Zero-crossings

Answer: (d). f'' crosses zero where f' peaks.

Q9. Which is NOT one of Canny's criteria?

- (a) Good localisation
- (b) Single response
- (c) Rotation invariance
- (d) Good detection

Answer: (c). Canny's three criteria are detection, localisation, single response.

Q10. Non-maximum suppression keeps a pixel if its gradient magnitude is maximal:

- (a) Across all scales
- (b) Along the edge direction
- (c) In a 5×5 window
- (d) Along the gradient direction

Answer: (d). Compare with neighbours p, r along the gradient (edge normal), possibly interpolated.

Q11. In hysteresis thresholding, a pixel with magnitude between Low and High is an edge iff:

- (a) It is a local maximum
- (b) It is above the image mean
- (c) It is connected to a strong edge pixel via pixels above Low
- (d) Its gradient is horizontal

Answer: (c). High starts edges, Low continues them.

Q12. Increasing σ in Canny:

- (a) Detects larger-scale edges and suppresses fine detail
- (b) Makes edges thicker only
- (c) Has no effect
- (d) Detects finer edges

Answer: (a). Large $\sigma \Rightarrow$ large-scale edges; small $\sigma \Rightarrow$ fine edges.

Q13. Holistically-Nested Edge Detection (HED) is characterised by:

- (a) Hand-crafted texture gradients
- (b) Pointwise mutual information
- (c) Random forests over patches
- (d) Deeply supervised side outputs of a CNN fused across scales

Answer: (d). Structured forests = Dollár; PMI = Isola; Pb = Martin et al.

Q14. Structured Forests predict boundaries:

- (a) At patch level (16×16), averaging over-lapping predictions
- (b) Only at corners
- (c) Using a CNN
- (d) Per pixel independently

Answer: (a). Nearby pixel predictions should not be independent.

B.2 Module 2.2: Blobs and Corners

Q15. In the Canny-to-lines algorithm, straightness of an edgelet is measured by:

- (a) Its mean gradient
- (b) The ratio of eigenvalues of its second-moment matrix
- (c) Its colour
- (d) Its length

Answer: (b). Straightness = λ_2/λ_1 (small = straight); orientation from the top eigenvector.

Q16. The discrete 4-neighbour Laplacian kernel is:

- (a) $[-1, 0, 1; -2, 0, 2; -1, 0, 1]$
- (b) $[1, 2, 1; 2, 4, 2; 1, 2, 1]$
- (c) $[1, 1, 1; 1, 1, 1; 1, 1, 1]$
- (d) $[0, 1, 0; 1, -4, 1; 0, 1, 0]$

Answer: (d). Sum of the two second differences.

Q17. The LoG is a good blob detector because it is:

- (a) Directional
- (b) Circularly symmetric
- (c) Separable
- (d) Non-linear

Answer: (b). Its shape resembles a blob (centre + ring), with no preferred orientation.

Q18. The aperture problem says that for a straight edge seen through a small window:

- (a) All motion can be measured (b) No motion can be measured
 (c) Only motion perpendicular to the edge can be measured (d) Only motion along the edge can be measured

Answer: (c). Displacement along the edge leaves the window unchanged.

Q19. The autocorrelation $E_{AC}(\Delta \mathbf{u})$ is approximated as:

- (a) $\Delta \mathbf{u}^\top A \Delta \mathbf{u}$ (b) $\det(A) \Delta \mathbf{u}$
 (c) $\|A\|$ (d) $\nabla I \cdot \Delta \mathbf{u}$

Answer: (a). First-order Taylor expansion gives a quadratic form in the structure matrix A .

Q20. The entries of the structure matrix A are:

- (a) Weighted sums of $I_x^2, I_x I_y, I_y^2$ (b) Eigenvalues of the Hessian
 (c) I^2 only (d) I_{xx}, I_{xy}, I_{yy}

Answer: (a). $A = w * [I_x^2, I_x I_y; I_x I_y, I_y^2]$. The Hessian (second derivatives) is used in SIFT.

Q21. An edge region has eigenvalues:

- (a) Both small (b) One large, one small
 (c) Both negative (d) Both large and similar

Answer: (b). $\lambda_1 \gg \lambda_2$.

Q22. The Harris response is:

- (a) $\text{tr}(A) - \kappa \det(A)^2$ (b) $\det(A) - \kappa \text{tr}(A)^2$
 (c) $\lambda_{\max} / \lambda_{\min}$ (d) $\det(A) + \text{tr}(A)$

Answer: (b). $M_c = \lambda_1 \lambda_2 - \kappa (\lambda_1 + \lambda_2)^2$, $\kappa \approx 0.04-0.06$.

Q23. For an edge-like window, the Harris response is typically:

- (a) Negative (b) Exactly zero
 (c) Large positive (d) Infinite

Answer: (a). $\det \approx 0$ while tr^2 is large.

Q24. Brown et al.'s harmonic-mean corner score is:

- (a) $\frac{\lambda_0 \lambda_1}{\lambda_0 + \lambda_1}$ (b) $\frac{\lambda_0 + \lambda_1}{2}$
 (c) $\sqrt{\lambda_0 \lambda_1}$ (d) $\lambda_0 - \alpha \lambda_1$

Answer: (a). $\det / \text{tr}$; smoother when $\lambda_0 \approx \lambda_1$.

Q25. The Harris detector is:

- (a) Rotation-invariant but not scale-invariant (b) Scale-invariant but not rotation-invariant
 (c) Neither (d) Scale- and rotation-invariant

Answer: (a). Eigenvalues are unchanged by rotation; a fixed window breaks scale invariance.

Q26. Under $I \rightarrow aI + b$, the Harris response:

- (a) Is multiplied by a^4 (b) Is multiplied by a^2
 (c) Shifts by b (d) Is unchanged

Answer: (a). $A \rightarrow a^2 A$; $\det$ and tr^2 scale by a^4 ; b has no effect.

Q27. The trace of a matrix equals:

- (a) The determinant (b) The product of eigenvalues
 (c) The largest eigenvalue (d) The sum of eigenvalues

Answer: (d). And the determinant is the product.

B.3 Module 2.3: Scale Space and Filter Banks

Q28. Automatic scale selection finds:

- (a) The largest possible scale (b) The scale where the image is sharpest
 (c) A local maximum of a function over both position and scale (d) The scale with most keypoints

Answer: (c). The characteristic scale follows the image zoom.

Q29. The LoG must be multiplied by σ^2 for scale selection because:

- (a) Otherwise responses decay with scale and are not comparable across scales (b) It makes it separable
 (c) It makes the kernel sum to 1 (d) It removes noise

Answer: (a). Scale normalisation makes $\sigma^2 \nabla^2 L$ equal at corresponding (x, σ) and $(sx, s\sigma)$.

Q30. A Laplacian pyramid level is computed as:

- (a) $L_i = G_i - \text{Smooth}(\text{Upsample}(G_{i+1}))$ (b) $L_i = G_i + \text{Up}(G_{i+1})$
 (c) $L_i = \text{Downsample}(G_i)$ (d) $L_i = \nabla^2 G_i$ only

Answer: (a). Band-pass details; exactly invertible.

Q31. Coarse-to-fine registration is efficient because:

- (a) Coarse images have more pixels (b) Large displacements become small at coarse levels, and each finer level only refines a small range
 (c) It avoids interpolation (d) It uses colour

Answer: (b). Displacement d at level L is $d/2^L$.

Q32. Block, checkerboard and striped patterns each with 50% white pixels show that:

- (a) Colour is necessary (b) Histograms fully describe texture
 (c) All textures are the same (d) Texture depends on spatial arrangement, not just the histogram

Answer: (d). Identical histograms, very different textures.

Q33. A filter bank is:

- (a) A single Gabor filter (b) A set of low-pass filters
 (c) An array of band-pass filters splitting the signal into sub-bands (d) A database of images

Answer: (c). Keep (squared/absolute) responses per filter.

Q34. A Gabor filter is:

- (a) A median filter (b) A sinusoid modulated by a Gaussian envelope
 (c) A box filter with a sinusoid (d) A difference of Gaussians

Answer: (b). Band-pass, oriented, V1-like.

Q35. In the 2D Gabor, γ controls:

- (a) Orientation (b) Wavelength
 (c) Phase (d) Spatial aspect ratio (ellipticity)

Answer: (d). λ wavelength, θ orientation, ψ phase, σ envelope width.

Q36. A bank of 16 Gabor filters with 11.25° steps covers:

- (a) 270° (b) 360°
 (c) 90° (d) 180°

Answer: (d). $16 \times 11.25^\circ = 180^\circ$.

Q37. For a steerable first-derivative-of-Gaussian, $G_1^\theta =$

- (a) $G_1^0 \cos \theta + G_1^{90} \sin \theta$ (b) $G_1^0 \cos^2 \theta$
 (c) $G_1^0 \sin \theta + G_1^{90} \cos \theta$ (d) $G_1^0 + G_1^{90}$

Answer: (a). Directional derivative $= \nabla G \cdot (\cos \theta, \sin \theta)$.

B.4 Module 2.4: SIFT and Variants

Q38. SIFT detects candidate keypoints as extrema of:

- (a) Harris response
- (b) Gradient magnitude
- (c) Difference of Gaussians over space and scale
- (d) Colour histograms

Answer: (c). $\text{DoG} \approx (k-1)\sigma^2 \nabla^2 G$.

Q39. How many neighbours does SIFT compare in the extremum test?

- (a) 8
- (b) 18
- (c) 26
- (d) 9

Answer: (c). $8 + 9 + 9$.

Q40. With $s = 3$ intervals per octave, $k =$

- (a) 3
- (b) $2^{1/3}$
- (c) $\sqrt{2}$
- (d) 1.6

Answer: (b). $k = 2^{1/s}$; 1.6 is Lowe's σ_0 .

Q41. The sub-pixel extremum offset in SIFT is:

- (a) Always 0.5
- (b) $\hat{\mathbf{s}} = D / \nabla D$
- (c) $\hat{\mathbf{s}} = -(\partial^2 D / \partial \mathbf{s}^2)^{-1} \partial D / \partial \mathbf{s}$
- (d) $\hat{\mathbf{s}} = \nabla D$

Answer: (c). Newton step from the quadratic Taylor model.

Q42. SIFT discards a keypoint as low-contrast if $|D(\hat{\mathbf{s}})| <$

- (a) 0.03
- (b) 0.3
- (c) 10
- (d) 0.8

Answer: (a). For images normalised to $[0, 1]$.

Q43. SIFT's edge-rejection test uses:

- (a) The orientation histogram
- (b) The Hessian of D and the ratio Tr^2 / Det
- (c) The descriptor norm
- (d) The structure matrix of gradients

Answer: (b). $\frac{\text{Tr}^2}{\text{Det}} = \frac{(r+1)^2}{r}$, reject if above threshold ($r = 10 \Rightarrow 12.1$).

Q44. $\frac{(r+1)^2}{r}$ is minimised when:

- (a) $r = 0$
- (b) $r \rightarrow \infty$
- (c) $r = 1$
- (d) $r = 10$

Answer: (c). Equal curvatures; value 4.

Q45. The SIFT orientation histogram has:

- (a) 36 bins
- (b) 128 bins
- (c) 16 bins
- (d) 8 bins

Answer: (a). 36 bins of 10° ; weighted by magnitude and Gaussian with $\sigma = 1.5 \times \text{scale}$.

Q46. An extra keypoint is created at the same location if a secondary orientation peak is within:

- (a) Never
- (b) 50% of the max
- (c) 80% of the max
- (d) 95% of the max

Answer: (c). About 15% of keypoints get multiple orientations.

Q47. The SIFT descriptor dimension is 128 because:

- (a) 4×4 cells $\times$ 8 orientation bins
- (b) 8×16 pixels
- (c) 2×64 Haar responses
- (d) 16×8 scales

Answer: (a). From a 16×16 window.

Q48. After unit-normalising, SIFT clips descriptor values at:

- (a) 0.2
- (b) 1.0
- (c) 0.8
- (d) 0.02

Answer: (a). Then renormalises, reducing the influence of large gradient magnitudes.

Q49. Unit-normalising the SIFT descriptor provides invariance to:

- (a) Scale
- (b) Rotation
- (c) Additive brightness change
- (d) Multiplicative contrast/gain change

Answer: (d). Additive changes are already removed by using gradients.

Q50. SURF is faster than SIFT mainly because it uses:

- (a) Fewer octaves
- (b) GPU only
- (c) Box filters with integral images and Haar wavelets
- (d) Larger descriptors

Answer: (c). $\sim 3\times$ faster; weaker on illumination/viewpoint invariance.

Q51. GLOH differs from SIFT by:

- (a) Using colour
- (b) Using Harris corners
- (c) Using no orientation
- (d) Using log-polar spatial bins (17) and 16 orientations, reduced by PCA to 128

Answer: (d). 272-D $\rightarrow$ 128-D.

Q52. MOPS achieves rotation invariance by:

- (a) Ignoring rotation
- (b) Rotating the patch to its dominant gradient orientation
- (c) Histogramming orientations
- (d) Using a circular window only

Answer: (b). Canonical orientation, then compare normalised patches.

B.5 Module 2.5: Segmentation

Q53. Gestalt theory's main limitation for computer vision is that it:

- (a) Only works on video
- (b) Ignores grouping
- (c) Is mathematically precise but slow
- (d) Is descriptive and insufficiently defined for direct algorithms

Answer: (d). A loose set of rules.

Q54. Watershed segmentation is usually applied to:

- (a) The histogram
- (b) The Fourier magnitude
- (c) The raw image
- (d) The gradient-magnitude image

Answer: (d). Boundaries are ridges of gradient magnitude.

Q55. The typical problem with watershed is:

- (a) It is NP-complete
- (b) It needs training data
- (c) Under-segmentation
- (d) Over-segmentation due to noise

Answer: (d). Hence marker-controlled/interactive use.

Q56. In Felzenszwalb–Huttenlocher, $\text{Int}(C)$ is:

- (a) The largest edge weight in the MST of C
- (b) The smallest edge between regions
- (c) The number of pixels in C
- (d) The mean edge weight in C

Answer: (a). Dif is the smallest edge *between* regions.

Q57. Two regions are merged in Felzenszwalb–Huttenlocher when:

- (a) $\text{Dif} > \text{MInt}$
- (b) $\text{Int}(C_1) = \text{Int}(C_2)$
- (c) $\text{Dif} \leq \text{MInt}$
- (d) Always

Answer: (c). The predicate D is false $\Rightarrow$ merge.

Q58. The mean-shift update $\mathbf{y}_{k+1}$ equals:

- (a) The gradient of the density
- (b) The nearest sample
- (c) The global mean
- (d) The kernel-weighted mean of samples around $\mathbf{y}_k$

Answer: (d). $\frac{\sum \mathbf{x}_i G(\mathbf{y}_k - \mathbf{x}_i)}{\sum G(\mathbf{y}_k - \mathbf{x}_i)}$.

Q59. The key parameter of mean-shift segmentation is:

- (a) Kernel bandwidth h
- (b) Threshold 0.2
- (c) Number of octaves
- (d) Number of clusters k

Answer: (a). k is found automatically as the number of modes.

Q60. Normalised cut improves on min-cut by:

- (a) Being solvable exactly in linear time
- (b) Ignoring edge weights
- (c) Using Euclidean distance
- (d) Normalising the cut by each side's total association, avoiding tiny isolated segments

Answer: (d). Exact Ncut is NP-complete; approximated by a generalised eigenproblem.

Q61. The first R-CNN used region proposals derived from:

- (a) Mean shift
- (b) Watershed
- (c) SLIC
- (d) CPMC (a min-cut segmentation)

Answer: (d). Constrained Parametric Min-Cuts.

B.6 Module 2.6: Other Feature Spaces

Q62. Shape context uses how many bins?

- (a) 36
- (b) 8×8
- (c) $12 \times 5 = 60$
- (d) $4 \times 4 \times 8$

Answer: (c). 12 angular $\times$ 5 log-radial.

Q63. Why log-polar bins in shape context?

- (a) Faster computation
- (b) More precision for nearby points, more tolerance for far points
- (c) Colour invariance
- (d) Rotation invariance

Answer: (b). Near points matter more for local shape.

Q64. MSERs are regions that:

- (a) Have maximal area
- (b) Have the highest gradient
- (c) Are always circular
- (d) Change little in area over a range of thresholds

Answer: (d). Small Ψ .

Q65. The standard HOG descriptor for a 64×128 window has dimension:

- (a) 128
- (b) 8100
- (c) 3780
- (d) 1764

Answer: (c). 105 blocks $\times$ 36.

Q66. PHOG improves on HOG by:

- (a) Removing normalisation
- (b) Concatenating HOGs over a spatial pyramid of $2^l \times 2^l$ cells
- (c) Using colour
- (d) Using log-polar bins

Answer: (b). Captures spatial layout of gradients.

Q67. The LBP code is invariant to:

- (a) Blur
- (b) Scale
- (c) Any monotonic grey-level transformation
- (d) Rotation (basic version)

Answer: (c). Only signs of differences matter. Rotation invariance needs the min-over-rotations variant.

B.7 Module 2.7: Human Visual System

Q68. Rods are:

- (a) Three types
- (b) Colour sensitive and sharp
- (c) Only in the fovea
- (d) Intensity sensitive, not colour, forming blurred images

Answer: (d). Cones: colour, sharp, need many photons, three types.

Q69. Retinal ganglion cell receptive fields are best modelled by:

- (a) Box filters
- (b) Haar wavelets
- (c) Centre-surround (LoG/DoG) filters
- (d) Gabor filters

Answer: (c). On-centre/off-surround and vice versa.

Q70. Negative afterimages are explained by:

- (a) V4 damage
- (b) Rod saturation
- (c) Optic chiasm crossing
- (d) Colour-opponent processing

Answer: (d). Fatigued channel $\Rightarrow$ its opponent dominates.

Q71. V1 complex cells are:

- (a) Position-specific bar detectors
- (b) Colour-opponent
- (c) Motion-insensitive centre-surround cells
- (d) Orientation-sensitive but position-invariant

Answer: (d). Simple cells are position-specific.

Q72. Direction-selective cortical cells can be modelled as:

- (a) Colour histograms
- (b) Isotropic LoG
- (c) Median filters
- (d) Oriented filters in $x-t$ space

Answer: (d). Motion = orientation in space-time.

Q73. The ventral (“what”) pathway is:

- (a) V1 $\rightarrow$ LGN
- (b) Retina $\rightarrow$ LGN $\rightarrow$ MT
- (c) V1 $\rightarrow$ V2 $\rightarrow$ V4 $\rightarrow$ TEO $\rightarrow$ TE
- (d) V1 $\rightarrow$ V2 $\rightarrow$ MT $\rightarrow$ MST $\rightarrow$ parietal

Answer: (c). The other is the dorsal “where” pathway.

Q74. Spatial hemi-neglect results from damage to:

- (a) The left retina
- (b) V4
- (c) Right parietal cortex
- (d) LGN

Answer: (c). Patient ignores the left visual world.

Q75. Sparse coding of natural images yields basis filters that are:

- (a) Random noise
- (b) Oriented and localised, Gabor-like
- (c) Global sinusoids
- (d) Uniform boxes

Answer: (b). Like V1 simple cells and CNN first layers (Olshausen & Field).

Formula Sheet

Edges (2.1)

- Forward diff. $f(x+1) - f(x) \quad (O(h))$; central $\frac{f(x+1) - f(x-1)}{2} \quad (O(h^2))$.
- Prewitt $[1, 1, 1]^T [-1, 0, 1]$; Sobel $[1, 2, 1]^T [-1, 0, 1]$; Roberts $[0, 1; -1, 0]$.
- $\nabla f = [f_x, f_y]$; $\theta = \text{atan2}(f_y, f_x)$ (edge normal); $\|\nabla f\| = \sqrt{f_x^2 + f_y^2}$.
- $\frac{d}{dx}(f * g) = f * \frac{dg}{dx}$; $\partial_x G = -\frac{x}{\sigma^2} G$.
- $\nabla^2 G = \frac{x^2 + y^2 - 2\sigma^2}{\sigma^4} G$; zero at $r = \sqrt{2}\sigma$.
- Canny: DoG filter $\rightarrow \|\nabla\|, \theta \rightarrow \text{NMS} \rightarrow \text{hysteresis}$ ($T_H : T_L \approx 2-3$).
- Pb: $\chi^2 = \frac{1}{2} \sum \frac{(g_i - h_i)^2}{g_i + h_i}$; PMI = $\log \frac{P(A,B)^{\rho}}{P(A)P(B)}$.

Blobs & corners (2.2)

- Scatter $M = \sum (\mathbf{p} - \boldsymbol{\mu})(\mathbf{p} - \boldsymbol{\mu})^T$; straightness $\lambda_{\min} / \lambda_{\max}$.
- Laplacian kernel $[0, 1, 0; 1, -4, 1; 0, 1, 0]$.
- $E_{AC}(\Delta \mathbf{u}) = \sum w [I(\mathbf{p} + \Delta \mathbf{u}) - I(\mathbf{p})]^2 \approx \Delta \mathbf{u}^T A \Delta \mathbf{u}$.
- $A = w * \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}$.
- Harris $M_c = \det A - \kappa \text{tr}^2 A$, $\kappa \approx 0.04-0.06$.
- Shi-Tomasi $\lambda_{\min}$; Triggs $\lambda_0 - \alpha \lambda_1$; Brown $\frac{\det}{\text{tr}}$.
- $\text{tr} = \sum \lambda_i$, $\det = \prod \lambda_i$.
- $I \rightarrow aI + b$: $M_c \rightarrow a^4 M_c$.

Scale space (2.3)

- Scale-normalised LoG $\sigma^2 \nabla^2 G * I$; blob $r = \sqrt{2}\sigma^*$.
- $G_{i+1} = \downarrow \text{Smooth}(G_i)$; $L_i = G_i - \text{Smooth}(\uparrow G_{i+1})$.
- Pyramid pixels $\leq \frac{4}{3} N^2$.
- Gabor: $e^{-\frac{x'^2 + y'^2}{2\sigma^2}} e^{i(2\pi x' / \lambda + \psi)}$,
 $x' = x \cos \theta + y \sin \theta$, $y' = -x \sin \theta + y \cos \theta$.
- Steerable: $G_1^\theta = G_1^0 \cos \theta + G_1^{90} \sin \theta$.

SIFT (2.4)

- $D = G(k\sigma) * I - G(\sigma) * I \approx (k-1)\sigma^2 \nabla^2 G * I$; $\partial_\sigma G = \sigma \nabla^2 G$.
- $k = 2^{1/s}$ ($s = 3$), $\sigma_0 = 1.6$, $s + 3$ Gaussians/octave; 26 neighbours.
- $\hat{\mathbf{s}} = -H_D^{-1} \nabla D$; $D(\hat{\mathbf{s}}) = D + \frac{1}{2} \nabla D^T \hat{\mathbf{s}}$; reject $|D| < 0.03$.
- Edge: $\frac{\text{Tr}^2}{\text{Det}} = \frac{(r+1)^2}{r} > \frac{121}{10} \Rightarrow \text{reject}$.
- $m = \sqrt{(\Delta_x \hat{I})^2 + (\Delta_y \hat{I})^2}$, $\theta = \tan^{-1} \frac{\Delta_y \hat{I}}{\Delta_x \hat{I}}$; 36 bins, Gaussian 1.5σ , peaks $\geq 80\%$.
- Descriptor $16 \times 16 \rightarrow 4 \times 4 \times 8 = 128$; normalise, clip 0.2, renormalise.
- Ratio test $d_1 / d_2 < 0.8$.
- Integral image box sum: $S_{11} - S_{01} - S_{10} + S_{00}$.
- GLOH $17 \times 16 = 272 \rightarrow 128$ (PCA).

Segmentation (2.5)

- $\text{Int}(C) = \max_{\text{MST}(C)} w$; $\text{Dif} = \min_{C_1-C_2} w$; $\text{MInt} = \min(\text{Int}(C_i) + \tau(C_i))$, $\tau = k/|C|$; merge iff $\text{Dif} \leq \text{MInt}$.
- Alpert coupling $\frac{\sum_{j \in C} p_{ij}}{\sum_{j \in V} p_{ij}} > 0.2$.
- KDE $f = \sum k(\|\mathbf{x} - \mathbf{x}_i\|^2 / h^2)$; $\nabla f \propto [\sum G] \mathbf{m}$;
 $\mathbf{y} \leftarrow \frac{\sum \mathbf{x}_i G(\mathbf{y} - \mathbf{x}_i)}{\sum G(\mathbf{y} - \mathbf{x}_i)}$.
- $\text{cut} = \sum_{A,B} w_{ij}$; $\text{Ncut} = \frac{\text{cut}}{\text{assoc}(A,V)} + \frac{\text{cut}}{\text{assoc}(B,V)}$;
 $(D - W)\mathbf{y} = \lambda D\mathbf{y}$.

Other features (2.6)

- Shape context 12×5 bins; $C_{ij} = \frac{1}{2} \sum \frac{(h_i - h_j)^2}{h_i + h_j}$.
- MSER $\Psi = \frac{|R^{\delta-\Delta}| - |R^{\delta+\Delta}|}{|R^{\delta}|}$.
- HOG: 8×8 cells, 9 bins, 2×2 blocks; $64 \times 128 \rightarrow 3780$.
- PHOG length $K \sum_l 4^l$.
- LBP = $\sum_p s(g_p - g_c) 2^p$.
- HVS (2.7)
 - Retina/LGN: centre-surround $\approx$ DoG/LoG; V1: oriented (Gabor) simple cells, position-invariant complex cells.
 - What: $V1 \rightarrow V2 \rightarrow V4 \rightarrow \text{TEO} \rightarrow \text{TE}$; Where: $V1 \rightarrow V2 \rightarrow \text{MT} \rightarrow \text{MST} \rightarrow \text{parietal}$.